

Babel

Code

Version 26.11.133272
2026/08/24

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	LaTeX: babel.sty (start)	8
3.3	base	9
3.4	key=value options and other general option	10
3.5	Post-process some options	11
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	15
4.2	Errors	23
4.3	More on selection	24
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	27
4.8	Shorthands	30
4.9	Language attributes	38
4.10	Support for saving and redefining macros	40
4.11	French spacing	41
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	48
4.15	Making glyphs available	49
4.15.1	Quotation marks	50
4.15.2	Letters	51
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	54
4.17	Load engine specific macros	54
4.18	Creating and modifying languages	54
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	67
4.21	French spacing (again)	73
4.22	Handle language system	74
4.23	Numerals	75
4.24	Casing	77
4.25	Getting info	77
4.26	BCP 47 related commands	78
5	Adjusting the Babel behavior	80
5.1	Cross referencing macros	82
5.2	Layout	85
5.3	Marks	86
5.4	Other packages	87
5.4.1	ifthen	87
5.4.2	varioref	88
5.4.3	hhline	88
5.5	Encoding and fonts	89
5.6	Basic bidi support	90
5.7	Local Language Configuration	94
5.8	Language options	94

6	The kernel of Babel	98
7	Error messages	98
8	Loading hyphenation patterns	102
9	luatex + xetex: common stuff	106
10	Hooks for XeTeX and LuaTeX	110
10.1	XeTeX	110
10.2	Support for interchar	111
10.3	Layout	113
10.4	8-bit TeX	115
10.5	LuaTeX	115
10.6	Southeast Asian scripts	122
10.7	CJK line breaking	124
10.8	Arabic justification	126
10.9	Common stuff	130
10.10	Automatic fonts and ids switching	130
10.11	Bidi	137
10.12	Layout	140
10.13	Lua: transforms	149
10.14	Lua: Auto bidi with basic and basic-r	158
11	Data for CJK	170
12	The ‘nil’ language	170
13	Calendars	171
13.1	Islamic	172
13.2	Hebrew	173
13.3	Persian	177
13.4	Coptic and Ethiopic	178
13.5	Julian	178
13.6	Buddhist	179
14	Support for Plain T_EX (plain.def)	180
14.1	Not renaming hyphen.tex	180
14.2	Emulating some L ^A T _E X features	181
14.3	General tools	181
14.4	Encoding related macros	185
15	Acknowledgements	188

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `<<name=value>>`, or with a series of lines between `<<*name>>` and `<</name>>`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<version=26.11.133272>>
2 <<date=2026/08/24>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<*Basic macros>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8     {\def#1{#2}}%
9     {\expandafter\def\expandafter#1\expandafter{#1#2}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@cs#1{\csname bbl@#1\endcsname}
17 \def\bbl@c#1{\csname bbl@#1\@language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
```

```

20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \< for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35   \let\<\noexpand
36   \let\<\bbl@exp@en
37   \let\[\bbl@exp@ue
38   \edef\bbl@exp@aux{\endgroup#1}%
39   \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{}%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@ \expandafter{\the\toks@##1}%
120     \else
121       \toks@ \expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize\undefined \else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax{%
129     \def\bbl@tempa{#1}%
130     \def\bbl@tempb{#2}%
131     \def\bbl@tempe{#3}}
132   \def\bbl@sreplace#1#2#3{%
133     \begingroup
134       \expandafter\bbl@parsedef\meaning#1\relax
135       \def\bbl@tempc{#2}%
136       \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137       \def\bbl@tempd{#3}%
138       \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139       \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140       \ifin@
141         \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142         \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143           \\makeatletter % "internal" macros with @ are assumed
144           \\scantokens{%
145             \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146             \noexpand\noexpand}%
147           \catcode64=\the\catcode64\relax}% Restore @
148       \else
149         \let\bbl@tempc\empty % Not \relax
150       \fi
151       \bbl@exp{% For the 'uplevel' assignments
152     \endgroup
153     \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bbl@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bbl@tempb{#1}%
158   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
159   \protected@edef\bbl@tempc{#2}%
160   \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
161   \ifx\bbl@tempb\bbl@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bbl@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bbl@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bbl@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bbl@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal `\let's` made by `\MakeUppercase` and `\MakeLowercase` between things like `\oe` and `\OE`.

```

184 \def\bbl@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bbl@afterelse\expandafter\MakeUppercase
190   \else
191     \bbl@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with `#`'s. Used to deal with `alph`, `Alph` and frenchspacing when there are already changes (with `\babel@save`).

```

196 \def\bbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
197   \toks@\expandafter\expandafter\expandafter{%
198     \csname extras\language\endcsname}%
199   \bbl@exp{\in{#1}{\the\toks@}}%
200   \ifin@\else
201     \@temptokena{#2}%
202     \edef\bbl@tempc{\the\@temptokena\the\toks@}%
203     \toks@\expandafter{\bbl@tempc#3}%
204     \expandafter\edef\csname extras\language\endcsname{\the\toks@}%
205   \fi}
206 <</Basic macros>>

```

Some files identify themselves with a $\LaTeX$ macro. The following code is placed before them to define (and then undefine) if not in $\LaTeX$.


```

207 <<*Make sure ProvidesFile is defined>> ≡
208 \ifx\ProvidesFile\undefined
209 \def\ProvidesFile#1[#2 #3 #4]{%
210 \wlog{File: #1 #4 #3 <#2>}%
211 \let\ProvidesFile\undefined}
212 \fi
213 <</Make sure ProvidesFile is defined>>

```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```

214 <<*Define core switching macros>> ≡
215 \ifx\language\undefined
216 \csname newcount\endcsname\language
217 \fi
218 <</Define core switching macros>>

```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```

219 <<*Define core switching macros>> ≡
220 \countdef\last@language=19
221 \def\addlanguage{\csname newlanguage\endcsname}
222 <</Define core switching macros>>

```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```

223 <*package>
224 \NeedsTeXFormat{LaTeX2e}
225 \ProvidesPackage{babel}%
226 [<@date> v<@version>]
227 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX

```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

228 \ifpackagewith{babel}{debug}
229 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
230 \let\bbl@debug\@firstofone
231 \ifx\directlua\undefined\else
232 \directlua{
233 Babel = Babel or {}
234 Babel.debug = true }%
235 \input{babel-debug.tex}%
236 \fi}
237 {\providecommand\bbl@trace[1]{}%
238 \let\bbl@debug\@gobble
239 \ifx\directlua\undefined\else
240 \directlua{
241 Babel = Babel or {}
242 Babel.debug = false }%
243 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

244 \def\bbl@error#1{% Implicit #2#3#4
245   \begingroup
246     \catcode`\=0 \catcode`\==12 \catcode`\`=12
247     \input errbabel.def
248   \endgroup
249   \bbl@error{#1}}
250 \def\bbl@warning#1{%
251   \begingroup
252     \def\{\MessageBreak}%
253     \PackageWarning{babel}{#1}%
254   \endgroup}
255 \def\bbl@infowarn#1{%
256   \begingroup
257     \def\{\MessageBreak}%
258     \PackageNote{babel}{#1}%
259   \endgroup}
260 \def\bbl@info#1{%
261   \begingroup
262     \def\{\MessageBreak}%
263     \PackageInfo{babel}{#1}%
264   \endgroup}

```

Many of the following options don't do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

265 <@Basic macros@>
266 \ifpackagewith{babel}{silent}
267   {\let\bbl@info@gobble
268    \let\bbl@infowarn@gobble
269    \let\bbl@warning@gobble}
270 {}
271 %
272 \def\AfterBabelLanguage#1{%
273   \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

274 \ifx\bbl@languages\undefined\else
275   \begingroup
276     \catcode`\^^I=12
277     \@ifpackagewith{babel}{showlanguages}{%
278       \begingroup
279         \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%
280         \wlog{<*languages>}%
281         \bbl@languages
282         \wlog{</languages>}%
283       \endgroup}{%
284     \endgroup
285     \def\bbl@elt#1#2#3#4{%
286       \ifnum#2=\z@
287         \gdef\bbl@nulllanguage{#1}%
288         \def\bbl@elt##1##2##3##4{%
289           \fi}%
290     \bbl@languages
291 \fi

```

3.3. base

The first 'real' option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\LaTeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with luatex) hyphenation patterns, even if we are not interested in the rest of babel.

```

292 \bbl@trace{Defining option 'base'}
293 \@ifpackagewith{babel}{base}{%
294   \let\bbl@onlyswitch\@empty
295   \let\bbl@provide@locale\relax
296   \input babel.def
297   \let\bbl@onlyswitch\@undefined
298   \ifx\directlua\@undefined
299     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
300   \else
301     \input luababel.def
302     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
303   \fi
304   \DeclareOption{base}{}%
305   \DeclareOption{showlanguages}{}%
306   \ProcessOptions
307   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
308   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
309   \global\let\@ifl@ter@\@ifl@ter
310   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
311   \endinput}{}%

```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to \BabelModifiers at \bbl@load@language; when no modifiers have been given, the former is \relax.

```

312 \bbl@trace{key=value and another general options}
313 \bbl@csarg\let\tempa\expandafter\csname opt@babel.sty\endcsname
314 \def\bbl@tempb#1.#2{% Removes trailing dot
315   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
316 \def\bbl@tempe#1=#2\@@{%
317   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
318 \def\bbl@tempd#1.#2\@nnil{%
319   \ifx\@empty#2%
320     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
321   \else
322     \in@{,provide=}{, #1}%
323     \ifin@
324       \edef\bbl@tempc{%
325         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
326     \else
327       \in@{$modifiers$}{$#1$}%
328       \ifin@
329         \bbl@tempe#2\@@
330     \else
331       \in@{=}{#1}%
332       \ifin@
333         \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
334       \else
335         \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
336         \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
337       \fi
338     \fi
339   \fi
340 \fi}
341 \let\bbl@tempc\@empty
342 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
343 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want

to use the shorthand characters in the preamble of their documents this can help.

```

344 \DeclareOption{KeepShorthandsActive}{}
345 \DeclareOption{activeacute}{}
346 \DeclareOption{activegrave}{}
347 \DeclareOption{debug}{}
348 \DeclareOption{debug-german}{} % Temporary
349 \DeclareOption{noconfigs}{}
350 \DeclareOption{showlanguages}{}
351 \DeclareOption{silent}{}
352 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
353 \chardef\bbl@iniflag\z@
354 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
355 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
356 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
357 \chardef\bbl@ldfflag\z@
358 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
359 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
360 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
361 \DeclareOption{licr=extended}{}
362 \DeclareOption{licr=unextended}{}
363 % Don't use. Experimental.
364 \newif\ifbbl@single
365 \DeclareOption{selectors=off}{\bbl@singletrue}
366 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key `main`, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

367 \let\bbl@opt@shorthands\@nnil
368 \let\bbl@opt@config\@nnil
369 \let\bbl@opt@main\@nnil
370 \let\bbl@opt@headfoot\@nnil
371 \let\bbl@opt@layout\@nnil
372 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

373 \def\bbl@tempa#1=#2\bbl@tempa{%
374   \bbl@csarg\ifx{opt\#1}\@nnil
375     \bbl@csarg\edef{opt\#1}{\#2}%
376   \else
377     \bbl@error{bad-package-option}{\#1}{\#2}{}%
378   \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a `=`), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```

379 \let\bbl@language@opts\@empty
380 \DeclareOption*{%
381   \bbl@xin{\string=}{\CurrentOption}%
382   \ifin@
383     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
384   \else
385     \bbl@add@list\bbl@language@opts{\CurrentOption}%
386   \fi}

```

Now we finish the first pass (and start over).

```

387 \ProcessOptions*

```

3.5. Post-process some options

```

388 \ifx\bbl@opt@provide\@nnil
389   \let\bbl@opt@provide\@empty % %%% MOVE above

```

```

390 \else
391   \chardef\bbl@iniflag\@ne
392   \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
393     \in@{,provide,},{, #1,}%
394     \ifin@
395       \def\bbl@opt@provide{#2}%
396     \fi}
397 \fi

```

If there is no `shorthands=<chars>`, the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```

398 \bbl@trace{Conditional loading of shorthands}
399 \def\bbl@sh@string#1{%
400   \ifx#1\@empty\else
401     \ifx#1t\string~%
402     \else\ifx#1c\string,%
403     \else\string#1%
404     \fi\fi
405   \expandafter\bbl@sh@string
406   \fi}
407 \ifx\bbl@opt@shorthands\@nnil
408   \def\bbl@ifshorthand#1#2#3{#2}%
409 \else\ifx\bbl@opt@shorthands\@empty
410   \def\bbl@ifshorthand#1#2#3{#3}%
411 \else

```

The following macro tests if a shorthand is one of the allowed ones.

```

412 \def\bbl@ifshorthand#1{%
413   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
414   \ifin@
415     \expandafter\@firstoftwo
416   \else
417     \expandafter\@secondoftwo
418   \fi}

```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```

419 \edef\bbl@opt@shorthands{%
420   \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```

421 \bbl@ifshorthand{'}%
422   {\PassOptionsToPackage{activeacute}{babel}}{}
423 \bbl@ifshorthand{`}%
424   {\PassOptionsToPackage{activegrave}{babel}}{}
425 \fi\fi

```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\@resetactivechars`, but seems to work.

```

426 \ifx\bbl@opt@headfoot\@nnil\else
427   \g@addto@macro\@resetactivechars{%
428     \set@typeset@protect
429     \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
430     \let\protect\noexpand}
431 \fi

```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to none.

```

432 \ifx\bbl@opt@safe\@undefined
433   \def\bbl@opt@safe{BR}
434   % \let\bbl@opt@safe\@empty % Pending of \cite
435 \fi

```

For layout an auxiliary macro is provided, available for packages and language styles.
Optimization: if there is no layout, just do nothing.

```

436 \bbl@trace{Defining IfBabelLayout}
437 \ifx\bbl@opt@layout\@nnil
438   \newcommand\IfBabelLayout[3]{#3}%
439 \else
440   \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
441     \in@{,layout,}{, #1,}%
442     \ifin@
443       \def\bbl@opt@layout{#2}%
444       \bbl@replace\bbl@opt@layout{ }{.}%
445     \fi}
446 \newcommand\IfBabelLayout[1]{%
447   \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
448   \ifin@
449     \expandafter\@firstoftwo
450   \else
451     \expandafter\@secondoftwo
452   \fi}
453 \fi
454 \</package>

```

3.6. Plain: babel.def (start)

Because of the way docstrip works, we need to insert some code for Plain here. However, the tools provided by the babel installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

455 \< *core>
456 \ifx\ldf@quit\@undefined\else
457 \endinput\fi % Same line!
458 \<@Make sure ProvidesFile is defined@>
459 \ProvidesFile{babel.def}[<@date@> v<@version@> Babel common definitions]
460 \ifx\AtBeginDocument\@undefined
461   \<@Emulate LaTeX@>
462 \fi
463 \<@Basic macros@>
464 \</core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\LaTeX$. After it, we will resume the $\LaTeX$ -only stuff.

4. babel.sty and babel.def (common)

```

465 \< *package | core>
466 \def\bbl@version{<@version@>}
467 \def\bbl@date{<@date@>}
468 \<@Define core switching macros@>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

469 \def\adddialect#1#2{%
470   \global\chardef#1#2\relax
471   \bbl@usehooks{adddialect}{#1}{#2}}%
472 \begingroup
473   \count@#1\relax
474   \def\bbl@elt##1##2##3##4{%
475     \ifnum\count@=##2\relax
476       \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
477       \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
478         set to \expandafter\string\csname l@##1\endcsname\%
479         (\string\language\the\count@). Reported}%
480       \def\bbl@elt####1#####2#####3#####4{}}%

```

```

481     \fi}%
482     \bbl@cs{languages}%
483 \endgroup}

```

\bbl@iflanguage executes code only if the language l@ exists. Otherwise raises an error.

The argument of \bbl@fixname has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when \foreignlanguage and the like appear in a \MakeXXXcase. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note l@ is encapsulated, so that its case does not change.

```

484 \def\bbl@fixname#1{%
485   \begingroup
486   \def\bbl@tempe{l}%
487   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
488   \bbl@tempd
489   {\lowercase\expandafter{\bbl@tempd}%
490    {\uppercase\expandafter{\bbl@tempd}%
491     \@empty
492     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
493      \uppercase\expandafter{\bbl@tempd}}}%
494     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
495      \lowercase\expandafter{\bbl@tempd}}}%
496   \@empty
497   \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
498   \bbl@tempd
499   \bbl@exp{\bbl@usehooks{language}{\language}{#1}}}%
500 \def\bbl@iflanguage#1{%
501   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that sr-latn-ba becomes fr-Latn-BA.

\bbl@bcpllookup returns \bbl@bcp with either the found ini tag or \relax. Temporary version, to be refactored with the new \text_bcp_parse:n, which is not currently fully expandable with invalid BCP 47 tags. The argument may be is some cases either a language name or a tag (e.g., \foreignlanguage. This was a huge mistake, because of the potential ambiguities (for example, the name vai-latin is also a syntactically valid tag).

```

502 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
503 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
504 \def\bbl@bcpllookup#1{%
505   \let\bbl@templ\@empty % language
506   \let\bbl@temps\@empty % script
507   \let\bbl@tempr\@empty % regions
508   \let\bbl@tempv\@empty % variant
509   \edef\bbl@tempa{#1}%
510   \bbl@replace\bbl@tempa{-}{,}%
511   \bbl@replace\bbl@tempa{_}{,}%
512   \bbl@foreach\bbl@tempa{%
513     \ifx\bbl@templ\@empty
514       \lowercase{\def\bbl@templ{##1}}%
515     \else\ifnum\bbl@cntchars{##1}=4
516       \bbl@xin{\@car##1\@nil}{0123456789}%
517       \ifin@
518         \def\bbl@tempv{##1}% eg, 1901
519       \else
520         \uppercase{\edef\bbl@tempb{\@car##1\@nil}}%
521         \lowercase{\edef\bbl@tempb{\bbl@tempb\@gobble##1}}%
522       \fi
523     \else\ifnum\bbl@cntchars{##1}<4
524       \uppercase{\def\bbl@tempr{##1}}%
525     \else\ifnum\bbl@cntchars{##1}>4
526       \lowercase{\def\bbl@tempv{##1}}%
527     \fi\fi\fi\fi}%

```

```

528 \bbl@exp{\bbl@bcplookup@i{\bbl@templ}{\bbl@temps}{\bbl@tempr}{\bbl@tempv}}
529 \def\bbl@bcplookup@try#1{%
530 \ifx\bbl@bcp\relax
531 \IfFileExists{babel-#1\bbl@tempe.ini}{\edef\bbl@bcp{#1\bbl@tempe}}{}%
532 \fi}
533 \def\bbl@bcplookup@i#1#2#3#4{%
534 \let\bbl@bcp\relax
535 \def\bbl@tempe{#4}%
536 \ifx\bbl@tempe\@empty\else
537 \edef\bbl@tempe{-#4}%
538 \fi
539 \edef\bbl@tempa{#2#3\bbl@tempe}% en
540 \ifx\bbl@tempa\@empty
541 \bbl@bcplookup@try{#1}%
542 \else
543 \edef\bbl@tempa{#3\bbl@tempe}% en-Latn
544 \ifx\bbl@tempa\@empty
545 \bbl@bcplookup@try{#1-#2}%
546 \bbl@bcplookup@try{#1}%
547 \else
548 \edef\bbl@tempa{#2\bbl@tempe}% en-US, en-001
549 \ifx\bbl@tempa\@empty
550 \bbl@bcplookup@try{#1-#3}%
551 \bbl@bcplookup@try{#1}%
552 \else % en-Latn-US
553 \bbl@bcplookup@try{#1-#2-#3}%
554 \bbl@bcplookup@try{#1-#2}%
555 \bbl@bcplookup@try{#1-#3}%
556 \bbl@bcplookup@try{#1}%
557 \fi
558 \fi
559 \fi
560 \ifx\bbl@bcp\relax
561 \ifx\bbl@tempe\@empty\else\bbl@bcplookup@i{#1}{#2}{#3}{\fi}%
562 \fi}
563 %
564 \let\bbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

565 \def\iflanguage#1{%
566 \bbl@iflanguage{#1}{%
567 \ifnum\csname l@#1\endcsname=\language
568 \expandafter\@firstoftwo
569 \else
570 \expandafter\@secondoftwo
571 \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

572 \let\bbl@select@type\z@
573 \edef\selectlanguage{%
574 \noexpand\protect
575 \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage_`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.


```
576 \ifx\@undefined\protect\let\protect\relax\fi
```

The following definition is preserved for backwards compatibility (e.g., arabi, koma). It is related to a trick for 2.09, now discarded.

```
577 \let\xstring\string
```

Since version 3.5 babel writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need T_EX's `aftergroup` mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bbl@pop@language` to be executed at the end of the group. It calls `\bbl@set@language` with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bbl@language@stack` and initially empty.

```
578 \def\bbl@language@stack{}
```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```
579 \def\bbl@push@language{%
580   \ifx\language\@undefined\else
581     \ifx\currentgrouplevel\@undefined
582       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
583     \else
584       \ifnum\currentgrouplevel=\z@
585         \xdef\bbl@language@stack{\language+}%
586       \else
587         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
588       \fi
589     \fi
590 }
```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro `\language`. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the '+'-sign) in `\language` and stores the rest of the string in `\bbl@language@stack`.

```
591 \def\bbl@pop@lang#1+#2\@@{%
592   \edef\language{#1}%
593   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before `\bbl@pop@lang` is executed T_EX first *expands* the stack, stored in `\bbl@language@stack`. The result of that is that the argument string of `\bbl@pop@lang` contains one or more language names, each followed by a '+'-sign (zero language names won't occur as this macro will only be called after something has been pushed on the stack).

```
594 \let\bbl@ifrestoring\@secondoftwo
595 \def\bbl@pop@language{%
596   \expandafter\bbl@pop@lang\bbl@language@stack\@@
597   \let\bbl@ifrestoring\@firstoftwo
598   \expandafter\bbl@set@language\expandafter{\language}%
599   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to `\bbl@set@language` to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of `\localeid`. This means `\l@. . .` will be reserved for hyphenation patterns (so that two locales can share the same rules).

```

600 \chardef\localeid\z@
601 \gdef\bbl@id@last{0} % No real need for a new counter
602 \def\bbl@id@assign{%
603   \bbl@ifunset\bbl@id@\language\language}%
604   {\count@\bbl@id@last\relax
605    \advance\count@\@ne
606    \global\bbl@csarg\chardef{id@\language\language}\count@
607    \xdef\bbl@id@last{\the\count@}%
608    \ifcase\bbl@engine\or
609      \directlua{
610        Babel.locale_props[\bbl@id@last] = {}
611        Babel.locale_props[\bbl@id@last].name = '\language'
612        Babel.locale_props[\bbl@id@last].vars = {}
613      }%
614    \fi}%
615  }%
616  \chardef\localeid\bbl@cl{id@}

```

The unprotected part of `\selectlanguage`. In case it is used as environment, declare `\endselectlanguage`, just for safety.

```

617 \let\bbl@select@opts@empty
618 \expandafter\def\csname selectlanguage \endcsname{%
619   \ifnextchar[\bbl@select@s{\bbl@select@s[]}}
620 \def\bbl@select@s[#1]#2{%
621   \def\bbl@select@opts{#1}%
622   \ifnum\bbl@hymapsel=\ccclv\let\bbl@hymapsel\tw\fi
623   \bbl@push@language
624   \aftergroup\bbl@pop@language
625   \bbl@set@language{#2}}
626 \let\endselectlanguage\relax

```

`\bbl@set@language` The macro `\bbl@set@language` takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of `\language`. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in `\language` are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining `\BabelContentsFiles`, but make sure they are loaded inside a group (as `aux`, `toc`, `lof`, and `lot` do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from hyperref, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

627 \def\BabelContentsFiles{toc,lof,lot}
628 \def\bbl@set@language#1{% from selectlanguage, pop@
629   % The old buggy way. Preserved for compatibility, but simplified
630   \edef\language{\expandafter\string#1\empty}%
631   \select@language{\language}%
632   \bbl@xin@{,main,},{,\bbl@select@opts,}%
633   \ifin@
634     \let\bbl@main@language\localename
635     \let\mainlocalename\localename
636   \fi
637   \let\bbl@select@opts@empty
638   % write to aux files
639   \expandafter\ifx\csname date\language\endcsname\relax\else

```

```

640 \if@filesw
641 \bbl@xin@{,nofiles,},{, \bbl@select@opts,}%
642 \ifin@ \else
643 \ifx \babel@aux \gobbletwo \else % Set if single in the first, redundant
644 \bbl@savelastskip
645 \protected@write\@auxout{\string \babel@aux{\bbl@auxname}{}}%
646 \bbl@restorelastskip
647 \fi
648 \bbl@usehooks{write}{}%
649 \fi
650 \fi
651 \fi}
652 %
653 \let \bbl@restorelastskip \relax
654 \let \bbl@savelastskip \relax
655 %
656 \def \select@language#1{% from set@, babel@aux, babel@toc
657 \ifx \bbl@select@name \empty
658 \def \bbl@select@name{select}%
659 \fi
660 % set hmap
661 \ifnum \bbl@hmapsel = \ccclv \chardef \bbl@hmapsel 4 \relax \fi
662 % set name (when coming from babel@aux)
663 \edef \language#1{%
664 \bbl@fixname \language#1
665 % define \localename when coming from set@, with a trick
666 \ifx \scantokens \undefined
667 \def \localename{??}%
668 \else
669 \bbl@exp{\scantokens{\def \localename{\language#1} \noexpand} \relax}%
670 \fi
671 \bbl@provide@locale
672 \bbl@iflanguage \language#1{%
673 \let \bbl@select@type \z@
674 \expandafter \bbl@switch \expandafter {\language#1}}
675 \def \babel@aux#1#2{%
676 \select@language#1}%
677 \bbl@foreach \BabelContentsFiles{% \relax -> don't assume vertical mode
678 \@writefile{##1}{\babel@toc{#1}{#2} \relax}}%
679 \def \babel@toc#1#2{%
680 \select@language#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring $\TeX$ in a certain pre-defined state.

The name of the language is stored in the control sequence `\language#1`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

681 \newif \ifbbl@usedategroup
682 \let \bbl@savextras \empty
683 \def \bbl@switch#1{% from select@, foreign@
684 % restore
685 \originalTeX
686 \expandafter \def \expandafter \originalTeX \expandafter {%
687 \csname noextras#1 \endcsname

```

```

688 \let\originalTeX\@empty
689 \babel@beginsave}%
690 \bbl@usehooks{afterreset}{}%
691 \languageshorthands{none}%
692 % set the locale id
693 \bbl@id@assign
694 % switch captions, date
695 \bbl@bsphack
696 \ifcase\bbl@select@type
697 \csname captions#1\endcsname\relax
698 \csname date#1\endcsname\relax
699 \else
700 \bbl@xin@{,captions,}{,\bbl@select@opts,}%
701 \ifin@
702 \csname captions#1\endcsname\relax
703 \fi
704 \bbl@xin@{,date,}{,\bbl@select@opts,}%
705 \ifin@ % if \foreign... within \<language>date
706 \csname date#1\endcsname\relax
707 \fi
708 \fi
709 \bbl@esphack
710 % switch extras
711 \csname bbl@preextras@#1\endcsname
712 \bbl@usehooks{beforeextras}{}%
713 \csname extras#1\endcsname\relax
714 \bbl@usehooks{afterextras}{}%
715 % > babel-ensure
716 % > babel-sh-<short>
717 % > babel-bidi
718 % > babel-fontspec
719 \let\bbl@savextras\@empty
720 % hyphenation - case mapping
721 \ifcase\bbl@opt@hyphenmap\or
722 \def\BabelLower##1##2{\lccode##1=##2\relax}%
723 \ifnum\bbl@hymap>4\else
724 \csname\language @bbl@hyphenmap\endcsname
725 \fi
726 \chardef\bbl@opt@hyphenmap\z@
727 \else
728 \ifnum\bbl@hymap>\bbl@opt@hyphenmap\else
729 \csname\language @bbl@hyphenmap\endcsname
730 \fi
731 \fi
732 \let\bbl@hymap\@cclv
733 % hyphenation - select rules
734 \ifnum\csname l@\language\endcsname=\l@unhyphenated
735 \edef\bbl@tempa{u}%
736 \else
737 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
738 \fi
739 % linebreaking - handle u, e, k (v in the future)
740 \bbl@xin@{/u}{/\bbl@tempa}%
741 \ifin@\else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
742 \ifin@\else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
743 \ifin@\else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
744 \ifin@\else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
745 % hyphenation - save mins
746 \babel@savevariable\lefthyphenmin
747 \babel@savevariable\righthyphenmin
748 \ifnum\bbl@engine=@ne
749 \babel@savevariable\hyphenationmin
750 \fi

```

```

751 \ifin@
752 % unhyphenated/kashida/elongated/padding = allow stretching
753 \language\l@unhyphenated
754 \babel@savevariable\emergencystretch
755 \emergencystretch\maxdimen
756 \babel@savevariable\hbadness
757 \hbadness\@M
758 \else
759 % other = select patterns
760 \bbl@patterns{#1}%
761 \fi
762 % hyphenation - set mins
763 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
764 \set@hyphenmins\tw@\thr@@\relax
765 \@nameuse{bbl@hyphenmins@}%
766 \else
767 \expandafter\expandafter\expandafter\set@hyphenmins
768 \csname #1hyphenmins\endcsname\relax
769 \fi
770 \@nameuse{bbl@hyphenmins@}%
771 \@nameuse{bbl@hyphenmins@\language}%
772 \@nameuse{bbl@hyphenatmin@}%
773 \@nameuse{bbl@hyphenatmin@\language}%
774 \let\bbl@selectortname\empty}

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

775 \edef\otherlanguage{%
776 \noexpand\protect
777 \expandafter\noexpand\csname otherlanguage \endcsname}
778 \expandafter\def\csname otherlanguage \endcsname{%
779 \@ifstar{\@nameuse{otherlanguage*}}\bbl@otherlanguage}
780 \def\bbl@otherlanguage#1{%
781 \def\bbl@selectortname{other}%
782 \ifnum\bbl@hymapsel=\@ccclv\let\bbl@hymapsel\thr@@\fi
783 \csname selectlanguage \endcsname{#1}%
784 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

785 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of `\foreign@language`.

```

786 \expandafter\def\csname otherlanguage*\endcsname{%
787 \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
788 \def\bbl@otherlanguage@s[#1]#2{%
789 \def\bbl@selectortname{other*}%
790 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
791 \def\bbl@select@opts{#1}%
792 \foreign@language{#2}}

```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```

793 \expandafter\let\csname endotherlanguage*\endcsname\relax

```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the

`\extras<language>` command doesn't make any `\global` changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a 'text' command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in `vmode` and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into `hmode` with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```

794 \providecommand\bbl@beforeforeign{}
795 \edef\foreignlanguage{%
796   \noexpand\protect
797   \expandafter\noexpand\csname foreignlanguage \endcsname}
798 \expandafter\def\csname foreignlanguage \endcsname{%
799   \@ifstar\bbl@foreign@s\bbl@foreign@x}
800 \providecommand\bbl@foreign@x[3][]{%
801   \begingroup
802     \def\bbl@selectorname{foreign}%
803     \def\bbl@select@opts{#1}%
804     \let\BabelText\@firstofone
805     \bbl@beforeforeign
806     \foreign@language{#2}%
807     \bbl@usehooks{foreign}{}%
808     \BabelText{#3}% Now in horizontal mode!
809   \endgroup}
810 \def\bbl@foreign@s#1#2{%
811   \begingroup
812     {\par}%
813     \def\bbl@selectorname{foreign*}%
814     \let\bbl@select@opts\@empty
815     \let\BabelText\@firstofone
816     \foreign@language{#1}%
817     \bbl@usehooks{foreign*}{}%
818     \bbl@dirparastext
819     \BabelText{#2}% Still in vertical mode!
820   {\par}%
821   \endgroup}
822 \providecommand\BabelWrapText[1]{%
823   \def\bbl@tempa{\def\BabelText###1}%
824   \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}
```

`\foreign@language` This macro does the work for `\foreignlanguage` and the `otherlanguage*` environment. First we need to store the name of the language and check that it is a known language. Then it just calls `bbl@switch`.

```

825 \def\foreign@language#1{%
826   % set name
827   \edef\language#1}%
828 \ifbbl@usedategroup
829   \bbl@add\bbl@select@opts{,date,}%
830   \bbl@usedategroupfalse
831 \fi
832 \bbl@fixname\language
833 \let\localname\language
834 \bbl@provide@locale
835 \bbl@iflanguage\language%
```

```

836 \let\bbl@select@type\@ne
837 \expandafter\bbl@switch\expandafter{\language\name}}

```

The following macro executes conditionally some code based on the selector being used.

```

838 \def\IfBabelSelectorTF#1{%
839 \bbl@xin@{\bbl@select@name,}{,\zap@space#1 \@empty,}%
840 \ifin@
841 \expandafter\@firstoftwo
842 \else
843 \expandafter\@secondoftwo
844 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

845 \let\bbl@hyphlist\@empty
846 \let\bbl@hyphenation@\relax
847 \let\bbl@pttnlist\@empty
848 \let\bbl@patterns@\relax
849 \let\bbl@hymapsel=\@cclv
850 \def\bbl@patterns#1{%
851 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
852 \csname l@#1\endcsname
853 \edef\bbl@tempa{#1}%
854 \else
855 \csname l@#1:\f@encoding\endcsname
856 \edef\bbl@tempa{#1:\f@encoding}%
857 \fi
858 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
859 % > luatex
860 \ifundefined{bbl@hyphenation@}{% Can be \relax!
861 \begingroup
862 \bbl@xin@{\number\language,}{,\bbl@hyphlist}%
863 \ifin@\else
864 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
865 \hyphenation{%
866 \bbl@hyphenation@
867 \@ifundefined{bbl@hyphenation@#1}%
868 \@empty
869 {\space\csname bbl@hyphenation@#1\endcsname}}%
870 \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
871 \fi
872 \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change \language and when the hyphenation rules specified were not loaded it has no effect. Note however, \lccode's and font encodings are not set at all, so in most cases you should use otherlanguage*.

```

873 \def\hyphenrules#1{%
874 \edef\bbl@tempf{#1}%
875 \bbl@fixname\bbl@tempf
876 \bbl@iflanguage\bbl@tempf{%
877 \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
878 \ifx\languageshortands\@undefined\else
879 \languageshortands{none}%
880 \fi
881 \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
882 \set@hyphenmins\tw@thr@@ \relax

```

```

883 \else
884 \expandafter\expandafter\expandafter\set@hyphenmins
885 \csname\bbl@tempf hyphenmins\endcsname\relax
886 \fi}}
887 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\<language>hyphenmins` is already defined this command has no effect.

```

888 \def\providehyphenmins#1#2{%
889 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
890 \@namedef{#1hyphenmins}{#2}%
891 \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

892 \def\set@hyphenmins#1#2{%
893 \lefthyphenmin#1\relax
894 \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_{\epsilon}$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by babel.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

895 \ifx\ProvidesFile\@undefined
896 \def\ProvidesLanguage#1[#2 #3 #4]{%
897 \wlog{Language: #1 #4 #3 <#2>}%
898 }
899 \else
900 \def\ProvidesLanguage#1{%
901 \begingroup
902 \catcode`\ 10 %
903 \@makeother\/%
904 \@ifnextchar[%]
905 {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}}
906 \def\@provideslanguage#1[#2]{%
907 \wlog{Language: #1 #2}%
908 \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
909 \endgroup}
910 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\text{\TeX}$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```

911 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi

```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```

912 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi

```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```

913 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}
914 \let\uselocale\setlocale
915 \let\locale\setlocale
916 \let\selectlocale\setlocale
917 \let\textlocale\setlocale
918 \let\textlanguage\setlocale
919 \let\languagetext\setlocale

```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn't been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for \language=0 in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about \PackageError it must be L^AT_EX 2_ε, so we can safely use its error handling interface. Otherwise we'll have to 'keep it simple'.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```

920 \edef\bbl@nulllanguage{\string\language=0}
921 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
922 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
923   \global\@namedef{#2}{\textbf{?#1?}}}%
924   \@nameuse{#2}%
925   \edef\bbl@tempa{#1}%
926   \bbl@sreplace\bbl@tempa{name}{}%
927   \bbl@warning{%
928     \@backslashchar#1 not set for '\language'. Please,\\%
929     define it after the language has been loaded\\%
930     (typically in the preamble) with:\\%
931     \string\setlocalecaption{\language}{\bbl@tempa}{..}\\%
932     Feel free to contribute on github.com/latex3/babel.\\%
933     Reported}}
934 \def\bbl@tentative{\protect\bbl@tentative@i}
935 \def\bbl@tentative@i#1{%
936   \bbl@warning{%
937     Some functions for '#1' are tentative.\\%
938     They might not work as expected and their behavior\\%
939     could change in the future.\\%
940     Reported}}
941 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}{}}
942 \def\@nopatterns#1{%
943   \bbl@warning
944     {No hyphenation patterns were preloaded for\\%
945     the language '#1' into the format.\\%
946     Please, configure your TeX system to add them and\\%
947     rebuild the format. Now I will use the patterns\\%
948     preloaded for \bbl@nulllanguage\space instead}}
949 \let\bbl@usehooks\@gobbletwo

Here ended the now discarded switch.def.
Here also (currently) ends the base option.

950 \ifx\bbl@onlyswitch\@empty\endinput\fi

```

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named \bbl@e@<language>. We register a hook at the afterextras event which just executes this macro in a “complete” selection (which, if undefined, is \relax and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro \bbl@e@<language> contains \bbl@ensure{\include}{\exclude}{\fontenc}, which in turn loops over the macros names in \bbl@captionslist, excluding (with the help of \in@) those in the exclude list. If the fontenc is given (and not \relax), the \fontencoding is also added. Then we loop over the include list, but if the macro already contains \foreignlanguage, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

951 \bbl@trace{Defining babelensure}
952 \newcommand\babelensure[2][]{%
953   \AddBabelHook{babel-ensure}{afterextras}{%
954     \ifcase\bbl@select@type
955       \bbl@cl{e}%

```

```

956 \fi}%
957 \begingroup
958 \let\bbbl@ens@include\@empty
959 \let\bbbl@ens@exclude\@empty
960 \def\bbbl@ens@fontenc{\relax}%
961 \def\bbbl@tempb##1{%
962 \ifx\@empty##1\else\noexpand##1\expandafter\bbbl@tempb\fi}%
963 \edef\bbbl@tempa{\bbbl@tempb#1\@empty}%
964 \def\bbbl@tempb##1=##2\@@{\@namedef{\bbbl@ens@##1}{##2}}%
965 \bbbl@foreach\bbbl@tempa{\bbbl@tempb##1\@@}%
966 \def\bbbl@tempc{\bbbl@ensure}%
967 \expandafter\bbbl@add\expandafter\bbbl@tempc\expandafter{%
968 \expandafter\bbbl@ens@include}%
969 \expandafter\bbbl@add\expandafter\bbbl@tempc\expandafter{%
970 \expandafter\bbbl@ens@exclude}%
971 \toks@{\expandafter\bbbl@tempc}%
972 \bbbl@exp{%
973 \endgroup
974 \def\<bbbl@e@#2>{\the\toks@{\bbbl@ens@fontenc}}}%
975 \def\bbbl@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
976 \def\bbbl@tempb##1{% elt for (excluding) \bbbl@captionslist list
977 \ifx##1\@undefined % 3.32 - Don't assume the macro exists
978 \edef##1{\noexpand\bbbl@nocaption
979 {\bbbl@stripslash##1}{\language\bbbl@stripslash##1}}}%
980 \fi
981 \ifx##1\@empty\else
982 \in@{##1}{#2}%
983 \ifin@else
984 \let\bbbl@tempc\language\language
985 \bbbl@replace\bbbl@tempc{-}{@}%
986 \bbbl@ifunset{\bbbl@ensure@\bbbl@tempc}%
987 {\bbbl@exp{%
988 \\\DeclareRobustCommand\<bbbl@ensure@\bbbl@tempc>[1]{%
989 \\\foreignlanguage{\language\language}%
990 {\ifx\relax#3\else
991 \\\fontencoding{#3}\selectfont
992 \fi
993 #####1}}}%
994 {}%
995 \toks@{\expandafter{##1}%
996 \edef##1{%
997 \bbbl@csarg\noexpand{ensure@\bbbl@tempc}%
998 {\the\toks@}}}%
999 \fi
1000 \expandafter\bbbl@tempb
1001 \fi}%
1002 \expandafter\bbbl@tempb\bbbl@captionslist\today\@empty
1003 \def\bbbl@tempa##1{% elt for include list
1004 \ifx##1\@empty\else
1005 \let\bbbl@tempc\language\language
1006 \bbbl@replace\bbbl@tempc{-}{@}%
1007 \bbbl@csarg\in@{ensure@\bbbl@tempc\expandafter}\expandafter{##1}%
1008 \ifin@else
1009 \bbbl@tempb##1\@empty
1010 \fi
1011 \expandafter\bbbl@tempa
1012 \fi}%
1013 \bbbl@tempa#1\@empty}
1014 \def\bbbl@captionslist{%
1015 \prefacename\refname\abstractname\bibname\chaptername\appendixname
1016 \contentsname\listfigurename\listtablename\indexname\figurename
1017 \tablename\partname\enclname\ccname\headtoname\pagename\seename
1018 \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```
1019 \bbl@trace{Short tags}
1020 \newcommand\babeltags[1]{%
1021   \edef\bbl@tempa{\zap@space#1 \@empty}%
1022   \def\bbl@tempb##1=##2\@{
1023     \edef\bbl@tempc{%
1024       \noexpand\newcommand
1025       \expandafter\noexpand\csname ##1\endcsname{%
1026         \noexpand\protect
1027         \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1028       \noexpand\newcommand
1029       \expandafter\noexpand\csname text##1\endcsname{%
1030         \noexpand\foreignlanguage{##2}}
1031     \bbl@tempc}%
1032   \bbl@for\bbl@tempa\bbl@tempa{%
1033     \expandafter\bbl@tempb\bbl@tempa\@{}}
```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```
1034 \bbl@trace{Compatibility with language.def}
1035 \ifx\directlua\@undefined\else
1036   \ifx\bbl@luapatterns\@undefined
1037     \input luababel.def
1038   \fi
1039 \fi
1040 \ifx\bbl@languages\@undefined
1041   \ifx\directlua\@undefined
1042     \openin0 = language.def
1043     \ifeof0
1044       \closein0
1045       \message{I couldn't find the file language.def}
1046     \else
1047       \closein0
1048       \begingroup
1049         \def\addlanguage#1#2#3#4#5{%
1050           \expandafter\ifx\csname lang@#1\endcsname\relax\else
1051             \global\expandafter\let\csname l@#1\endcsname
1052             \csname lang@#1\endcsname
1053           \fi}%
1054         \def\uselanguage#1{%
1055           \input language.def
1056         \endgroup
1057       \fi
1058     \fi
1059   \chardef\l@english\z@
1060 \fi
```

\addto It takes two arguments, a *<control sequence>* and TeX-code to be added to the *<control sequence>*.

If the *<control sequence>* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```
1061 \def\addto#1#2{%
1062   \ifx#1\@undefined
1063     \def#1{#2}%
1064   \else
1065     \ifx#1\relax
```

```

1066     \def#1{#2}%
1067     \else
1068     {\toks@ \expandafter{#1#2}%
1069     \xdef#1{\the\toks@}}%
1070     \fi
1071 \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1072 \bbl@trace{Hooks}
1073 \newcommand\AddBabelHook[3][ ]{%
1074   \bbl@i funset{\bbl@hk@#2}{\EnableBabelHook{#2}}}%
1075   \def\bbl@tempa##1,##3=##2,##3\@empty{\def\bbl@tempb{##2}}%
1076   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1077   \bbl@i funset{\bbl@ev@#2@#3@#1}%
1078   {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1079   {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1080   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1081 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1082 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1083 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1084 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1085   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1086   \def\bbl@elth##1{%
1087     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1088     \bbl@cs{ev@#2@#3}%
1089     \ifx\language\@undefined\else % Test required for Plain (?)
1090       \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1091       \def\bbl@elth##1{%
1092         \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1093         \bbl@cs{ev@#2@#3}%
1094       \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1095 \def\bbl@evargs{,% <- don't delete this comma
1096   everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1097   adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1098   beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1099   hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1100   beforestart=0,language=2,begindocument=1}
1101 \ifx\NewHook\@undefined\else % Test for Plain (?)
1102   \def\bbl@tempa#1=#2\@{ \NewHook{babel/#1}\NewHook{babel/*/#1}}
1103   \bbl@foreach\bbl@evargs{\bbl@tempa#1\@{
1104 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1105 \providecommand\PassOptionsToLocale[2]{%
1106   \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit `\LdfInit` macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, ‘=’, because it is sometimes used in constructions with the `\let` primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to `\LdfInit` is a control sequence. We do that by looking at the first token after passing #2 through `string`. When it is equal to `\@backslashchar` we are dealing with a control sequence which we can compare with `\@undefined`.

If so, we call `\ldf@quit` to set the main language, restore the category code of the `@`-sign and call `\endinput`

When #2 was *not* a control sequence we construct one and compare it with `\relax`.

Finally we check `\originalTeX`.

```

1107 \bbl@trace{Macros for setting language files up}
1108 \def\bbl@ldfinit{%
1109   \let\bbl@screset\@empty
1110   \let\BabelStrings\bbl@opt@string
1111   \let\BabelOptions\@empty
1112   \let\BabelLanguages\relax
1113   \ifx\originalTeX\@undefined
1114     \let\originalTeX\@empty
1115   \else
1116     \originalTeX
1117   \fi}
1118 \def\LdfInit#1#2{%
1119   \chardef\atcatcode=\catcode`\@
1120   \catcode`\@=11\relax
1121   \chardef\eqcatcode=\catcode`\=
1122   \catcode`\==12\relax
1123   \@ifpackagewith{babel}{ensureinfo=off}{}%
1124   {\ifx\InputIfFileExists\@undefined\else
1125     \bbl@ifunset{bbl@lname@#1}%
1126     {{\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1127       \def\language#1}%
1128       \bbl@id@assign
1129       \bbl@load@info{#1}}}%
1130     {}%
1131   \fi}%
1132   \expandafter\if\expandafter\@backslashchar
1133     \expandafter\@car\string#2\@nil
1134   \ifx#2\@undefined\else
1135     \ldf@quit{#1}%
1136   \fi
1137   \else
1138     \expandafter\ifx\csname#2\endcsname\relax\else
1139       \ldf@quit{#1}%
1140     \fi
1141   \fi
1142   \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1143 \def\ldf@quit#1{%
1144   \expandafter\main@language\expandafter{#1}%
1145   \catcode`\@=\atcatcode \let\atcatcode\relax
1146   \catcode`\==\eqcatcode \let\eqcatcode\relax
1147   \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1148 \def\bbl@afterldf{%
1149   \bbl@afterlang
1150   \let\bbl@afterlang\relax
1151   \let\BabelModifiers\relax
1152   \let\bbl@screset\relax}%
1153 \def\ldf@finish#1{%
1154   \loadlocalcfg{#1}%
1155   \bbl@afterldf
1156   \expandafter\main@language\expandafter{#1}%
1157   \catcode`\@=\atcatcode \let\atcatcode\relax
1158   \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in *LaTeX*.

```

1159 \@onlypreamble\LdfInit
1160 \@onlypreamble\ldf@quit
1161 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1162 \def\main@language#1{%
1163   \def\bbl@main@language{#1}%
1164   \let\language\name\bbl@main@language
1165   \let\localename\bbl@main@language
1166   \let\mainlocalename\bbl@main@language
1167   \bbl@id@assign
1168   \ifcase\bbl@engine\or
1169     \ifx\setattribute\undefined\else
1170       \setattribute\bbl@attr@locale\localeid
1171     \fi
1172   \fi
1173   \bbl@patterns{\language\name}}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1174 \def\bbl@beforestart{%
1175   \def\@nolanerr##1{%
1176     \bbl@carg\chardef{l@##1}\z@
1177     \bbl@warning{Undefined language '##1' in aux.\Reported}}%
1178   \bbl@usehooks{beforestart}{}%
1179   \global\let\bbl@beforestart\relax}
1180 \AtBeginDocument{%
1181   {\@nameuse\bbl@beforestart}}% Group!
1182   \if@filesw
1183     \providecommand\babel@aux[2]{}%
1184     \immediate\write\@mainaux{\unexpanded{%
1185       \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1186     \immediate\write\@mainaux{\string\@nameuse\bbl@beforestart}}%
1187   \fi
1188   \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1189   \ifbbl@single % must go after the line above.
1190     \renewcommand\selectlanguage[1]{}%
1191     \renewcommand\foreignlanguage[2]{#2}%
1192     \global\let\babel@aux\@gobbletwo % Also as flag
1193   \fi}

```

A bit of optimization. Select in heads/feet the language only if necessary.

```

1194 \def\select@language@x#1{%
1195   \ifcase\bbl@select@type

```

```

1196 \bbl@ifsamestring\language{#1}{\select@language{#1}}%
1197 \else
1198 \select@language{#1}%
1199 \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1200 \bbl@trace{Shorhands}
1201 \def\bbl@withactive#1#2{%
1202 \begingroup
1203 \lccode`~=#2\relax
1204 \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1205 \def\bbl@add@special#1{% 1:a macro like \", \?, etc.
1206 \bbl@add\dospecials{\do#1}% test \@sanitize = \relax, for back. compat.
1207 \bbl@ifunset{\@sanitize}{\bbl@add\@sanitize{\makeother#1}}%
1208 \ifx\nfss@catcodes\undefined\else
1209 \begingroup
1210 \catcode`#1\active
1211 \nfss@catcodes
1212 \ifnum\catcode`#1=\active
1213 \endgroup
1214 \bbl@add\nfss@catcodes{\makeother#1}%
1215 \else
1216 \endgroup
1217 \fi
1218 \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char<char>` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char<char>` by default (`<char>` being the character to be made active). Later its definition can be changed to expand to `\active@char<char>` by calling `\bbl@activate{<char>}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char"` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\<level>@group`, `\<level>@active` and `\<next-level>@active` (except in system).

```

1219 \def\bbl@active@def#1#2#3#4{%
1220 \@namedef{#3#1}{%
1221 \expandafter\ifx\csname#2@sh@#1\endcsname\relax
1222 \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1223 \else
1224 \bbl@afterfi\csname#2@sh@#1\endcsname
1225 \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```

1226 \long\@namedef{#3@arg#1}##1{%
1227   \expandafter\ifx\csname#2@sh@#1@\string##1@\endcsname\relax
1228     \bbl@afterelse\csname#4#1\endcsname##1%
1229   \else
1230     \bbl@afterfi\csname#2@sh@#1@\string##1@\endcsname
1231   \fi}}%

```

\initiate@active@char calls \@initiate@active@char with 3 arguments. All of them are the same character with different catcodes: active, other (\string'ed) and the original one. This trick simplifies the code a lot.

```

1232 \def\initiate@active@char#1{%
1233   \bbl@ifunset{active@char\string#1}%
1234   {\bbl@withactive
1235     {\expandafter\@initiate@active@char\expandafter}#1\string#1}%
1236   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them \relax and preserving some degree of protection).

```

1237 \def\@initiate@active@char#1#2#3{%
1238   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1239   \ifx#1\@undefined
1240     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1241   \else
1242     \bbl@csarg\let{oridef@#2}#1%
1243     \bbl@csarg\edef{oridef@#2}{%
1244       \let\noexpand#1%
1245       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1246   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define \normal@char<char> to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example ') the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to "8000 *a posteriori*").

```

1247 \ifx#1#3\relax
1248   \expandafter\let\csname normal@char#2\endcsname#3%
1249 \else
1250   \bbl@info{Making #2 an active character}%
1251   \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1252   \@namedef{normal@char#2}{%
1253     \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1254   \else
1255     \@namedef{normal@char#2}{#3}%
1256   \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at \begin{document}. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of \bibitem for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1257 \bbl@restoreactive{#2}%
1258 \AtBeginDocument{%
1259   \catcode`#2\active
1260   \if@filesw
1261     \immediate\write\@mainaux{\catcode`\string#2\active}%
1262   \fi}%
1263 \expandafter\bbl@add@special\csname#2\endcsname
1264 \catcode`#2\active
1265 \fi

```


Now we have set `\normal@char<char>`, we must define `\active@char<char>`, to be executed when the character is activated. We define the first level expansion of `\active@char<char>` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active<char>` to start the search of a definition in the user, language and system levels (or eventually `normal@char<char>`).

```

1266 \let\bbl@tempa\@firstoftwo
1267 \if\string^#2%
1268   \def\bbl@tempa{\noexpand\textormath}%
1269 \else
1270   \ifx\bbl@mathnormal\@undefined\else
1271     \let\bbl@tempa\bbl@mathnormal
1272   \fi
1273 \fi
1274 \expandafter\edef\csname active@char#2\endcsname{%
1275   \bbl@tempa
1276     {\noexpand\if@safe@actives
1277       \noexpand\expandafter
1278       \expandafter\noexpand\csname normal@char#2\endcsname
1279     \noexpand\else
1280       \noexpand\expandafter
1281       \expandafter\noexpand\csname bbl@doactive#2\endcsname
1282     \noexpand\fi}%
1283   {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1284 \bbl@csarg\edef{doactive#2}{%
1285   \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

`\active@prefix<char>\normal@char<char>`

(where `\active@char<char>` is *one* control sequence!).

```

1286 \bbl@csarg\edef{active@#2}{%
1287   \noexpand\active@prefix\noexpand#1%
1288   \expandafter\noexpand\csname active@char#2\endcsname}%
1289 \bbl@csarg\edef{normal@#2}{%
1290   \noexpand\active@prefix\noexpand#1%
1291   \expandafter\noexpand\csname normal@char#2\endcsname}%
1292 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn’t exist we check for a shorthand with an argument.

```

1293 \bbl@active@def#2\user@group{user@active}{language@active}%
1294 \bbl@active@def#2\language@group{language@active}{system@active}%
1295 \bbl@active@def#2\system@group{system@active}{normal@char}%

```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ‘ ’ ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```

1296 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1297   {\expandafter\noexpand\csname normal@char#2\endcsname}%
1298 \expandafter\edef\csname\user@group @sh#2@\string\protect'\endcsname
1299   {\expandafter\noexpand\csname user@active#2\endcsname}%

```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (‘) active we need to change `\pr@m@s` as well. Also, make sure that a single ‘ in math mode ‘does the right thing’. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```

1300 \if\string'#2%
1301   \let\prim@s\bbl@prim@s

```

```

1302 \let\active@math@prime#1%
1303 \fi
1304 \bbl@usehooks{initiateactive}{\#1}{\#2}{\#3}}

```

The following package options control the behavior of shorthands in math mode.

```

1305 <<*More package options>> ≡
1306 \DeclareOption{math=active}{}
1307 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1308 <</More package options>>

```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```

1309 \ifpackagewith{babel}{KeepShorthandsActive}%
1310 {\let\bbl@restoreactive\@gobble}%
1311 {\def\bbl@restoreactive#1{%
1312 \bbl@exp{%
1313 \\\AfterBabelLanguage\\CurrentOption
1314 {\catcode`#1=\the\catcode`#1\relax}%
1315 \\\AtEndOfPackage
1316 {\catcode`#1=\the\catcode`#1\relax}}}%
1317 \AtEndOfPackage{\let\bbl@restoreactive\@gobble}}

```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```

1318 \def\bbl@sh@select#1#2{%
1319 \expandafter\ifx\csname#1@sh@#2@sel\endcsname\relax
1320 \bbl@afterelse\bbl@scndcs
1321 \else
1322 \bbl@afterfi\csname#1@sh@#2@sel\endcsname
1323 \fi}

```

\active@prefix Used in the expansion of active characters has a function similar to `\OT1-cmd` in that it protects the active character whenever `\protect` is *not* `\@typeset@protect`. The `\@gobble` is needed to remove a token such as `\activechar`: (when the double colon was the active character to be dealt with). There are two definitions, depending of `\ifincsname` is available. If there is, the expansion will be more robust.

```

1324 \begingroup
1325 \bbl@ifunset{ifincsname}
1326 {\gdef\active@prefix#1{%
1327 \ifx\protect\@typeset@protect
1328 \else
1329 \ifx\protect\@unexpandable@protect
1330 \noexpand#1%
1331 \else
1332 \protect#1%
1333 \fi
1334 \expandafter\@gobble
1335 \fi}}
1336 {\gdef\active@prefix#1{%
1337 \ifincsname
1338 \string#1%
1339 \expandafter\@gobble
1340 \else
1341 \ifx\protect\@typeset@protect
1342 \else
1343 \ifx\protect\@unexpandable@protect
1344 \noexpand#1%

```

```

1345         \else
1346         \protect#1%
1347         \fi
1348         \expandafter\expandafter\expandafter\@gobble
1349         \fi
1350     \fi}}
1351 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char⟨char⟩. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string’ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1352 \newif\if@safe@actives
1353 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1354 \def\bbl@restore@actives{\if@safe@actives\@safe@activesfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char⟨char⟩ in the case of \bbl@activate, or \normal@char⟨char⟩ in the case of \bbl@deactivate.

```

1355 \chardef\bbl@activated\z@
1356 \def\bbl@activate#1{%
1357   \chardef\bbl@activated\@ne
1358   \bbl@withactive{\expandafter\let\expandafter}#1%
1359   \csname bbl@active@\string#1\endcsname}
1360 \def\bbl@deactivate#1{%
1361   \chardef\bbl@activated\tw@
1362   \bbl@withactive{\expandafter\let\expandafter}#1%
1363   \csname bbl@normal@\string#1\endcsname}

```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```

1364 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1365 \def\bbl@scndcs#1#2{\csname#2\endcsname}

```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro \babel@texpdf improves the interoperativity with hyperref and takes 4 arguments: (1) The T_EX code in text mode, (2) the string for hyperref, (3) the T_EX code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently hyperref doesn’t discriminate the mode). This macro may be used in ldf files.

```

1366 \def\babel@texpdf#1#2#3#4{%
1367   \ifx\texorpdfstring\@undefined
1368     \textormath{#1}{#3}%
1369   \else
1370     \texorpdfstring{\textormath{#1}{#3}}{#2}%
1371     % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1372   \fi}
1373 %

```

```

1374 \def\declare@shorthand#1#2{\@decl@short{#1}#2\@nil}
1375 \def\@decl@short#1#2#3\@nil#4{%
1376   \def\bbl@tempa{#3}%
1377   \ifx\bbl@tempa\@empty
1378     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1379     \bbl@ifunset{#1@sh@\string#2@}{}%
1380     {\def\bbl@tempa{#4}%
1381      \expandafter\ifx\csname#1@sh@\string#2@\endcsname\bbl@tempa
1382      \else
1383        \bbl@info
1384        {Redefining #1 shorthand \string#2\\%
1385         in language \CurrentOption}%
1386        \fi}%
1387     \@namedef{#1@sh@\string#2@}{#4}%
1388   \else
1389     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1390     \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1391     {\def\bbl@tempa{#4}%
1392      \expandafter\ifx\csname#1@sh@\string#2@\string#3@\endcsname\bbl@tempa
1393      \else
1394        \bbl@info
1395        {Redefining #1 shorthand \string#2\string#3\\%
1396         in language \CurrentOption}%
1397        \fi}%
1398     \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1399   \fi}

```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```

1400 \def\textormath{%
1401   \ifmmode
1402     \expandafter\@secondoftwo
1403   \else
1404     \expandafter\@firstoftwo
1405   \fi}

```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```

1406 \def\user@group{user}
1407 \def\language@group{english}
1408 \def\system@group{system}

```

\useshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```

1409 \def\useshorthands{%
1410   \@ifstar\bbl@usesh@s{\bbl@usesh@x{}}
1411 \def\bbl@usesh@s#1{%
1412   \bbl@usesh@x
1413   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bbl@activate{#1}}}%
1414   {#1}}
1415 \def\bbl@usesh@x#1#2{%
1416   \bbl@ifshorthand{#2}%
1417   {\def\user@group{user}%
1418    \initiate@active@char{#2}%
1419    #1%
1420    \bbl@activate{#2}}%
1421   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(*language*) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bbl@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```

1422 \def\user@language@group{user@\language@group}
1423 \def\bbl@set@user@generic#1#2{%
1424   \bbl@ifunset{user@generic@active#1}%
1425   {\bbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1426    \bbl@active@def#1\user@group{user@generic@active}{\language@active}%
1427    \expandafter\edef\csname#2@sh@#1@\endcsname{%
1428      \expandafter\noexpand\csname normal@char#1\endcsname}%
1429    \expandafter\edef\csname#2@sh@#1@\string\protect@\endcsname{%
1430      \expandafter\noexpand\csname user@active#1\endcsname}}%
1431   \@empty}
1432 \newcommand\defineshorthand[3][user]{%
1433   \edef\bbl@tempa{\zap@space#1 \@empty}%
1434   \bbl@for\bbl@tempb\bbl@tempa{%
1435     \if*\expandafter\@car\bbl@tempb\@nil
1436       \edef\bbl@tempb{user@\expandafter\@gobble\bbl@tempb}%
1437       \@expandtwoargs
1438       \bbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbl@tempb
1439     \fi
1440     \declare@shorthand{\bbl@tempb}{#2}{#3}}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```

1441 \def\languageshorthands#1{%
1442   \bbl@ifsamestring{none}{#1}{}{%
1443     \bbl@once{short-\localename-#1}{%
1444       \bbl@info{'\localename' activates '#1' shorthands.\@Reported}}}%
1445   \def\language@group{#1}}
```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with \aliasshorthands{"}{/} is \active@prefix / \active@char/, so we still need to let the latter to \active@char".

```

1446 \def\aliasshorthand#1#2{%
1447   \bbl@ifshorthand{#2}%
1448   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1449     \ifx\document\@notprerr
1450       \@notshorthand{#2}%
1451     \else
1452       \initiate@active@char{#2}%
1453       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1454       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1455       \bbl@activate{#2}%
1456     \fi
1457   \fi}%
1458   {\bbl@error{shorthand-is-off}{#2}{}}}
```

\@notshorthand

```

1459 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}
```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to \bbl@switch@sh, adding \@nil at the end to denote the end of the list of characters.

```

1460 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1461 \DeclareRobustCommand*\shorthandoff{%
1462   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1463 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}
```

\bbl@switch@sh The macro \bbl@switch@sh takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of \bbl@switch@sh.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as \active@char" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and \active. With the starred version, the original catcode and the original definition, saved in @initiate@active@char, are restored.

```

1464 \def\bbl@switch@sh#1#2{%
1465   \ifx#2\@nnil\else
1466     \bbl@ifunset{\bbl@active@\string#2}%
1467     {\bbl@error{not-a-shorthand-b}{\string#2}}}%
1468     {\ifcase#1%   off, on, off*
1469       \catcode`#2\relax
1470       \or
1471       \catcode`#2\active
1472       \bbl@ifunset{\bbl@shdef@\string#2}%
1473       {}%
1474       {\bbl@withactive{\expandafter\let\expandafter}#2%
1475         \csname bbl@shdef@\string#2\endcsname
1476         \bbl@csarg\let{shdef@\string#2}\relax}%
1477       \ifcase\bbl@activated\or
1478       \bbl@activate{#2}%
1479       \else
1480       \bbl@deactivate{#2}%
1481       \fi
1482       \or
1483       \bbl@ifunset{\bbl@shdef@\string#2}%
1484       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1485       {}%
1486       \csname bbl@oricat@\string#2\endcsname
1487       \csname bbl@oridef@\string#2\endcsname
1488       \fi}%
1489   \bbl@afterfi\bbl@switch@sh#1%
1490 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1491 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1492 \def\bbl@putsh#1{%
1493   \bbl@ifunset{\bbl@active@\string#1}%
1494   {\bbl@putsh@i#1\@empty\@nnil}%
1495   {\csname bbl@active@\string#1\endcsname}}
1496 \def\bbl@putsh@i#1#2\@nnil{%
1497   \csname\language@group @sh@\string#1@%
1498   \ifx\@empty#2\else\string#2@\fi\endcsname}
1499 %
1500 \ifx\bbl@opt@shorthands\@nnil\else
1501   \let\bbl@s@initiate@active@char\initiate@active@char
1502   \def\initiate@active@char#1{%
1503     \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1504   \let\bbl@s@switch@sh\bbl@switch@sh
1505   \def\bbl@switch@sh#1#2{%
1506     \ifx#2\@nnil\else
1507     \bbl@afterfi
1508     \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1509     \fi}
1510   \let\bbl@s@activate\bbl@activate
1511   \def\bbl@activate#1{%
1512     \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1513   \let\bbl@s@deactivate\bbl@deactivate
1514   \def\bbl@deactivate#1{%
1515     \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}

```

1516 \fi

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

1517 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting \prime for each right quote in mathmode is \prim@s. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1518 \def\bbl@prim@s{%
1519   \prime\futurelet\@let@token\bbl@pr@m@s}
1520 \def\bbl@if@primes#1#2{%
1521   \ifx#1\@let@token
1522     \expandafter\@firstoftwo
1523   \else\ifx#2\@let@token
1524     \bbl@afterelse\expandafter\@firstoftwo
1525   \else
1526     \bbl@afterfi\expandafter\@secondoftwo
1527   \fi\fi}
1528 \begingroup
1529   \catcode`\^=7 \catcode`\*=active \lccode`\*=``^
1530   \catcode`\'=12 \catcode`\`=active \lccode`\`=``'
1531   \lowercase{%
1532     \gdef\bbl@pr@m@s{%
1533       \bbl@if@primes" '%
1534       \pr@@@s
1535       {\bbl@if@primes*^ \pr@@@t\egroup}}}
1536 \endgroup

```

Usually the ~ is active and expands to \penalty\@M_. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character ~ as a start character for a shorthand, it is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```

1537 \initiate@active@char{~}
1538 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1539 \bbl@activate{~}

```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```

1540 \expandafter\def\csname OT1dqpos\endcsname{127}
1541 \expandafter\def\csname T1dqpos\endcsname{4}

```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```

1542 \ifx\f@encoding\undefined
1543   \def\f@encoding{OT1}
1544 \fi

```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro `\languageattribute` checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in `\bbl@known@attrs`. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated $\TeX$ code.

```

1545 \bbl@trace{Language attributes}
1546 \newcommand\languageattribute[2]{%
1547   \def\bbl@tempc{#1}%
1548   \bbl@fixname\bbl@tempc
1549   \bbl@iflanguage\bbl@tempc{%
1550     \bbl@vforeach{#2}{%
1551       \ifx\bbl@known@attrs\undefined
1552         \in@false
1553       \else
1554         \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1555       \fi
1556       \ifin@
1557         \bbl@warning{%
1558           You have more than once selected the attribute '##1'\%
1559           for language #1. Reported}%
1560       \else
1561         \bbl@info{Activated '##1' attribute for\\%
1562           '\bbl@tempc'. Reported}%
1563         \bbl@exp{%
1564           \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1565         \edef\bbl@tempa{\bbl@tempc-##1}%
1566         \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1567         {\csname\bbl@tempc @attr##1\endcsname}%
1568         {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1569       \fi}}
1570 \onlypreamble\languageattribute

```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```

1571 \def\bbl@declare@ttribute#1#2#3{%
1572   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1573   \ifin@
1574     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1575   \fi
1576   \bbl@add@list\bbl@attributes{#1-#2}%
1577   \expandafter\def\csname#1@attr@#2\endcsname{#3}}

```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```

1578 \def\bbl@ifattributeset#1#2#3#4{%
1579   \ifx\bbl@known@attrs\undefined
1580     \in@false
1581   \else
1582     \bbl@xin@{,#1-#2,}{,\bbl@known@attrs,}%
1583   \fi
1584   \ifin@
1585     \bbl@afterelse#3%

```



```

1586 \else
1587   \bbl@afterfi#4%
1588 \fi}

```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```

1589 \def\bbl@ifknown@ttrib#1#2{%
1590   \let\bbl@tempa\@secondoftwo
1591   \bbl@loopx\bbl@tempb{#2}{%
1592     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{, #1,}%
1593     \ifin@
1594       \let\bbl@tempa\@firstoftwo
1595     \else
1596       \fi}%
1597   \bbl@tempa}

```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at $\begin{document}$ time (if any is present).

```

1598 \def\bbl@clear@ttribs{%
1599   \ifx\bbl@attributes\@undefined\else
1600     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1601       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1602     \let\bbl@attributes\@undefined
1603   \fi}
1604 \def\bbl@clear@ttrib#1-#2.{%
1605   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1606 \AtBeginDocument{\bbl@clear@ttribs}

```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using $\babel@save$, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see $\selectlanguage$ and $\originalTeX$). Note undefined macros are not undefined any more when saved – they are $\relax$ 'ed.

\babel@savecnt

\babel@beginsave The initialization of a new save cycle: reset the counter to zero.

```

1607 \bbl@trace{Macros for saving definitions}
1608 \def\babel@beginsave{\babel@savecnt\z@}

```

Before it's forgotten, allocate the counter and initialize all.

```

1609 \newcount\babel@savecnt
1610 \babel@beginsave

```

\babel@save

\babel@savevariable The macro $\babel@save\langle csname \rangle$ saves the current meaning of the control sequence $\langle csname \rangle$ to $\originalTeX$ (which has to be expandable, i.e., you shouldn't let it to $\relax$). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to $\originalTeX$ and the counter is incremented. The macro $\babel@savevariable\langle variable \rangle$ saves the value of the variable. $\langle variable \rangle$ can be anything allowed after the $\the$ primitive. To avoid messing saved definitions up, they are saved only the very first time.

```

1611 \def\babel@save#1{%
1612   \def\bbl@tempa{{, #1,}}% Clumsy, for Plain

```

```

1613 \expandafter\babel@add\expandafter\babel@tempa\expandafter{%
1614   \expandafter{\expandafter,\babel@savedextras,}}%
1615 \expandafter\in@\babel@tempa
1616 \ifin@%else
1617   \babel@add\babel@savedextras{,#1,%}
1618   \babel@carg\let\babel@number\babel@savecnt\#1\relax
1619   \toks@\expandafter{\originalTeX\let#1=}%
1620   \babel@exp{%
1621     \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1622   \advance\babel@savecnt\@ne
1623   \fi}
1624 \def\babel@savevariable#1{%
1625   \toks@\expandafter{\originalTeX #1=}%
1626   \babel@exp{\def\\originalTeX{\the\toks@\the#1\relax}}}

```

\babel@redefine To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```

1627 \def\babel@redefine#1{%
1628   \edef\babel@tempa{\babel@stripslash#1}%
1629   \expandafter\let\csname org@\babel@tempa\endcsname#1%
1630   \expandafter\def\csname\babel@tempa\endcsname{
1631     \@onlypreamble\babel@redefine

```

\babel@redefine@long This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```

1632 \def\babel@redefine@long#1{%
1633   \edef\babel@tempa{\babel@stripslash#1}%
1634   \expandafter\let\csname org@\babel@tempa\endcsname#1%
1635   \long\expandafter\def\csname\babel@tempa\endcsname{
1636     \@onlypreamble\babel@redefine@long

```

\babel@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo_`. So it is necessary to check whether `\foo_` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo_`.

```

1637 \def\babel@redefineroobust#1{%
1638   \edef\babel@tempa{\babel@stripslash#1}%
1639   \babel@ifunset{\babel@tempa\space}%
1640     {\expandafter\let\csname org@\babel@tempa\endcsname#1%
1641       \babel@exp{\def\\#1{\protect\<\babel@tempa\space>}}}%
1642     {\babel@exp{\let\<org@\babel@tempa>\<\babel@tempa\space>}}}%
1643     \@namedef{\babel@tempa\space}}
1644 \@onlypreamble\babel@redefineroobust

```

4.11. French spacing

\babel@frenchspacing

\babel@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don’t want that. The command `\babel@frenchspacing` switches it on when it isn’t already in effect and `\babel@nonfrenchspacing` switches it off if necessary.

```

1645 \def\babel@frenchspacing{%
1646   \ifnum\the\sfcodes\<.\@m
1647     \let\babel@nonfrenchspacing\relax
1648   \else
1649     \frenchspacing
1650     \let\babel@nonfrenchspacing\nonfrenchspacing
1651   \fi}
1652 \let\babel@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with ini files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1653 \let\bbl@elt\relax
1654 \edef\bbl@fs@chars{%
1655   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1656   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1657   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1658 \def\bbl@pre@fs{%
1659   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1660   \edef\bbl@save@sfcodes{\bbl@fs@chars}}%
1661 \def\bbl@post@fs{%
1662   \bbl@save@sfcodes
1663   \edef\bbl@tempa{\bbl@cl{frspc}}%
1664   \edef\bbl@tempa{\expandafter\@car\bbl@tempa\@nil}%
1665   \if u\bbl@tempa      % do nothing
1666   \else\if n\bbl@tempa % non french
1667     \def\bbl@elt##1##2##3{%
1668       \ifnum\sfcode`##1=##2\relax
1669         \babel@savevariable{\sfcode`##1}%
1670         \sfcode`##1=##3\relax
1671       \fi}%
1672     \bbl@fs@chars
1673   \else\if y\bbl@tempa % french
1674     \def\bbl@elt##1##2##3{%
1675       \ifnum\sfcode`##1=##3\relax
1676         \babel@savevariable{\sfcode`##1}%
1677         \sfcode`##1=##2\relax
1678       \fi}%
1679     \bbl@fs@chars
1680   \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@<language>` for language ones. See `\bbl@patterns` above for further details. We make sure there is a space between words when multiple commands are used.

```

1681 \bbl@trace{Hyphens}
1682 \@onlypreamble\babelhyphenation
1683 \AtEndOfPackage{%
1684   \newcommand\babelhyphenation[2][\@empty]{%
1685     \ifx\bbl@hyphenation@\relax
1686       \let\bbl@hyphenation@\@empty
1687     \fi
1688     \ifx\bbl@hyphlist\@empty\else
1689       \bbl@warning{%
1690         You must not intermingle \string\selectlanguage\space and\\%
1691         \string\babelhyphenation\space or some exceptions will not\\%
1692         be taken into account. Reported}%
1693       \fi
1694     \ifx\@empty#1%
1695       \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1696     \else
1697       \bbl@vforeach{#1}{%
1698         \def\bbl@tempa{##1}%
1699         \bbl@fixname\bbl@tempa
1700         \bbl@iflanguage\bbl@tempa{%
1701           \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1702             \bbl@ifunset{bbl@hyphenation@\bbl@tempa}%
1703             {}%
1704             {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1705             #2}}}%

```

```
1706 \fi}}
```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```
1707 \ifx\NewDocumentCommand\@undefined\else
1708 \NewDocumentCommand\babelhyphenmins{sommo}{%
1709 \IfNoValueTF{#2}%
1710 {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1711 \IfValueT{#5}{%
1712 \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1713 \IfBooleanT{#1}{%
1714 \lefthyphenmin=#3\relax
1715 \righthyphenmin=#4\relax
1716 \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1717 {\edef\bbl@tempb{\zap@space#2 \@empty}%
1718 \bbl@for\bbl@tempa\bbl@tempb{%
1719 \@namedef\bbl@hyphenmins@{\bbl@tempa}{\set@hyphenmins{#3}{#4}}}%
1720 \IfValueT{#5}{%
1721 \@namedef\bbl@hyphenatmin@{\bbl@tempa}{\hyphenationmin=#5\relax}}}%
1722 \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1723 \fi}
```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than `\nobreak\hskip 0pt plus 0pt`. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```
1724 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1725 \def\bbl@t@one{Tl}
1726 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}
```

\babelhyphen Macros to insert common hyphens. Note the space before @ in `\babelhyphen`. Instead of protecting it with `\DeclareRobustCommand`, which could insert a `\relax`, we use the same procedure as shorthands, with `\active@prefix`.

```
1727 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1728 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1729 \def\bbl@hyphen{%
1730 \ifstar\bbl@hyphen@i @}{\bbl@hyphen@i \@empty}}
1731 \def\bbl@hyphen@i#1#2{%
1732 \lowercase{\bbl@ifunset\bbl@hy@#1#2\@empty}}%
1733 {\csname bbl@#1usehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1734 {\lowercase{\csname bbl@hy@#1#2\@empty\endcsname}}}
```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. `\nobreak` is always preceded by `\leavevmode`, in case the shorthand starts a paragraph.

```
1735 \def\bbl@usehyphen#1{%
1736 \leavevmode
1737 \ifdim\lastskip>\z@\mbox{#1}\else\nobreak#1\fi
1738 \nobreak\hskip\z@skip}
1739 \def\bbl@usehyphen#1{%
1740 \leavevmode\ifdim\lastskip>\z@\mbox{#1}\else#1\fi}
```

The following macro inserts the hyphen char.

```
1741 \def\bbl@hyphenchar{%
1742 \ifnum\hyphenchar\font=\m@ne
1743 \babelnullhyphen
1744 \else
1745 \char\hyphenchar\font
1746 \fi}
```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in `\ldf`’s. After a space, the `\mbox` in `\bbl@hy@nbreak` is redundant.

```

1747 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1748 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1749 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1750 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1751 \def\bbl@hy@nbreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1752 \def\bbl@hy@nbreak{\mbox{\bbl@hyphenchar}}
1753 \def\bbl@hy@repeat{%
1754   \bbl@usehyphen{%
1755     \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1756 \def\bbl@hy@repeat{%
1757   \bbl@usehyphen{%
1758     \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1759 \def\bbl@hy@empty{\hskip\z@skip}
1760 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro `\bbl@disc` is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1761 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by `luatex` and `xetex`. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1762 \bbl@trace{Multiencoding strings}
1763 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated `\SetCase`.

```

1764 <<{*More package options}>> ≡
1765 \DeclareOption{nocase}{}
1766 <</More package options>>

```

The following package options control the behavior of `\SetString`.

```

1767 <<{*More package options}>> ≡
1768 \let\bbl@opt@strings@nnil % accept strings=value
1769 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1770 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1771 \def\BabelStringsDefault{generic}
1772 <</More package options>>

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1773 \@onlypreamble\StartBabelCommands
1774 \def\StartBabelCommands{%
1775   \begingroup
1776   \@tempcnta="7F
1777   \def\bbl@tempa{%
1778     \ifnum\@tempcnta>"FF\else
1779       \catcode\@tempcnta=11
1780       \advance\@tempcnta\@ne
1781       \expandafter\bbl@tempa
1782     \fi}%
1783   \bbl@tempa
1784   <@Macros local to BabelCommands@>
1785   \def\bbl@provstring##1##2{%
1786     \providecommand##1{##2}%

```

```

1787 \bbl@toglobal##1}%
1788 \global\let\bbl@scafter\@empty
1789 \let\StartBabelCommands\bbl@startcmds
1790 \ifx\BabelLanguages\relax
1791 \let\BabelLanguages\CurrentOption
1792 \fi
1793 \begingroup
1794 \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1795 \StartBabelCommands}
1796 \def\bbl@startcmds{%
1797 \ifx\bbl@screset\@nnil\else
1798 \bbl@usehooks{stopcommands}{}%
1799 \fi
1800 \endgroup
1801 \begingroup
1802 \ifstar
1803 {\ifx\bbl@opt@strings\@nnil
1804 \let\bbl@opt@strings\BabelStringsDefault
1805 \fi
1806 \bbl@startcmds@i}%
1807 \bbl@startcmds@i}
1808 \def\bbl@startcmds@i#1#2{%
1809 \edef\bbl@L{\zap@space#1 \@empty}%
1810 \edef\bbl@G{\zap@space#2 \@empty}%
1811 \bbl@startcmds@ii}
1812 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of \SetString. There are two main cases, depending of if there is an optional argument: without it and strings=encoded, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and strings=encoded, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in strings=) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1813 \newcommand\bbl@startcmds@ii[1][\@empty]{%
1814 \let\SetString\@gobbletwo
1815 \let\bbl@stringdef\@gobbletwo
1816 \let\AfterBabelCommands\@gobble
1817 \ifx\@empty#1%
1818 \def\bbl@sc@label{generic}%
1819 \def\bbl@encstring##1##2{%
1820 \ProvideTextCommandDefault##1{##2}%
1821 \bbl@toglobal##1%
1822 \expandafter\bbl@toglobal\csname\string?\string##1\endcsname}%
1823 \let\bbl@sctest\in@true
1824 \else
1825 \let\bbl@sc@charset\space % <- zapped below
1826 \let\bbl@sc@fontenc\space % <- " "
1827 \def\bbl@tempa##1=##2\@nil{%
1828 \bbl@csarg\edef{sc\zap@space##1 \@empty}{##2 }}%
1829 \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1830 \def\bbl@tempa##1 ##2{% space -> comma
1831 ##1%
1832 \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1833 \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1834 \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1835 \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1836 \def\bbl@encstring##1##2{%
1837 \bbl@foreach\bbl@sc@fontenc{%
1838 \bbl@ifunset{T@###1}%
1839 }%

```

```

1840         {\ProvideTextCommand##1{####1}{##2}%
1841         \bbl@toglobal##1%
1842         \expandafter
1843         \bbl@toglobal\cscname####1\string##1\endcsname}}}%
1844     \def\bbl@scstest{%
1845         \bbl@xin@{\bbl@opt@strings,}{,\bbl@sc@label,\bbl@sc@fontenc,}}%
1846     \fi
1847     \ifx\bbl@opt@strings\@nnil          % i.e., no strings key -> defaults
1848     \else\ifx\bbl@opt@strings\relax      % i.e., strings=encoded
1849         \let\AfterBabelCommands\bbl@aftercmds
1850         \let\SetString\bbl@setstring
1851         \let\bbl@stringdef\bbl@encstring
1852     \else          % i.e., strings=value
1853     \bbl@scstest
1854     \ifin@
1855         \let\AfterBabelCommands\bbl@aftercmds
1856         \let\SetString\bbl@setstring
1857         \let\bbl@stringdef\bbl@provstring
1858     \fi\fi\fi
1859     \bbl@scswitch
1860     \ifx\bbl@G\@empty
1861         \def\SetString##1##2{%
1862             \bbl@error{missing-group}{##1}{}}}%
1863     \fi
1864     \ifx\@empty#1%
1865         \bbl@usehooks{defaultcommands}}}%
1866     \else
1867         \@expandtwoargs
1868         \bbl@usehooks{encodedcommands}{\bbl@sc@charset}{\bbl@sc@fontenc}}}%
1869     \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\<group>\<language>` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date<language>` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has been loaded).

```

1870 \def\bbl@forlang#1#2{%
1871     \bbl@for#1\bbl@L{%
1872         \bbl@xin@{,#1,}{,\BabelLanguages,}%
1873         \ifin@#2\relax\fi}}
1874 \def\bbl@scswitch{%
1875     \bbl@forlang\bbl@tempa{%
1876         \ifx\bbl@G\@empty\else
1877             \ifx\SetString@gobbletwo\else
1878                 \edef\bbl@GL{\bbl@G\bbl@tempa}%
1879                 \bbl@xin@{\bbl@GL,}{,\bbl@screset,}%
1880             \ifin@\else
1881                 \global\expandafter\let\cscname\bbl@GL\endcsname\@undefined
1882                 \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1883             \fi
1884         \fi
1885     \fi}}
1886 \AtEndOfPackage{%
1887     \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{}{#2}}}%
1888     \let\bbl@scswitch\relax}
1889 \@onlypreamble\EndBabelCommands
1890 \def\EndBabelCommands{%
1891     \bbl@usehooks{stopcommands}}}%
1892 \endgroup
1893 \endgroup

```

```

1894 \bbl@scafter}
1895 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1896 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1897 \bbl@forlang\bbl@tempa{%
1898 \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1899 \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1900 {\bbl@exp{%
1901 \global\bbl@add\<\bbl@G\bbl@tempa>{\bbl@scset\#1\<\bbl@LC>}}}%
1902 }%
1903 \def\BabelString{#2}%
1904 \bbl@usehooks{stringprocess}{}%
1905 \expandafter\bbl@stringdef
1906 \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1907 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1908 <<*Macros local to BabelCommands>> ≡
1909 \def\SetStringLoop##1##2{%
1910 \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1911 \count@\z@
1912 \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1913 \advance\count@\@ne
1914 \toks@\expandafter{\bbl@tempa}%
1915 \bbl@exp{%
1916 \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%
1917 \count@=\the\count@relax}}}%
1918 <</Macros local to BabelCommands>>

```

Delaying code Now the definition of \AfterBabelCommands when it is activated.

```

1919 \def\bbl@aftercmds#1{%
1920 \toks@\expandafter{\bbl@scafter#1}%
1921 \xdef\bbl@scafter{\the\toks@}}

```

Case mapping The command \SetCase is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1922 <<*Macros local to BabelCommands>> ≡
1923 \newcommand\SetCase[3][{}]{%
1924 \def\bbl@tempa####1####2{%
1925 \ifx####1\@empty\else
1926 \bbl@carg\bbl@add{extras\CurrentOption}{%
1927 \bbl@carg\babel@save{c__text_uppercase\_string####1_tl}%
1928 \bbl@carg\def{c__text_uppercase\_string####1_tl}{####2}%
1929 \bbl@carg\babel@save{c__text_lowercase\_string####2_tl}%
1930 \bbl@carg\def{c__text_lowercase\_string####2_tl}{####1}%
1931 \expandafter\bbl@tempa
1932 \fi}%
1933 \bbl@tempa##1\@empty\@empty
1934 \bbl@carg\bbl@tglobal{extras\CurrentOption}}}%
1935 <</Macros local to BabelCommands>>

```


Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1936 <<*Macros local to BabelCommands>> ≡
1937 \newcommand\SetHyphenMap[1]{%
1938   \bbl@forlang\bbl@tempa{%
1939     \expandafter\bbl@stringdef
1940     \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1941 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1942 \newcommand\BabelLower[2]{% one to one.
1943   \ifnum\lccode#1=#2\else
1944     \babel@savevariable{\lccode#1}%
1945     \lccode#1=#2\relax
1946   \fi}
1947 \newcommand\BabelLowerMM[4]{% many-to-many
1948   \@tempcnta=#1\relax
1949   \@tempcntb=#4\relax
1950   \def\bbl@tempa{%
1951     \ifnum\@tempcnta>#2\else
1952       \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1953       \advance\@tempcnta#3\relax
1954       \advance\@tempcntb#3\relax
1955       \expandafter\bbl@tempa
1956     \fi}%
1957   \bbl@tempa}
1958 \newcommand\BabelLowerM0[4]{% many-to-one
1959   \@tempcnta=#1\relax
1960   \def\bbl@tempa{%
1961     \ifnum\@tempcnta>#2\else
1962       \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1963       \advance\@tempcnta#3
1964       \expandafter\bbl@tempa
1965     \fi}%
1966   \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1967 <<*More package options>> ≡
1968 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1969 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap@ne}
1970 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1971 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@}
1972 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1973 <</More package options>>

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1974 \AtEndOfPackage{%
1975   \ifx\bbl@opt@hyphenmap\@undefined
1976     \bbl@xin@{,}{\bbl@language@opts}%
1977     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1978   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1979 \newcommand\setlocalecaption{%
1980   \@ifstar\bbl@setcaption@s\bbl@setcaption@x}
1981 \def\bbl@setcaption@x#1#2#3{% language caption-name string
1982   \bbl@trim@def\bbl@tempa{#2}%
1983   \bbl@xin@{.template}{\bbl@tempa}%
1984   \ifin@

```

```

1985 \bbl@ini@captions@template{#3}{#1}%
1986 \else
1987 \edef\bbl@tempd{%
1988 \expandafter\expandafter\expandafter
1989 \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
1990 \bbl@xin@
1991 {\expandafter\string\csname #2name\endcsname}%
1992 {\bbl@tempd}%
1993 \ifin@ % Renew caption
1994 \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
1995 \ifin@
1996 \bbl@exp{%
1997 \\bbl@ifsamestring{\bbl@tempa}{\language}%
1998 {\bbl@scset\<#2name>\<#1#2name>}%
1999 {}}%
2000 \else % Old way converts to new way
2001 \bbl@ifunset{#1#2name}%
2002 {\bbl@exp{%
2003 \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2004 \\bbl@ifsamestring{\bbl@tempa}{\language}%
2005 {\def\<#2name>\<#1#2name>}}%
2006 {}}}%
2007 {}%
2008 \fi
2009 \else
2010 \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2011 \ifin@ % New way
2012 \bbl@exp{%
2013 \\bbl@add\<captions#1>\bbl@scset\<#2name>\<#1#2name>}}%
2014 \\bbl@ifsamestring{\bbl@tempa}{\language}%
2015 {\bbl@scset\<#2name>\<#1#2name>}}%
2016 {}}}%
2017 \else % Old way, but defined in the new way
2018 \bbl@exp{%
2019 \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2020 \\bbl@ifsamestring{\bbl@tempa}{\language}%
2021 {\def\<#2name>\<#1#2name>}}%
2022 {}}}%
2023 \fi%
2024 \fi
2025 \@namedef{#1#2name}{#3}%
2026 \toks@ \expandafter\bbl@captionslist}%
2027 \bbl@exp{\in@{\<#2name>}{\the\toks@}}%
2028 \ifin@ \else
2029 \bbl@exp{\bbl@add\bbl@captionslist{\<#2name>}}%
2030 \bbl@tglobal\bbl@captionslist
2031 \fi
2032 \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2033 \bbl@trace{Macros related to glyphs}
2034 \def\set@low@box#1{\setbox\tw@ \hbox{,}\setbox\z@ \hbox{#1}%
2035 \dimen\z@ \ht\z@ \advance\dimen\z@ -\ht\tw@%
2036 \setbox\z@ \hbox{\lower\dimen\z@ \box\z@}\ht\z@ \ht\tw@ \dp\z@ \dp\tw@}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2037 \def\save@sf@q#1{\leavevmode
2038   \begingroup
2039     \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2040   \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2041 \ProvideTextCommand{\quotedblbase}{OT1}{%
2042   \save@sf@q{\set@low@box{\textquotedblright}/}%
2043   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2044 \ProvideTextCommandDefault{\quotedblbase}{%
2045   \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2046 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2047   \save@sf@q{\set@low@box{\textquoteright}/}%
2048   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2049 \ProvideTextCommandDefault{\quotesinglbase}{%
2050   \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```

2051 \ProvideTextCommand{\guillemetleft}{OT1}{%
2052   \ifmmode
2053     \ll
2054   \else
2055     \save@sf@q{\nobreak
2056       \raise.2ex\hbox{\scriptscriptstyle\ll}\bbl@allowhyphens}%
2057     \fi}
2058 \ProvideTextCommand{\guillemetright}{OT1}{%
2059   \ifmmode
2060     \gg
2061   \else
2062     \save@sf@q{\nobreak
2063       \raise.2ex\hbox{\scriptscriptstyle\gg}\bbl@allowhyphens}%
2064     \fi}
2065 \ProvideTextCommand{\guillemotleft}{OT1}{%
2066   \ifmmode
2067     \ll
2068   \else
2069     \save@sf@q{\nobreak
2070       \raise.2ex\hbox{\scriptscriptstyle\ll}\bbl@allowhyphens}%
2071     \fi}
2072 \ProvideTextCommand{\guillemotright}{OT1}{%
2073   \ifmmode
2074     \gg
2075   \else
2076     \save@sf@q{\nobreak
2077       \raise.2ex\hbox{\scriptscriptstyle\gg}\bbl@allowhyphens}%
2078     \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2079 \ProvideTextCommandDefault{\guillemetleft}{%
2080   \UseTextSymbol{OT1}{\guillemetleft}}

```

```

2081 \ProvideTextCommandDefault{\guillemetright}{%
2082   \UseTextSymbol{OT1}{\guillemetright}}
2083 \ProvideTextCommandDefault{\guillemotleft}{%
2084   \UseTextSymbol{OT1}{\guillemotleft}}
2085 \ProvideTextCommandDefault{\guillemotright}{%
2086   \UseTextSymbol{OT1}{\guillemotright}}

```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```

2087 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2088   \ifmmode
2089     <%
2090   \else
2091     \save@sf@q{\nobreak
2092       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2093   \fi}
2094 \ProvideTextCommand{\guilsinglright}{OT1}{%
2095   \ifmmode
2096     >%
2097   \else
2098     \save@sf@q{\nobreak
2099       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2100   \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2101 \ProvideTextCommandDefault{\guilsinglleft}{%
2102   \UseTextSymbol{OT1}{\guilsinglleft}}
2103 \ProvideTextCommandDefault{\guilsinglright}{%
2104   \UseTextSymbol{OT1}{\guilsinglright}}

```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter ‘ij’. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```

2105 \DeclareTextCommand{\ij}{OT1}{%
2106   i\kern-0.02em\bbl@allowhyphens j}
2107 \DeclareTextCommand{\IJ}{OT1}{%
2108   I\kern-0.02em\bbl@allowhyphens J}
2109 \DeclareTextCommand{\ij}{T1}{\char188}
2110 \DeclareTextCommand{\IJ}{T1}{\char156}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2111 \ProvideTextCommandDefault{\ij}{%
2112   \UseTextSymbol{OT1}{\ij}}
2113 \ProvideTextCommandDefault{\IJ}{%
2114   \UseTextSymbol{OT1}{\IJ}}

```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```

2115 \def\crrtic@{\hrule height0.1ex width0.3em}
2116 \def\crttic@{\hrule height0.1ex width0.33em}
2117 \def\ddj@{%
2118   \setbox0\hbox{d}\dimen@=\ht0
2119   \advance\dimen@lex
2120   \dimen@.45\dimen@
2121   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2122   \advance\dimen@ii.5ex
2123   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}

```

```

2124 \def\DDJ@{%
2125   \setbox0\hbox{D}\dimen@=.55\ht0
2126   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2127   \advance\dimen@ii.15ex % correction for the dash position
2128   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2129   \dimen\thr@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2130   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crttic@}}}}
2131 %
2132 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2133 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2134 \ProvideTextCommandDefault{\dj}{%
2135   \UseTextSymbol{OT1}{\dj}}
2136 \ProvideTextCommandDefault{\DJ}{%
2137   \UseTextSymbol{OT1}{\DJ}}

```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```

2138 \DeclareTextCommand{\SS}{OT1}{SS}
2139 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}

```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```

2140 \ProvideTextCommandDefault{\glq}{%
2141   \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}

```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```

2142 \ProvideTextCommand{\grq}{T1}{%
2143   \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}
2144 \ProvideTextCommand{\grq}{TU}{%
2145   \textormath{\textquoteleft}{\mbox{\textquoteleft}}}
2146 \ProvideTextCommand{\grq}{OT1}{%
2147   \save@sf@q{\kern-.0125em
2148     \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%
2149     \kern.07em\relax}}
2150 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}{\grq}}

```

\glqq

\grqq The ‘german’ double quotes.

```

2151 \ProvideTextCommandDefault{\glqq}{%
2152   \textormath{\quotedblbase}{\mbox{\quotedblbase}}}

```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```

2153 \ProvideTextCommand{\grqq}{T1}{%
2154   \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2155 \ProvideTextCommand{\grqq}{TU}{%
2156   \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2157 \ProvideTextCommand{\grqq}{OT1}{%
2158   \save@sf@q{\kern-.07em
2159     \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%
2160     \kern.07em\relax}}
2161 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}{\grqq}}

```

\flq

\frq The ‘french’ single guillemets.

```
2162 \ProvideTextCommandDefault{\flq}{%
2163   \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}
2164 \ProvideTextCommandDefault{\frq}{%
2165   \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

\flqq

\frqq The ‘french’ double guillemets.

```
2166 \ProvideTextCommandDefault{\flqq}{%
2167   \textormath{\guillemetleft}{\mbox{\guillemetleft}}}
2168 \ProvideTextCommandDefault{\frqq}{%
2169   \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command `\` needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```
2170 \def\umlauthigh{%
2171   \def\bbl@umlauta##1{\leavevmode\bgroup%
2172     \accent\csname\fontencoding dqpos\endcsname
2173     ##1\bbl@allowhyphens\egroup}%
2174   \let\bbl@umlaute\bbl@umlauta}
2175 \def\umlautlow{%
2176   \def\bbl@umlauta{\protect\lower@umlaut}}
2177 \def\umlautelow{%
2178   \def\bbl@umlaute{\protect\lower@umlaut}}
2179 \umlauthigh
```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra *(dimen)* register.

```
2180 \expandafter\ifx\csname U@D\endcsname\relax
2181   \csname newdimen\endcsname U@D
2182 \fi
```

The following code fools TeX’s `make_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we’ll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```
2183 \def\lower@umlaut#1{%
2184   \leavevmode\bgroup
2185   \U@D lex%
2186   {\setbox\z@\hbox{%
2187     \char\csname\fontencoding dqpos\endcsname}%
2188     \dimen@ -.45ex\advance\dimen@\ht\z@
2189     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2190   \accent\csname\fontencoding dqpos\endcsname
2191   \fontdimen5\font\U@D #1%
2192   \egroup}
```

For all vowels we declare `\` to be a composite command which uses `\bbl@umlauta` or `\bbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these

declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but babel sets them for *all* languages – you may want to redefine `\bbl@umlauta` and/or `\bbl@umlaute` for a language in the corresponding `ldf` (using the babel switching mechanism, of course).

```

2193 \AtBeginDocument{%
2194   \DeclareTextCompositeCommand{"}{OT1}{a}{\bbl@umlauta{a}}%
2195   \DeclareTextCompositeCommand{"}{OT1}{e}{\bbl@umlaute{e}}%
2196   \DeclareTextCompositeCommand{"}{OT1}{i}{\bbl@umlaute{i}}%
2197   \DeclareTextCompositeCommand{"}{OT1}{\i}{\bbl@umlaute{i}}%
2198   \DeclareTextCompositeCommand{"}{OT1}{o}{\bbl@umlauta{o}}%
2199   \DeclareTextCompositeCommand{"}{OT1}{u}{\bbl@umlauta{u}}%
2200   \DeclareTextCompositeCommand{"}{OT1}{A}{\bbl@umlauta{A}}%
2201   \DeclareTextCompositeCommand{"}{OT1}{E}{\bbl@umlaute{E}}%
2202   \DeclareTextCompositeCommand{"}{OT1}{I}{\bbl@umlaute{I}}%
2203   \DeclareTextCompositeCommand{"}{OT1}{O}{\bbl@umlauta{O}}%
2204   \DeclareTextCompositeCommand{"}{OT1}{U}{\bbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in Amharic.

```

2205 \ifx\l@english\undefined
2206   \chardef\l@english\z@
2207 \fi
2208 % The following is used to cancel rules in ini files (see Amharic).
2209 \ifx\l@unhyphenated\undefined
2210   \newlanguage\l@unhyphenated
2211 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2212 \bbl@trace{Bidi layout}
2213 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2214 \bbl@trace{Input engine specific macros}
2215 \ifcase\bbl@engine
2216   \input txtbabel.def
2217 \or
2218   \input luababel.def
2219 \or
2220   \input xebabel.def
2221 \fi
2222 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}
2223 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}
2224 \ifx\babelposthyphenation\undefined
2225   \let\babelposthyphenation\babelprehyphenation
2226   \let\babelpatterns\babelprehyphenation
2227   \let\babelcharproperty\babelprehyphenation
2228 \fi
2229 </package | core>

```

4.18. Creating and modifying languages

Continue with $\TeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an `ini` file. It may be used in conjunction to previously loaded `ldf` files.

```

2230 <{*package}>

```

```

2231 \bbl@trace{Creating languages and reading ini files}
2232 \let\bbl@extend@ini\@gobble
2233 \newcommand\babelprovide[2][\{%
2234   \let\bbl@savelangname\language
2235   \edef\bbl@savelocaleid{\the\localeid}%
2236   \global\let\bbl@afterload\@empty
2237   % Set name and locale id
2238   \edef\language{#2}%
2239   \bbl@id@assign
2240   % Initialize keys
2241   \bbl@vforeach{captions,date,import,main,script,language,%
2242     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2243     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2244     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2245     @import}%
2246     {\bbl@csarg\let{KVP@##1}\@nnil}%
2247   \global\let\bbl@release@transforms\@empty
2248   \global\let\bbl@release@casing\@empty
2249   \let\bbl@calendars\@empty
2250   \global\let\bbl@inidata\@empty
2251   \global\let\bbl@extend@ini\@gobble
2252   \global\let\bbl@included@inis\@empty
2253   \gdef\bbl@key@list{;}%
2254   \bbl@ifunset{bbl@passto@#2}%
2255     {\def\bbl@tempa{#1}}%
2256     {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2257 \expandafter\bbl@forkv\expandafter{\bbl@tempa}{%
2258   \in@{/}{##1}% With /, (re)sets a value in the ini
2259   \ifin@
2260     \bbl@renewinikey##1\@{##2}%
2261   \else
2262     \bbl@csarg\ifx{KVP@##1}\@nnil\else
2263       \bbl@error{unknown-provide-key}{##1}{}%
2264     \fi
2265     \bbl@csarg\def{KVP@##1}{##2}%
2266   \fi}%
2267 \chardef\bbl@howloaded=0:none; 1:ldf without ini; 2:ini
2268 \bbl@ifunset{date#2}\z{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2269 % == init ==
2270 \ifx\bbl@screset\undefined
2271   \bbl@ldfinit
2272 \fi
2273 % ==
2274 % If there is no import (last wins), use @import (internal, there
2275 % must be just one). To consider any order (because
2276 % \PassOptionsToLocale).
2277 \ifx\bbl@KVP@import\@nnil
2278   \let\bbl@KVP@import\bbl@KVP@@import
2279 \fi
2280 % == date (as option) ==
2281 % \ifx\bbl@KVP@date\@nnil\else
2282 % \fi
2283 % ==
2284 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2285 \ifcase\bbl@howloaded
2286   \let\bbl@lbkflag\@empty % new
2287 \else
2288   \ifx\bbl@KVP@hyphenrules\@nnil\else
2289     \let\bbl@lbkflag\@empty
2290   \fi
2291   \ifx\bbl@KVP@import\@nnil\else
2292     \let\bbl@lbkflag\@empty
2293   \fi

```



```

2294 \fi
2295 % == import, captions ==
2296 \ifx\bbbl@KVP@import\@nnil\else
2297   \bbbl@exp{\bbbl@ifblank{\bbbl@KVP@import}}%
2298   {\ifx\bbbl@initload\relax
2299     \begingroup
2300       \def\BabelBeforeIni##1##2{\gdef\bbbl@KVP@import{##1}\endinput}%
2301       \bbbl@input@texini{##2}%
2302     \endgroup
2303   \else
2304     \xdef\bbbl@KVP@import{\bbbl@initload}%
2305   \fi}%
2306 {}%
2307 \let\bbbl@KVP@date\@empty
2308 \fi
2309 \let\bbbl@KVP@captions@\bbbl@KVP@captions
2310 \ifx\bbbl@KVP@captions\@nnil
2311   \let\bbbl@KVP@captions\bbbl@KVP@import
2312 \fi
2313 % ==
2314 \ifx\bbbl@KVP@transforms\@nnil\else
2315   \bbbl@replace\bbbl@KVP@transforms{ }{,}%
2316 \fi
2317 % ==
2318 \ifx\bbbl@KVP@mapdot\@nnil\else
2319   \def\bbbl@tempa{\@empty}%
2320   \ifx\bbbl@KVP@mapdot\bbbl@tempa\else
2321     \bbbl@exp{\gdef<\bbbl@map@. @\language>{\bbbl@KVP@mapdot}}%
2322   \fi
2323 \fi
2324 % Load ini
2325 % -----
2326 \ifcase\bbbl@howloaded
2327   \bbbl@provide@new{#2}%
2328 \else
2329   \bbbl@ifblank{#1}%
2330   {}% With \bbbl@load@basic below
2331   {\bbbl@provide@renew{#2}}%
2332 \fi
2333 % Post tasks
2334 % -----
2335 % == subsequent calls after the first provide for a locale ==
2336 \ifx\bbbl@inidata\@empty\else
2337   \bbbl@extend@ini{#2}%
2338 \fi
2339 % == ensure captions ==
2340 \ifx\bbbl@KVP@captions\@nnil\else
2341   \bbbl@ifunset{\bbbl@extracaps@#2}%
2342   {\bbbl@exp{\bbbl@babelensure[exclude=\\today]{#2}}}%
2343   {\bbbl@exp{\bbbl@babelensure[exclude=\\today,
2344     include=\bbbl@extracaps@#2]{#2}}}%
2345   \let\bbbl@tempc\language
2346   \bbbl@replace\bbbl@tempc{-}{@}%
2347   \bbbl@ifunset{\bbbl@ensure@\bbbl@tempc}%
2348   {\bbbl@exp{%
2349     \\DeclareRobustCommand<\bbbl@ensure@\bbbl@tempc>[1]{%
2350       \\foreignlanguage{\language}%
2351       {###1}}}%
2352   {}%
2353   \bbbl@exp{%
2354     \\bbbl@tglobal<\bbbl@ensure@\bbbl@tempc>%
2355     \\bbbl@tglobal<\bbbl@ensure@\bbbl@tempc\space>%
2356 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2357 \bbl@load@basic{#2}%
2358 % == script, language ==
2359 % Override the values from ini or defines them
2360 \ifx\bbl@KVP@script\@nnil\else
2361   \bbl@csarg\edef{sname@#2}{\bbl@KVP@script}%
2362 \fi
2363 \ifx\bbl@KVP@language\@nnil\else
2364   \bbl@csarg\edef{lname@#2}{\bbl@KVP@language}%
2365 \fi
2366 \ifcase\bbl@engine\or
2367   \bbl@ifunset{\bbl@chrng@\languagename}{}%
2368   {\directlua{
2369     Babel.set_chranges_b('\bbl@cl{sbcpr}', '\bbl@cl{chrng}') }}%
2370 \fi
2371 % == Line breaking: intraspace, intrapenalty ==
2372 % For CJK, East Asian, Southeast Asian, if interspace in ini
2373 \ifx\bbl@KVP@intraspace\@nnil\else % We can override the ini or set
2374   \bbl@csarg\edef{intsp@#2}{\bbl@KVP@intraspace}%
2375 \fi
2376 \bbl@provide@intraspace
2377 % == Line breaking: justification ==
2378 \ifx\bbl@KVP@justification\@nnil\else
2379   \let\bbl@KVP@linebreaking\bbl@KVP@justification
2380 \fi
2381 \ifx\bbl@KVP@linebreaking\@nnil\else
2382   \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2383   {,elongated,kashida,cjk,padding,unhyphenated,}%
2384   \ifin@
2385     \bbl@csarg\xdef
2386       {\lnbrk@\languagename}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2387   \fi
2388 \fi
2389 \bbl@xin@{/e}{\,\bbl@cl{\lnbrk}}%
2390 \ifin@else\bbl@xin@{/k}{\,\bbl@cl{\lnbrk}}\fi
2391 \ifin@\bbl@arabicjust\fi
2392 \bbl@xin@{/p}{\,\bbl@cl{\lnbrk}}%
2393 \ifin@\AtBeginDocument{\@nameuse{\bbl@tibetanjust}}\fi
2394 % == Line breaking: hyphenate.other.(locale|script) ==
2395 \ifx\bbl@lbfkflag\@empty
2396   \bbl@ifunset{\bbl@hyotl@\languagename}{}%
2397   {\bbl@csarg\bbl@replace{\hyotl@\languagename}{ }{,}%
2398     \bbl@startcommands*{\languagename}{}%
2399     \bbl@csarg\bbl@foreach{\hyotl@\languagename}{%
2400       \ifcase\bbl@engine
2401         \ifnum##1<257
2402           \SetHyphenMap{\BabelLower{##1}{##1}}%
2403         \fi
2404       \else
2405         \SetHyphenMap{\BabelLower{##1}{##1}}%
2406       \fi}%
2407   \bbl@endcommands}%
2408 \bbl@ifunset{\bbl@hyots@\languagename}{}%
2409 {\bbl@csarg\bbl@replace{\hyots@\languagename}{ }{,}%
2410   \bbl@csarg\bbl@foreach{\hyots@\languagename}{%
2411     \ifcase\bbl@engine
2412       \ifnum##1<257
2413         \global\lccode##1=##1\relax
2414       \fi
2415     \else
2416       \global\lccode##1=##1\relax

```

```

2417         \fi}}%
2418 \fi
2419 % == Counters: maparabic ==
2420 % Native digits, if provided in ini (TeX level, xe and lua)
2421 \ifcase\bbbl@engine\else
2422     \bbbl@ifunset{\bbbl@dgnat@\language\language}\fi}%
2423     {\expandafter\ifx\csname \bbbl@dgnat@\language\language\endcsname\@empty\else
2424         \expandafter\expandafter\expandafter
2425         \bbbl@setdigits\csname \bbbl@dgnat@\language\language\endcsname
2426         \ifx\bbbl@KVP@maparabic\@nnil\else
2427             \ifx\bbbl@latinarabic\@undefined
2428                 \expandafter\let\expandafter\@arabic
2429                 \csname \bbbl@counter@\language\language\endcsname
2430             \else % i.e., if layout=counters, which redefines \@arabic
2431                 \expandafter\let\expandafter\bbbl@latinarabic
2432                 \csname \bbbl@counter@\language\language\endcsname
2433             \fi
2434         \fi
2435     \fi}%
2436 \fi
2437 % == Counters: mapdigits ==
2438 % > luababel.def
2439 % == Counters: alph, Alph ==
2440 \ifx\bbbl@KVP@alph\@nnil\else
2441     \bbbl@exp{%
2442         \\bbbl@add\<\bbbl@preextras@\language\language>{%
2443             \\bbabel@save\\@alph
2444             \let\\@alph\<\bbbl@cntr@\bbbl@KVP@alph @\language\language>}}}%
2445 \fi
2446 \ifx\bbbl@KVP@Alph\@nnil\else
2447     \bbbl@exp{%
2448         \\bbbl@add\<\bbbl@preextras@\language\language>{%
2449             \\bbabel@save\\@Alph
2450             \let\\@Alph\<\bbbl@cntr@\bbbl@KVP@Alph @\language\language>}}}%
2451 \fi
2452 % == Counters: mapdot ==
2453 \ifx\bbbl@KVP@mapdot\@nnil\else
2454     \bbbl@foreach\bbbl@list@the{%
2455         \bbbl@ifunset{the##1}\fi}%
2456         {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2457         \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}}%
2458         \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2459             \bbbl@exp{\gdef\<the##1>{\{the##1\}}}%
2460         \fi}}}%
2461 \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2462 \bbbl@foreach\bbbl@tempb{%
2463     \bbbl@ifunset{label##1}\fi}%
2464     {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2465     \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}}%
2466     \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2467         \bbbl@exp{\gdef\<label##1>{\{label##1\}}}%
2468     \fi}}}%
2469 \fi
2470 % == Casing ==
2471 \bbbl@release@casing
2472 \ifx\bbbl@KVP@casing\@nnil\else
2473     \bbbl@csarg\xdef{casing@\language\language}%
2474     {\@nameuse{\bbbl@casing@\language\language}\bbbl@maybextx\bbbl@KVP@casing}%
2475 \fi
2476 % == Calendars ==
2477 \ifx\bbbl@KVP@calendar\@nnil
2478     \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}}%
2479 \fi

```

```

2480 \def\bbl@tempe##1 ##2\@{% Get first calendar
2481   \def\bbl@tempa{##1}%
2482   \bbl@exp{\bbl@tempe\bbl@KVP@calendar\space\\@}%
2483 \def\bbl@tempe##1.##2.##3\@{%
2484   \def\bbl@tempc{##1}%
2485   \def\bbl@tempb{##2}%
2486 \expandafter\bbl@tempe\bbl@tempa..\@@
2487 \bbl@csarg\edef{calpr@\language\name}{%
2488   \ifx\bbl@tempc\empty\else
2489     calendar=\bbl@tempc
2490   \fi
2491   \ifx\bbl@tempb\empty\else
2492     ,variant=\bbl@tempb
2493   \fi}%
2494 % == engine specific extensions ==
2495 % Defined in XXXbabel.def
2496 \bbl@provide@extra{#2}%
2497 % == require.babel in ini ==
2498 % To load or reload the babel-*.tex, if require.babel in ini
2499 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2500   \bbl@ifunset{\bbl@rqtex@\language\name}{}%
2501     {\expandafter\ifx\csname\bbl@rqtex@\language\name\endcsname\empty\else
2502       \let\BabelBeforeIni@gobbletwo
2503       \chardef\atcatcode=\catcode\@
2504       \catcode\@=11\relax
2505       \def\CurrentOption{#2}%
2506       \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2507       \catcode\@=\atcatcode
2508       \let\atcatcode\relax
2509       \global\bbl@csarg\let{rqtex@\language\name}\relax
2510     \fi}%
2511 \bbl@foreach\bbl@calendars{%
2512   \bbl@ifunset{\bbl@ca##1}{%
2513     \chardef\atcatcode=\catcode\@
2514     \catcode\@=11\relax
2515     \InputIfFileExists{babel-ca-##1.tex}{}}}%
2516   \catcode\@=\atcatcode
2517   \let\atcatcode\relax}%
2518 {}}%
2519 \fi
2520 % == frenchspacing ==
2521 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2522 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2523 \ifin@
2524   \bbl@extras@wrap{\bbl@pre@fs}%
2525   {\bbl@pre@fs}%
2526   {\bbl@post@fs}%
2527 \fi
2528 % == transforms ==
2529 % > luababel.def
2530 \def\CurrentOption{#2}%
2531 \@nameuse{\bbl@icsave@#2}%
2532 % == main ==
2533 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2534   \let\language\bbl@savelangname
2535   \chardef\localeid\bbl@savelocaleid\relax
2536 \fi
2537 % == ==
2538 \ifx\ldf@finish\@onlypreamble\else
2539   \bbl@afterload
2540 \fi
2541 % == hyphenrules (apply if current) ==
2542 \ifx\bbl@KVP@hyphenrules\@nnil\else

```

```

2543 \ifnum\bbl@savelocaleid=\localeid
2544 \language\@nameuse{l\@languagename}%
2545 \fi
2546 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2547 \def\bbl@provide@new#1{%
2548 \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2549 \@namedef{extras#1}{}%
2550 \@namedef{noextras#1}{}%
2551 \bbl@startcommands*{#1}{captions}%
2552 \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2553 \def\bbl@tempb##1{% elt for \bbl@captionslist
2554 \ifx##1\@nnil\else
2555 \bbl@exp{%
2556 \\SetString\\##1{%
2557 \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2558 \expandafter\bbl@tempb
2559 \fi}%
2560 \expandafter\bbl@tempb\bbl@captionslist\@nnil
2561 \else
2562 \ifx\bbl@initoload\relax
2563 \bbl@read@ini{\bbl@KVP@captions}2% % Here letters cat = 11
2564 \else
2565 \bbl@read@ini{\bbl@initoload}2% % Same
2566 \fi
2567 \fi
2568 \StartBabelCommands*{#1}{date}%
2569 \ifx\bbl@KVP@date\@nnil
2570 \bbl@exp{%
2571 \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2572 \else
2573 \bbl@savetoday
2574 \bbl@savedate
2575 \fi
2576 \bbl@endcommands
2577 \bbl@load@basic{#1}%
2578 % == hyphenmins == (only if new)
2579 \bbl@exp{%
2580 \gdef\<#1hyphenmins>{%
2581 {\bbl@ifunset{\bbl@lfthm#1}{2}{\bbl@cs{lfthm#1}}}%
2582 {\bbl@ifunset{\bbl@rgthm#1}{3}{\bbl@cs{rgthm#1}}}}}%
2583 % == hyphenrules (also in renew) ==
2584 \bbl@provide@hyphens{#1}%
2585 % == main ==
2586 \ifx\bbl@KVP@main\@nnil\else
2587 \expandafter\main@language\expandafter{#1}%
2588 \fi}
2589 %
2590 \def\bbl@provide@renew#1{%
2591 \ifx\bbl@KVP@captions\@nnil\else
2592 \StartBabelCommands*{#1}{captions}%
2593 \bbl@read@ini{\bbl@KVP@captions}2% % Here all letters cat = 11
2594 \EndBabelCommands
2595 \fi
2596 \ifx\bbl@KVP@date\@nnil\else
2597 \StartBabelCommands*{#1}{date}%
2598 \bbl@savetoday
2599 \bbl@savedate
2600 \EndBabelCommands
2601 \fi
2602 % == hyphenrules (also in new) ==

```

```

2603 \ifx\bbbl@lbkflag\@empty
2604   \bbbl@provide@hyphens{#1}%
2605 \fi
2606 % == main ==
2607 \ifx\bbbl@KVP@main\@nnil\else
2608   \expandafter\main@language\expandafter{#1}%
2609 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2610 \def\bbbl@load@basic#1{%
2611   \ifcase\bbbl@howloaded\or\or
2612     \ifcase\csname bbl@llevel@\language\endcsname
2613       \bbbl@csarg\let\lname@\language\relax
2614     \fi
2615   \fi
2616   \bbbl@ifunset{bbbl@lname@#1}%
2617   {\def\BabelBeforeIni##1##2{%
2618     \begingroup
2619       \let\bbbl@ini@captions@aux\@gobbletwo
2620       \def\bbbl@inidate #####1.####2.####3.####4\relax #####5####6{%
2621         \bbbl@read@ini{##1}1%
2622         \ifx\bbbl@initoload\relax\endinput\fi
2623       \endgroup}%
2624     \begingroup           % boxed, to avoid extra spaces:
2625       \ifx\bbbl@initoload\relax
2626         \bbbl@input@texini{#1}%
2627       \else
2628         \setbox\z@\hbox{\BabelBeforeIni{\bbbl@initoload}{}}%
2629       \fi
2630     \endgroup}%
2631   {}}

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2632 \def\bbbl@load@info#1{%
2633   \def\BabelBeforeIni##1##2{%
2634     \begingroup
2635       \bbbl@read@ini{##1}0%
2636       \endinput           % babel- .tex may contain onlypreamble's
2637     \endgroup}%          boxed, to avoid extra spaces:
2638   {\bbbl@input@texini{#1}}}

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2639 \def\bbbl@provide@hyphens#1{%
2640   \@tempcnta\m@ne % a flag
2641   \ifx\bbbl@KVP@hyphenrules\@nnil\else
2642     \bbbl@replace\bbbl@KVP@hyphenrules{ }{,}%
2643     \bbbl@foreach\bbbl@KVP@hyphenrules{%
2644       \ifnum\@tempcnta=\m@ne % if not yet found
2645         \bbbl@ifsamestring{##1}{+}%
2646         {\bbbl@carg\addlanguage{l@#1}}%
2647       }%
2648       \bbbl@ifunset{l@#1}% After a possible +
2649     }%
2650     {\@tempcnta\@nameuse{l@#1}}%
2651   \fi}%
2652   \ifnum\@tempcnta=\m@ne
2653     \bbbl@warning{%
2654       Requested 'hyphenrules' for '\language' not found:\%

```

```

2655      \bbl@KVP@hyphenrules.\\%
2656      Using the default value. Reported}%
2657  \fi
2658  \fi
2659  \ifnum\@tempcnta=\m@ne          % if no opt or no language in opt found
2660    \ifx\bbl@KVP@captions@@\@nnil
2661      \bbl@ifunset{\bbl@hyphr@#1}{}% use value in ini, if exists
2662      {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2663       {}}%
2664      {\bbl@ifunset{\l@bbl@cl{hyphr}}}%
2665      {}}%          if hyphenrules found:
2666      {\@tempcnta\@nameuse{\l@bbl@cl{hyphr}}}}}%
2667  \fi
2668  \fi
2669  \bbl@ifunset{\l@#1}%
2670  {\ifnum\@tempcnta=\m@ne
2671   \bbl@carg\adddialect{\l@#1}\language
2672   \else
2673   \bbl@carg\adddialect{\l@#1}\@tempcnta
2674   \fi}%
2675  {\ifnum\@tempcnta=\m@ne\else
2676   \global\bbl@carg\chardef{\l@#1}\@tempcnta
2677   \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2678 \def\bbl@input@texini#1{%
2679   \bbl@bsphack
2680   \bbl@exp{%
2681     \catcode`\\%=14 \catcode`\\%=0
2682     \catcode`\\={1 \catcode`\\}=2
2683     \lowercase{\InputIfFileExists{babel-#1.tex}}{}}%
2684     \catcode`\\%=the\catcode`\%relax
2685     \catcode`\\%=the\catcode`\%relax
2686     \catcode`\\={the\catcode`\%relax
2687     \catcode`\\}=the\catcode`\%relax}%
2688   \bbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```

2689 \def\bbl@inline#1\bbl@inline{%
2690   \ifnextchar[\bbl@iniset{\@ifnextchar;\bbl@iniskip\bbl@inistore}#1\@}% ]
2691 \def\bbl@iniset[#1]#2\@{\def\bbl@section{#1}}
2692 \def\bbl@iniskip#1\@{\%          if starts with ;
2693 \def\bbl@inistore#1=#2\@{\%          full (default)
2694   \bbl@trim@def\bbl@tempa{#1}%
2695   \bbl@trim\toks@{#2}%
2696   \bbl@ifsamestring{\bbl@tempa}{\include}%
2697   {\bbl@read@subini{\the\toks@}}%
2698   {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2699   \ifin@else
2700     \bbl@xin@{,identification/include.}%
2701     {,\bbl@section/\bbl@tempa}%
2702     \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2703     \bbl@exp{%
2704       \\g@addto@macro\\bbl@inidata{%
2705         \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2706     \fi}}
2707 \def\bbl@inistore@min#1=#2\@{\% minimal (maybe set in \bbl@read@ini)
2708   \bbl@trim@def\bbl@tempa{#1}%
2709   \bbl@trim\toks@{#2}%
2710   \bbl@xin@{.identification.}{.\bbl@section.}%
2711   \ifin@

```

```

2712 \bbl@exp{\g@addto@macro\bbbl@inidata{%
2713 \bbbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2714 \fi}

```

4.19. Main loop in ‘provide’

Now, the ‘main loop’, `\bbl@read@ini`, which **must be executed inside a group**. At this point, `\bbbl@inidata` may contain data declared in `\babelprovide`, with ‘slashed’ keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, ‘export’ some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with `\babelprovide` it’s either 1 (without import) or 2 (which import). The value `−1` is used with `\DocumentMetadata`.

`\bbl@loop@ini` is the reader, line by line (1: stream), and calls `\bbl@iniline` to save the key/value pairs. If `\bbbl@inistore` finds the `@include` directive, the input stream is switched temporarily and `\bbl@read@subini` is called.

When the language is being set based on the document metadata (#2 in `\bbl@read@ini` is `−1`), there is an interlude to get the name, after the data have been collected, and before it’s processed.

```

2715 \def\bbl@loop@ini#1{%
2716 \loop
2717 \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2718 \endlinechar\m@ne
2719 \read#1 to \bbl@line
2720 \endlinechar\^^M
2721 \ifx\bbl@line\empty\else
2722 \expandafter\bbl@iniline\bbl@line\bbl@iniline
2723 \fi
2724 \repeat}
2725 %
2726 \def\bbl@read@subini#1{%
2727 \ifx\bbl@readsubstream\undefined
2728 \csname newread\endcsname\bbl@readsubstream
2729 \fi
2730 \openin\bbl@readsubstream=babel-#1.ini
2731 \ifeof\bbl@readsubstream
2732 \bbl@error{no-ini-file}{#1}{}}%
2733 \else
2734 {\bbl@loop@ini\bbl@readsubstream}%
2735 \fi
2736 \closein\bbl@readsubstream}
2737 %
2738 \ifx\bbl@readstream\undefined
2739 \csname newread\endcsname\bbl@readstream
2740 \fi
2741 \def\bbl@read@ini#1#2{%
2742 \global\let\bbl@extend@ini@gobble
2743 \openin\bbl@readstream=babel-#1.ini
2744 \ifeof\bbl@readstream
2745 \bbl@error{no-ini-file}{#1}{}}%
2746 \else
2747 % == Store ini data in \bbl@inidata ==
2748 \catcode\ =10 \catcode\ =12
2749 \catcode\ [=12 \catcode\ ]=12 \catcode\ ==12 \catcode\ &=12
2750 \catcode\ ;=12 \catcode\ |=12 \catcode\ %=14 \catcode\ -=12
2751 \ifnum#2=\m@ne % Just for the info
2752 \edef\language{tag \bbl@metalang}%
2753 \fi
2754 \ifnum#2=\m@ne
2755 \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.}%
2756 Fetching the locale name from babel-#1.ini@gobble}%
2757 \else
2758 \bbl@info{Importing
2759 \ifcase#2font and identification \or basic \fi

```



```

2760             data for '\language'\%
2761             from babel-#1.ini. Reported}%
2762 \fi
2763 \ifnum#2<\@ne
2764     \global\let\bbl@inidata\@empty
2765     \let\bbl@inistore\bbl@inistore@min % Remember it's local
2766 \fi
2767 \def\bbl@section{identification}%
2768 \bbl@exp{%
2769     \\bbl@inistore tag.ini=#1\\@@
2770     \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\@@}%
2771 \bbl@loop@ini\bbl@readstream
2772 % == Process stored data ==
2773 \ifnum#2=\m@ne
2774     \def\bbl@tempa##1 ##2\@@{##1}% Get first name
2775     \def\bbl@elt##1##2##3{%
2776         \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2777         {\edef\language{\bbl@tempa##3 \@@}%
2778          \let\localename\language
2779          \bbl@id@assign
2780          \def\bbl@elt###1####2####3{}}%
2781         {}}%
2782     \bbl@inidata
2783 \fi
2784 \bbl@csarg\xdef{lini@\language}{#1}%
2785 \bbl@read@ini@aux
2786 % == 'Export' data ==
2787 \bbl@ini@exports{#2}%
2788 \global\bbl@csarg\let{inidata@\language}\bbl@inidata
2789 \global\let\bbl@inidata\@empty
2790 \bbl@xin@{,\language,},{,\bbl@ini@loaded,}%
2791 \ifin@else
2792     \bbl@exp{\\bbl@add@list\\bbl@ini@loaded{\language}}%
2793     \bbl@tglobal\bbl@ini@loaded
2794 \fi
2795 \@ifpackagewith{babel}{licr=unextended}{}%
2796     {\ifcase\bbl@engine\else % Find a better place
2797         \bbl@xin@{,\bbl@cl{sbc},},{,CyrL,}%
2798         \ifin@
2799             \bbl@once{licr-cryl}{\g@addto@macro\bbl@afterload{\input{cyrLuni.def}}}%
2800         \fi
2801     \fi}%
2802 \fi
2803 \closein\bbl@readstream}
2804 \def\bbl@read@ini@aux{%
2805     \let\bbl@savestrings\@empty
2806     \let\bbl@savetoday\@empty
2807     \let\bbl@savestate\@empty
2808     \def\bbl@elt##1##2##3{%
2809         \def\bbl@section{##1}%
2810         \in@{=date.}{=##1}% Find a better place
2811         \ifin@
2812             \bbl@ifunset{bbl@inikv@##1}%
2813             {\bbl@ini@calendar{##1}}%
2814             {}%
2815         \fi
2816         \bbl@ifunset{bbl@inikv@##1}{}%
2817         {\csname bbl@inikv@##1\endcsname{##2}{##3}}}%
2818     \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2819 \def\bbl@extend@ini@aux#1{%

```

```

2820 \bbl@startcommands*{#1}{captions}%
2821 % Activate captions/... and modify exports
2822 \bbl@csarg\def\inikv@captions.licr{##1##2}%
2823 \setlocalecaption{#1}{##1}{##2}}%
2824 \def\bbl@inikv@captions##1##2{%
2825 \bbl@ini@captions@aux{##1}{##2}}%
2826 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2827 \def\bbl@exportkey##1##2##3{%
2828 \bbl@ifunset\bbl@kv@##2}{%
2829 {\expandafter\ifx\csname bbl@kv@##2\endcsname\@empty\else
2830 \bbl@exp{\global\let\<bbl@##1@language\>\<bbl@kv@##2>}}%
2831 \fi}}%
2832 % As with \bbl@read@ini, but with some changes
2833 \bbl@read@ini@aux
2834 \bbl@ini@exports\tw@
2835 % Update inidata@lang by pretending the ini is read.
2836 \def\bbl@elt##1##2##3{%
2837 \def\bbl@section{##1}%
2838 \bbl@iniline##2=##3\bbl@iniline}%
2839 \csname bbl@inidata@#1\endcsname
2840 \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2841 \StartBabelCommands*{#1}{date}% And from the import stuff
2842 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2843 \bbl@savetoday
2844 \bbl@savedate
2845 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2846 \def\bbl@ini@calendar#1{%
2847 \lowercase{\def\bbl@tempa{= #1 =}}%
2848 \bbl@replace\bbl@tempa{=date.gregorian}{}%
2849 \bbl@replace\bbl@tempa{=date.}{}%
2850 \in@{.licr=}{#1=}%
2851 \ifin@
2852 \ifcase\bbl@engine
2853 \bbl@replace\bbl@tempa{.licr=}{}%
2854 \else
2855 \let\bbl@tempa\relax
2856 \fi
2857 \fi
2858 \ifx\bbl@tempa\relax\else
2859 \bbl@replace\bbl@tempa{=}{}%
2860 \ifx\bbl@tempa\@empty\else
2861 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2862 \fi
2863 \bbl@exp{%
2864 \def\<bbl@inikv@#1>####1####2{%
2865 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2866 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2867 \def\bbl@renewinikey#1/#2\@#3{%
2868 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2869 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2870 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2871 \bbl@trim\toks@{#3}% value
2872 \bbl@exp{%
2873 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2874 \\\g@addto@macro\\bbl@inidata{%
2875 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2876 \def\bbl@exportkey#1#2#3{%
2877   \bbl@ifunset{\bbl@kv@#2}%
2878   {\bbl@csarg\gdef{\bbl@#1@{language}}{#3}}%
2879   {\expandafter\ifx\csname\bbl@kv@#2\endcsname\@empty
2880     \bbl@csarg\gdef{\bbl@#1@{language}}{#3}%
2881     \else
2882     \bbl@exp{\global\let<\bbl@#1@{language}>\<\bbl@kv@#2>}%
2883     \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note `\bbl@ini@exports` is called always (via `\bbl@inisec`), while `\bbl@after@ini` must be called explicitly after `\bbl@read@ini` if necessary.

Although BCP 47 doesn't treat 'x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by `onchar`; the language system is set with the names, and then `fontspec` maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in `pdftex` to select a font encoding valid (and preloaded) for a language loaded on the fly.

```

2884 \def\bbl@iniwarning#1{%
2885   \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2886   {\bbl@warning%
2887     From babel-\bbl@cs{lini@{language}}.ini:\\%
2888     \bbl@cs{kv@identification.warning#1}\\%
2889     Reported}}%
2890 %
2891 \let\bbl@release@transforms\@empty
2892 \let\bbl@release@casing\@empty

```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```

2893 \def\bbl@ini@exports#1{%
2894   % Identification always exported
2895   \bbl@iniwarning}%
2896   \ifcase\bbl@engine
2897     \bbl@iniwarning{.pdflatex}%
2898   \or
2899     \bbl@iniwarning{.lualatex}%
2900   \or
2901     \bbl@iniwarning{.xelatex}%
2902   \fi%
2903   \bbl@exportkey{lllevel}{identification.load.level}{}%
2904   \bbl@exportkey{elname}{identification.name.english}{}%
2905   \bbl@exp{\bbl@exportkey{lname}{identification.name.opentype}%
2906     {\csname\bbl@elname@{language}\endcsname}}%
2907   \bbl@exportkey{tbcpl}{identification.tag.bcp47}{}%
2908   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2909   \bbl@exportkey{lbcpl}{identification.language.tag.bcp47}{}%
2910   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2911   \bbl@exportkey{esname}{identification.script.name}{}%
2912   \bbl@exp{\bbl@exportkey{sname}{identification.script.name.opentype}%
2913     {\csname\bbl@esname@{language}\endcsname}}%
2914   \bbl@exportkey{sbcp}{identification.script.tag.bcp47}{}%
2915   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2916   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2917   \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2918   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%

```

```

2919 \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2920 \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2921 % Also maps bcp47 -> languagename
2922 \bbl@csarg\xdef{bcp@map@{bbl@cl{tbc}}}{\languagename}%
2923 \ifcase\bbl@engine\or
2924   \directlua{%
2925     Babel.locale_props[\the\bbl@cs{id@}\languagename]].script
2926     = '\bbl@cl{sbcp}'}%
2927 \fi
2928 % Conditional
2929 \ifnum#1>\z@      % -1 or 0 = only info, 1 = basic, 2 = (re)new
2930   \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2931   \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2932   \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2933   \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%
2934   \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2935   \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2936   \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2937   \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2938   \bbl@exportkey{intsp}{typography.intraspace}{}%
2939   \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2940   \bbl@exportkey{chrng}{characters.ranges}{}%
2941   \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2942   \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2943   \ifnum#1=\tw@      % only (re)new
2944     \bbl@exportkey{rqtex}{identification.require.babel}{}%
2945     \bbl@tglobal\bbl@savetoday
2946     \bbl@tglobal\bbl@savestate
2947     \bbl@savestrings
2948   \fi
2949 \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@{section}.<key>.

```

2950 \def\bbl@inikv#1#2{%      key=value
2951   \toks@{#2}%              This hides #'s from ini values
2952   \bbl@csarg\edef{kv@{bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2953 \let\bbl@inikv@identification\bbl@inikv
2954 \let\bbl@inikv@date\bbl@inikv
2955 \let\bbl@inikv@typography\bbl@inikv
2956 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2957 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@{languagename}}@empty x-\fi}
2958 \def\bbl@inikv@characters#1#2{%
2959   \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2960   {\bbl@exp{%
2961     \\g@addto@macro\\bbl@release@casing{%
2962       \\bbl@casemapping}{\languagename}{\unexpanded{#2}}}%
2963   {\in@{casing.}{#1}% e.g., casing.Uv = uV
2964     \ifin@
2965       \lowercase{\def\bbl@tempb{#1}}%
2966       \bbl@replace\bbl@tempb{casing.}%
2967       \bbl@exp{\\g@addto@macro\\bbl@release@casing{%
2968         \\bbl@casemapping
2969         {\\bbl@maybextx\bbl@tempb}{\languagename}{\unexpanded{#2}}}%
2970       \else
2971       \bbl@inikv{#1}{#2}%

```

2972 \fi}}

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2973 \def\bbl@inikv@counters#1#2{%
2974   \bbl@ifsamestring{#1}{digits}%
2975   {\bbl@error{digits-is-reserved}{}}{}%
2976   }%
2977 \def\bbl@tempc{#1}%
2978 \bbl@trim@def{\bbl@tempb*}{#2}%
2979 \in@{.1$}{#1$}%
2980 \ifin@
2981   \bbl@replace\bbl@tempc{.1}{}%
2982   \bbl@csarg\protected@xdef{cnt@#1\bbl@tempc @\language}{%
2983     \noexpand\bbl@alphanumeric{\bbl@tempc}}%
2984 \fi
2985 \in@{.F.}{#1}%
2986 \ifin@else\in@{.S.}{#1}\fi
2987 \ifin@
2988   \bbl@csarg\protected@xdef{cnt@#1@\language}{\bbl@tempb*}%
2989 \else
2990   \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
2991   \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
2992   \bbl@csarg{\global\expandafter\let}{cnt@#1@\language}\bbl@tempa
2993 \fi
2994 \in@{.D$}{#1$}%
2995 \ifin@else\in@{.C$}{#1$}\fi
2996 \ifin@
2997   \bbl@replace\bbl@tempc{.D}{}%
2998   \bbl@replace\bbl@tempc{.C}{}%
2999   \bbl@exp{\gdef\<bbl@cnt@#1\bbl@tempc @\language>####1{%
3000     \ \expandafter\ \bbl@zhnumeral\ \expandafter{\ \number####1}{\bbl@tempc}}}%
3001 \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3002 \ifcase\bbl@engine
3003   \bbl@csarg\def{inikv@captions.licr}#1#2{%
3004     \bbl@ini@captions@aux{#1}{#2}}
3005 \else
3006   \def\bbl@inikv@captions#1#2{%
3007     \bbl@ini@captions@aux{#1}{#2}}
3008 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3009 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3010   \bbl@replace\bbl@tempa{.template}{}%
3011   \def\bbl@toreplace{#1}{}%
3012   \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3013   \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3014   \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3015   \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}%
3016   \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3017   \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3018   \ifin@
3019     \@nameuse{\bbl@patch\bbl@tempa}%
3020     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3021   \fi
3022   \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3023   \ifin@
3024     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3025   \bbl@exp{\gdef\<fnun@\bbl@tempa>{%

```

```

3026      \\bbl@ifunset{bbl@bbl@tempa fmt@\\language}%
3027      {[fnum@bbl@tempa]}%
3028      {\\@nameuse{bbl@bbl@tempa fmt@\\language}}}%
3029 \fi}
3030 %
3031 \def\bbl@ini@captions@aux#1#2{%
3032   \bbl@trim@def\bbl@tempa{#1}%
3033   \bbl@xin@{.template}{\bbl@tempa}%
3034   \ifin@
3035     \bbl@ini@captions@template{#2}\language
3036   \else
3037     \bbl@ifblank{#2}%
3038     {\bbl@exp{%
3039       \toks@{\\bbl@nocaption{\bbl@tempa name}\language\bbl@tempa name}}}%
3040     {\bbl@trim\toks@{#2}}}%
3041   \bbl@exp{%
3042     \\bbl@add\\bbl@savestrings{%
3043       \\SetString<\bbl@tempa name>{\the\toks@}}%
3044     \toks@expandafter{\bbl@captionslist}%
3045     \bbl@exp{\\in@{\<\bbl@tempa name>}\the\toks@}}%
3046   \ifin@else
3047     \bbl@exp{%
3048       \\bbl@add<\bbl@extracaps@language>{\<\bbl@tempa name>}%
3049       \\bbl@tglobal<\bbl@extracaps@language>}%
3050   \fi
3051 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3052 \def\bbl@list@the{%
3053   part,chapter,section,subsection,subsubsection,paragraph,%
3054   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3055   table,page,footnote,mpfootnote,mpfn}
3056 %
3057 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3058   \bbl@ifunset{bbl@map@#1@language}%
3059   {\@nameuse{#1}}%
3060   {\@nameuse{bbl@map@#1@language}}%
3061 %
3062 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3063   \ifincsname#1\else
3064     \bbl@ifunset{bbl@map@#1@language}%
3065     {#1}%
3066     {\@nameuse{bbl@map@#1@language}}%
3067   \fi}
3068 %
3069 \def\bbl@inikv@labels#1#2{%
3070   \in@{.map}{#1}%
3071   \ifin@
3072     \in@{,dot.map,}{, #1,}%
3073   \ifin@
3074     \global\@namedef{bbl@map@.@@@language}{#2}%
3075   \fi
3076   \ifx\bbl@KVP@labels\@nnil\else
3077     \bbl@xin@{ map }{ \bbl@KVP@labels\space}%
3078     \ifin@
3079       \def\bbl@tempc{#1}%
3080       \bbl@replace\bbl@tempc{.map}{}%
3081       \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3082       \bbl@exp{%
3083         \gdef<\bbl@map@bbl@tempc @language>%
3084         {\ifin@<#2>\else\\loccounter{#2}\fi}}%
3085       \bbl@foreach\bbl@list@the{%
3086         \bbl@ifunset{the##1}{%

```

```

3087      {\bbl@ncarg\let\bbl@tempd{the##1}%
3088      \bbl@exp{%
3089        \\bbl@sreplace\<the##1>%
3090        {\<\bbl@tempc>{##1}}%
3091        {\\bbl@map@cnt{\bbl@tempc}{##1}}%
3092        \\bbl@sreplace\<the##1>%
3093        {\<\empty @\bbl@tempc>\<c@##1>%
3094        {\\bbl@map@cnt{\bbl@tempc}{##1}}%
3095        \\bbl@sreplace\<the##1>%
3096        {\\csname @\bbl@tempc\\endcsname\<c@##1>%
3097        {\\bbl@map@cnt{\bbl@tempc}{##1}}}%
3098      \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3099      \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3100      \fi}%
3101    \fi
3102  \fi
3103 %
3104 \else
3105   % The following code is still under study. You can test it and make
3106   % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3107   % language dependent.
3108   \in@{enumerate.}{#1}%
3109   \ifin@
3110     \def\bbl@tempa{#1}%
3111     \bbl@replace\bbl@tempa{enumerate.}{}%
3112     \def\bbl@toreplace{#2}%
3113     \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace{}}%
3114     \bbl@replace\bbl@toreplace{[]}{\csname the}%
3115     \bbl@replace\bbl@toreplace{[]}{\endcsname{}}%
3116     \toks@{\expandafter\bbl@toreplace}%
3117     \bbl@exp{%
3118       \\bbl@add\<extras\language>{%
3119       \\babel@save\<labelenum\romannumeral\bbl@tempa>%
3120       \def\<labelenum\romannumeral\bbl@tempa>{\the\toks@}%
3121       \\bbl@toggle\<extras\language>}%
3122     \fi
3123   \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3124 \def\bbl@chapttype{chapter}
3125 \ifx\@makechapterhead\undefined
3126   \let\bbl@patchchapter\relax
3127 \else\ifx\thechapter\undefined
3128   \let\bbl@patchchapter\relax
3129 \else\ifx\ps@headings\undefined
3130   \let\bbl@patchchapter\relax
3131 \else
3132   \def\bbl@patchchapter{%
3133     \global\let\bbl@patchchapter\relax
3134     \gdef\bbl@chfmt{%
3135       \bbl@ifunset{\bbl@\bbl@chapttype fmt@\language}%
3136       {\@chapapp\space\thechapter}%
3137       {\@nameuse{\bbl@\bbl@chapttype fmt@\language}}}%
3138     \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3139     \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3140     \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3141     \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3142     \bbl@toggle\appendix
3143     \bbl@toggle\ps@headings
3144     \bbl@toggle\chaptermark

```

```

3145 \bbl@toglobal\@makechapterhead}
3146 \let\bbl@patchappendix\bbl@patchchapter
3147 \fi\fi\fi
3148 \ifx\@part\@undefined
3149 \let\bbl@patchpart\relax
3150 \else
3151 \def\bbl@patchpart{%
3152 \global\let\bbl@patchpart\relax
3153 \gdef\bbl@partformat{%
3154 \bbl@ifunset{bbl@partfmt@\language}%
3155 {\partname\nobreakspace\thepart}%
3156 {\@nameuse{bbl@partfmt@\language}}}%
3157 \bbl@sreplace\@part{\partname\nobreakspace\thepart}{\bbl@partformat}%
3158 \bbl@toglobal\@part}
3159 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3160 \let\bbl@calendar\@empty
3161 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3162 \def\bbl@localedate#1#2#3#4{%
3163 \begingroup
3164 \edef\bbl@they{#2}%
3165 \edef\bbl@them{#3}%
3166 \edef\bbl@thed{#4}%
3167 \edef\bbl@tempe{%
3168 \bbl@ifunset{bbl@calpr@\language}{\bbl@cl{calpr}},%
3169 #1}%
3170 \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3171 \bbl@replace\bbl@tempe{ }{}%
3172 \bbl@replace\bbl@tempe{convert}{convert=}%
3173 \let\bbl@ld@calendar\@empty
3174 \let\bbl@ld@variant\@empty
3175 \let\bbl@ld@convert\relax
3176 \def\bbl@tempb##1=##2\@{\@namedef{bbl@ld@##1}{##2}}%
3177 \bbl@foreach\bbl@tempe{\bbl@tempb##1\@}%
3178 \bbl@replace\bbl@ld@calendar{gregorian}{}%
3179 \ifx\bbl@ld@calendar\@empty\else
3180 \ifx\bbl@ld@convert\relax\else
3181 \ifx\bbl@ld@convert\@empty
3182 \let\bbl@ld@convert\bbl@ld@calendar
3183 \else
3184 \edef\bbl@ld@convert{\expandafter\@gobble\bbl@ld@convert}%
3185 \fi
3186 \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3187 {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3188 \fi
3189 \fi
3190 \@nameuse{bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3191 \edef\bbl@calendar{% Used in \month..., too
3192 \bbl@ld@calendar
3193 \ifx\bbl@ld@variant\@empty\else
3194 .\bbl@ld@variant
3195 \fi}%
3196 \bbl@cased
3197 {\@nameuse{bbl@date@\language @\bbl@calendar}%
3198 \bbl@they\bbl@them\bbl@thed}%
3199 \endgroup}
3200 %
3201 \def\bbl@printdate#1{%
3202 \ifnextchar[{\bbl@printdate@i{#1}}{\bbl@printdate@i{#1}[]}}
3203 \def\bbl@printdate@i#1[#2]#3#4#5{%
3204 \bbl@usedategroupttrue

```



```

3205 \def\bbl@tempc{#1}%
3206 \bbl@replace\bbl@tempc{-}{@@}%
3207 \@nameuse{bbl@ensure@\bbl@tempc}{\localedate[#2]{#3}{#4}{#5}}
3208 %
3209 e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3210 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3211 \bbl@trim@def\bbl@tempa{#1.#2}%
3212 \bbl@ifsamestring{\bbl@tempa}{months.wide}% to savedate
3213 {\bbl@trim@def\bbl@tempa{#3}%
3214 \bbl@trim\toks@{#5}%
3215 \@temptokena\expandafter{\bbl@savedate}%
3216 \bbl@exp{% Reverse order - in ini last wins
3217 \def\\bbl@savedate{%
3218 \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3219 \the\@temptokena}}}%
3220 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3221 {\lowercase{\def\bbl@tempb{#6}}}%
3222 \bbl@trim@def\bbl@toreplace{#5}%
3223 \bbl@TG@@date
3224 \global\bbl@csarg\let{date@\language @\bbl@tempb}\bbl@toreplace
3225 \ifx\bbl@savetoday\@empty
3226 \bbl@exp{%
3227 \\AfterBabelCommands{%
3228 \gdef\<\language date>{\protect\<\language date >}%
3229 \gdef\<\language date >{\bbl@printdate{\language}}}%
3230 \def\\bbl@savetoday{%
3231 \\SetString\\today{%
3232 \<\language date>[convert]%
3233 {\the\year}{\the\month}{\the\day}}}%
3234 \fi}%
3235 {}%

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after `\bbl@replace \toks@` contains the resulting string, which is used by `\bbl@replace@finish@iii` (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3236 \let\bbl@calendar\@empty
3237 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3238 \bbl@xin@{\$calendrica:}{\$#2}%
3239 \ifin@
3240 \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3241 \edef\bbl@tempa{\$#2}%
3242 \bbl@replace\bbl@tempa{\$calendrica:}{}%
3243 \bbl@replace\bbl@tempa{-}{_}%
3244 \bbl@afterelse
3245 \bbl@exp{\bbl@calendrica#1\\@@{\bbl@tempa}}%
3246 \else
3247 \bbl@afterfi
3248 \@nameuse{bbl@ca#2}#1\\@@
3249 \fi}
3250 \def\bbl@calendrica#1-#2-#3\@#4#5#6#7{%
3251 \directlua{
3252 local d = Babel.calendrica.new_date(
3253 {year=\number#1, month=\number#2, day=\number#3},
3254 Babel.calendrica.cal_gregorian)
3255 local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3256 token.set_macro('\bbl@stripslash#5', r.year)
3257 token.set_macro('\bbl@stripslash#6', r.month)
3258 token.set_macro('\bbl@stripslash#7', r.day) }}
3259 \newcommand\babelDateSpace{\nobreakspace}
3260 \newcommand\babelDateDot{.\@}
3261 \newcommand\babelDated[1]{\number#1}

```

```

3262 \newcommand\BabelDatedd[1]{\ifnum#1<10 0\fi\number#1}
3263 \newcommand\BabelDateM[1]{\number#1}
3264 \newcommand\BabelDateMM[1]{\ifnum#1<10 0\fi\number#1}
3265 \newcommand\BabelDateMMMM[1]{\%
3266 \csname month\romannumeral#1\bbbl@calendar name\endcsname}}%
3267 \newcommand\BabelDatey[1]{\number#1}}%
3268 \newcommand\BabelDateyy[1]{\%
3269 \ifnum#1<10 0\number#1 %
3270 \else\ifnum#1<100 \number#1 %
3271 \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3272 \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3273 \else
3274 \bbbl@error{limit-two-digits}{}}}%
3275 \fi\fi\fi\fi}}
3276 \newcommand\BabelDateyyyy[1]{\number#1}
3277 \newcommand\BabelDateU[1]{\number#1}}%
3278 \def\bbbl@replace@finish@iii#1{%
3279 \bbbl@exp{\def\#1####1####2####3{\the\toks@}}
3280 \def\bbbl@TG@@date{%
3281 \bbbl@replace\bbbl@toreplace{[ ]}{\BabelDateSpace{}}%
3282 \bbbl@replace\bbbl@toreplace{[.]}{\BabelDateDot{}}%
3283 \bbbl@replace\bbbl@toreplace{[y]}{\BabelDatey{####1}}%
3284 \bbbl@replace\bbbl@toreplace{[y]}{\bbbl@datecctr[####1]}%
3285 \bbbl@replace\bbbl@toreplace{[yy]}{\BabelDateyy{####1}}%
3286 \bbbl@replace\bbbl@toreplace{[yyyy]}{\BabelDateyyyy{####1}}%
3287 \bbbl@replace\bbbl@toreplace{[M]}{\BabelDateM{####2}}%
3288 \bbbl@replace\bbbl@toreplace{[M]}{\bbbl@datecctr[####2]}%
3289 \bbbl@replace\bbbl@toreplace{[MM]}{\BabelDateMM{####2}}%
3290 \bbbl@replace\bbbl@toreplace{[MMMM]}{\BabelDateMMMM{####2}}%
3291 \bbbl@replace\bbbl@toreplace{[d]}{\BabelDated{####3}}%
3292 \bbbl@replace\bbbl@toreplace{[d]}{\bbbl@datecctr[####3]}%
3293 \bbbl@replace\bbbl@toreplace{[dd]}{\BabelDatedd{####3}}%
3294 \bbbl@replace\bbbl@toreplace{[U]}{\BabelDateU{####1}}%
3295 \bbbl@replace\bbbl@toreplace{[U]}{\bbbl@datecctr[####1]}%
3296 \bbbl@replace@finish@iii\bbbl@toreplace}
3297 \def\bbbl@datecctr{\expandafter\bbbl@xdatecctr\expandafter}
3298 \def\bbbl@xdatecctr[#1#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. \nonfrenchspacing is set by document too early, so it's a hack.

```

3299 \AddToHook{begindocument/before}{%
3300 \let\bbbl@normalsf\normalsfcodes
3301 \let\normalsfcodes\relax}
3302 \AtBeginDocument{%
3303 \ifx\bbbl@normalsf\empty
3304 \ifnum\sfcodes\.\@m
3305 \let\normalsfcodes\frenchspacing
3306 \else
3307 \let\normalsfcodes\nonfrenchspacing
3308 \fi
3309 \else
3310 \let\normalsfcodes\bbbl@normalsf
3311 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with \babelprehyphenation and \babelprehyphenation), wrapped with \bbbl@transforms@aux ...\relax, and stores them in \bbbl@release@transforms. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then \bbbl@transforms@aux adds the braces.

```

3312 \bbbl@csarg\let\inikv@transforms.prehyphenation\bbbl@inikv

```

```

3313 \bbl@csarg\let\inikv@transforms.posthyphenation\bbl@inikv
3314 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3315   #1[#2]{#3}{#4}{#5}}
3316 \begingroup
3317   \catcode`\%=12
3318   \catcode`\&=14
3319   \gdef\bbl@transforms#1#2#3{%&%
3320     \directlua{
3321       local str = [==[#2]==]
3322       str = str:gsub('%.%d+%.%d+$', '')
3323       token.set_macro('babeltempa', str)
3324     }&%
3325     \def\babeltempc{}&%
3326     \bbl@xin@{,\babeltempa,}{,\bbl@KVP@transforms,}&%
3327     \ifin@else
3328       \bbl@xin@{:,\babeltempa,}{,\bbl@KVP@transforms,}&%
3329     \fi
3330     \ifin@
3331       \bbl@foreach\bbl@KVP@transforms{%&%
3332         \bbl@xin@{:,\babeltempa,}{,##1,}&%
3333         \ifin@ &% font:font:transform syntax
3334         \directlua{
3335           local t = {}
3336           for m in string.gmatch('##1'..' ':'(.)') do
3337             table.insert(t, m)
3338           end
3339           table.remove(t)
3340           token.set_macro('babeltempc', ',fonts=' .. table.concat(t, ' '))
3341         }&%
3342       \fi}&%
3343     \in@{.0$}{#2$}&%
3344     \ifin@
3345       \directlua{%&% (\attribute) syntax
3346         local str = string.match([[ \bbl@KVP@transforms]],
3347           '%([^( )- )[^)]-\babeltempa')
3348         if str == nil then
3349           token.set_macro('babeltempb', '')
3350         else
3351           token.set_macro('babeltempb', ',attribute=' .. str)
3352         end
3353       }&%
3354     \toks@{#3}&%
3355     \bbl@exp{%&%
3356       \\g@addto@macro\\bbl@release@transforms{%&%
3357         \relax &% Closes previous \bbl@transforms@aux
3358         \\bbl@transforms@aux
3359         \\#1{label=\babeltempa\babeltempb\babeltempc}&%
3360         {\language\the\toks@}}&%
3361     \else
3362       \g@addto@macro\bbl@release@transforms{, {#3}}&%
3363     \fi
3364   \fi}
3365 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3366 \def\bbl@provide@lsys#1{%
3367   \bbl@ifunset\bbl@lname@#1{%
3368     {\bbl@load@info{#1}}%

```

```

3369 {}%
3370 \bbl@csarg\let{lsys@#1}\@empty
3371 \bbl@ifunset{bbl@sname@#1}{\bbl@csarg\gdef{sname@#1}{Default}}{}%
3372 \bbl@ifunset{bbl@sotf@#1}{\bbl@csarg\gdef{sotf@#1}{DFLT}}{}%
3373 \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3374 \bbl@ifunset{bbl@lname@#1}{}%
3375 {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3376 \ifcase\bbl@engine\or\or
3377 \bbl@ifunset{bbl@prehc@#1}{}%
3378 {\bbl@exp{\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3379 {}%
3380 {\ifx\bbl@xenohyph\undefined
3381 \global\let\bbl@xenohyph\bbl@xenohyph@
3382 \ifx\AtBeginDocument\@notprerr
3383 \expandafter\@secondoftwo % to execute right now
3384 \fi
3385 \AtBeginDocument{%
3386 \bbl@patchfont{\bbl@xenohyph}%
3387 {\expandafter\select@language\expandafter{\language name}}}%
3388 \fi}%
3389 \fi
3390 \bbl@csarg\bbl@toqlobal{lsys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the ini file. Somewhat convoluted because there are 10 digits, but only 9 arguments in T_EX. Non-digits characters are kept. The first macro is the generic “localized” command.

[illegible]

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```
3422 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}  
3423   \ifx\##1%           % \ before, in case #1 is multiletter
```

```

3424 \bbl@exp{%
3425 \def\\bbl@tempa###1{%
3426 \<ifcase>###1\space\the\toks@<else>\\@ctrerr<fi>}}%
3427 \else
3428 \toks@<expandafter{\the\toks@<or #1>%
3429 \expandafter\bbl@buildifcase
3430 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before \@@ collects digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210. Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```

3431 \newcommand\localenumeral[2]{%
3432 \bbl@ifunset\bbl@cntr@#1@<language>{%
3433 {#2}%
3434 {\bbl@cs{cntr@#1@<language>}{#2}}}
3435 \def\bbl@localecntr#1#2{\localenumeral{#2}{#1}}
3436 \newcommand\localecounter[2]{%
3437 \expandafter\bbl@localecntr
3438 \expandafter{\number\csname c@#2\endcsname}{#1}}
3439 \def\bbl@alphanumeric#1#2{%
3440 \expandafter\bbl@alphanumeric@i\number#2 76543210\@@{#1}}
3441 \def\bbl@alphanumeric@i#1#2#3#4#5#6#7#8\@@#9{%
3442 \ifcase@car#8@nil\or % Currently <10000, but prepared for bigger
3443 \bbl@alphanumeric@ii{#9}00000#1\or
3444 \bbl@alphanumeric@ii{#9}00000#1#2\or
3445 \bbl@alphanumeric@ii{#9}0000#1#2#3\or
3446 \bbl@alphanumeric@ii{#9}000#1#2#3#4\else
3447 \bbl@error{counter-too-large}{>9999}{}}%
3448 \fi}
3449 \def\bbl@alphanumeric@ii#1#2#3#4#5#6#7#8{%
3450 \bbl@ifunset\bbl@cntr@#1.F.\number#5#6#7#8@<language>{%
3451 {\bbl@cs{cntr@#1.4@<language>}{#5}%
3452 \bbl@cs{cntr@#1.3@<language>}{#6}%
3453 \bbl@cs{cntr@#1.2@<language>}{#7}%
3454 \bbl@cs{cntr@#1.1@<language>}{#8}%
3455 \ifnum#6#7#8>\z@
3456 \bbl@ifunset\bbl@cntr@#1.S.321@<language>}{}%
3457 {\bbl@cs{cntr@#1.S.321@<language>}{}}%
3458 \fi}%
3459 {\bbl@cs{cntr@#1.F.\number#5#6#7#8@<language>}}}

```

Some Chinese counters follow special rules.

```

3460 \def\bbl@zhnumeral#1#2{%
3461 \ifnum#1<10 \bbl@zhnum000#1{#2}%
3462 \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3463 \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3464 \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3465 \else\bbl@error{counter-too-large}{#1}{}}%
3466 \fi\fi\fi\fi}
3467 \def\bbl@zhnum#1#2#3#4#5{%
3468 \ifnum\numexpr#1#2#3#4\relax=\z@
3469 \localenumeral{#5.C}\@ne
3470 \else
3471 \ifnum#1>\z@\localenumeral{#5.D}{#1}\localenumeral{#5.C}5\fi % Thousands
3472 \ifnum#2>\z@\localenumeral{#5.D}{#2}\localenumeral{#5.C}4\fi % Hundreds
3473 \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumeral{#5.C}\@ne\fi
3474 \fi\fi
3475 \ifnum#3>\z@ % Tens
3476 \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumeral{#5.D}{#3}\fi % No  for 10-19:
3477 \else\localenumeral{#5.D}{#3}%
3478 \fi
3479 \localenumeral{#5.C}\thr@@

```

```

3480 \else
3481 \ifnum#2>\z@ \ifnum#4>\z@ \localenumeral{#5.C}\@ne\fi\fi
3482 \fi
3483 \ifnum#4>\z@ \localenumeral{#5.D}{#4}\fi % Units
3484 \fi}

```

4.24. Casing

```

3485 \newcommand\BabelUppercaseMapping[3]{%
3486 \DeclareUppercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3487 \newcommand\BabelTitlecaseMapping[3]{%
3488 \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3489 \newcommand\BabelLowercaseMapping[3]{%
3490 \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

The parser for casing and casing. (variant).
3491 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3492 \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3493 \else
3494 \def\bbl@uftocode#1{\expandafter`\string#1}
3495 \fi
3496 \def\bbl@casemapping#1#2#3{% 1:variant
3497 \def\bbl@tempa##1 ##2{% Loop
3498 \bbl@casemapping@i{##1}%
3499 \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3500 \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3501 \def\bbl@tempe{0}% Mode (upper/lower...)
3502 \def\bbl@tempc{#3}% Casing list
3503 \expandafter\bbl@tempa\bbl@tempc\@empty}
3504 \def\bbl@casemapping@i#1{%
3505 \def\bbl@tempb{#1}%
3506 \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3507 \@nameuse{regex_replace_all:nnN}%
3508 {[ \x{c0}-\x{ff}][ \x{80}-\x{bf}]*}{\0}}\bbl@tempb
3509 \else
3510 \@nameuse{regex_replace_all:nnN}{.}{\0}}\bbl@tempb
3511 \fi
3512 \expandafter\bbl@casemapping@ii\bbl@tempb\@@}
3513 \def\bbl@casemapping@ii#1#2#3\@@{%
3514 \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3515 \ifin@
3516 \edef\bbl@tempe{%
3517 \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3518 \else
3519 \ifcase\bbl@tempe\relax
3520 \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3521 \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3522 \or
3523 \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3524 \or
3525 \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3526 \or
3527 \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3528 \fi
3529 \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3530 \def\bbl@localeinfo#1#2{%
3531 \bbl@ifunset{bbl@info@#2}{#1}%
3532 {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3533 {\bbl@cs{\csname bbl@info@#2\endcsname @\languagename}}}}

```

```

3534 \newcommand\localeinfo[1]{%
3535   \ifx*#1\@empty
3536     \bbl@afterelse\bbl@localeinfo}%
3537   \else
3538     \bbl@localeinfo
3539     {\bbl@error{no-ini-info}{}}}%
3540   {#1}%
3541 \fi}
3542 % \@namedef{bbl@info@name.locale}{lcname}
3543 \@namedef{bbl@info@tag.ini}{lini}
3544 \@namedef{bbl@info@name.english}{elname}
3545 \@namedef{bbl@info@name.opentype}{lname}
3546 \@namedef{bbl@info@tag.bcp47}{tbcp}
3547 \@namedef{bbl@info@language.tag.bcp47}{lbcp}
3548 \@namedef{bbl@info@tag.opentype}{lotf}
3549 \@namedef{bbl@info@script.name}{esname}
3550 \@namedef{bbl@info@script.name.opentype}{sname}
3551 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3552 \@namedef{bbl@info@script.tag.opentype}{soft}
3553 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3554 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3555 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3556 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3557 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3558 << *More package options >> ≡
3559 \DeclareOption{ensureinfo=off}{}
3560 << /More package options >>
3561 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3562 \newcommand\getlocaleproperty{%
3563   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3564 \def\bbl@getproperty@s#1#2#3{%
3565   \let#1\relax
3566   \def\bbl@elt##1##2##3{%
3567     \bbl@ifsamestring{##1/##2}{#3}%
3568     {\providecommand#1{##3}%
3569     \def\bbl@elt####1####2####3{}}}%
3570   {}}%
3571   \bbl@cs{inidata@#2}}%
3572 \def\bbl@getproperty@x#1#2#3{%
3573   \bbl@getproperty@s{#1}{#2}{#3}%
3574   \ifx#1\relax
3575     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3576   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3577 \let\bbl@ini@loaded\@empty
3578 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3579 \def\ShowLocaleProperties#1{%
3580   \typeout{}}%
3581   \typeout{*** Properties for language '#1' ***}%
3582   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3583   \@nameuse{bbl@inidata@#1}%
3584   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 27 tag to the actual language, if bcp47.toname is enabled (i.e., if \bbl@bcptoname

is true), and (2) lazy loading. With `autoload.bcp47` enabled *and* lazy loading, we must first build a name for the language, with the help of `autoload.bcp47.prefix`. Then we use `\provideprovide` passing the options set with `autoload.bcp47.options` (by default `import`). Finally, and if the locale has not been loaded before, we use `\provideprovide` with the language name as passed to the selector.

```

3585 \newif\ifbbl@bcpallowed
3586 \bbl@bcpallowedfalse
3587 \def\bbl@autoload@options{@import}
3588 \def\bbl@provide@locale{%
3589   \ifx\babelprovide\undefined
3590     \bbl@error{base-on-the-fly}{}}}%
3591 \fi
3592 \let\bbl@auxname\language
3593 \ifbbl@bcptoname
3594   \bbl@ifunset{bbl@bcp@map@\language}{}% Move uplevel??
3595   {\edef\language{\@nameuse{bbl@bcp@map@\language}}}%
3596   \let\localename\language}%
3597 \fi
3598 \ifbbl@bcpallowed
3599   \expandafter\ifx\csname date\language\endcsname\relax
3600     \expandafter
3601     \bbl@bcplookup{\language}%-\@empty-\@empty-\@empty\@@
3602     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3603       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3604       \let\localename\language
3605       \expandafter\ifx\csname date\language\endcsname\relax
3606         \let\bbl@initoload\bbl@bcp
3607         \bbl@exp{\\babelprovide[\bbl@autoload@bcptoptions]{\language}}}%
3608         \let\bbl@initoload\relax
3609       \fi
3610       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localename}%
3611     \fi
3612   \fi
3613 \fi
3614 \expandafter\ifx\csname date\language\endcsname\relax
3615   \IfFileExists{babel-\language.tex}%
3616   {\bbl@exp{\\babelprovide[\bbl@autoload@options]{\language}}}%
3617   {}%
3618 \fi}

```

$\TeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension.`<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to `tag.bcp47`.

```

3619 \providecommand\BCPdata{}
3620 \ifx\renewcommand\undefined\else
3621   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3622   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3623     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3624     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3625     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3626   \def\bbl@bcpdata@ii#1#2{%
3627     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3628     {\bbl@error{unknown-ini-field}{#1}}}%
3629     {\bbl@ifunset{bbl@csname\bbl@info@#1.tag.bcp47\endcsname @#2}{}%
3630     {\bbl@cs{csname\bbl@info@#1.tag.bcp47\endcsname @#2}}}%
3631 \fi
3632 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3633 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```


5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```
3634 \newcommand\babeladjust[1]{%
3635   \bbl@forkv{#1}{%
3636     \bbl@ifunset{bbl@ADJ@##1@##2}%
3637     {\bbl@cs{ADJ@##1}{##2}}%
3638     {\bbl@cs{ADJ@##1@##2}}}
3639 %
3640 \def\bbl@adjust@lua#1#2{%
3641   \ifvmode
3642     \ifnum\currentgrouplevel=\z@
3643       \directlua{ Babel.#2 }%
3644       \expandafter\expandafter\expandafter\@gobble
3645     \fi
3646   \fi
3647   {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3648 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3649   \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3650 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3651   \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3652 \@namedef{bbl@ADJ@bidi.text@on}{%
3653   \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3654 \@namedef{bbl@ADJ@bidi.text@off}{%
3655   \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3656 \@namedef{bbl@ADJ@bidi.math@on}{%
3657   \let\bbl@noamsmath\@empty}
3658 \@namedef{bbl@ADJ@bidi.math@off}{%
3659   \let\bbl@noamsmath\relax}
3660 %
3661 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3662   \bbl@adjust@lua{bidi}{digits_mapped=true}}
3663 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3664   \bbl@adjust@lua{bidi}{digits_mapped=false}}
3665 %
3666 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3667   \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3668 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3669   \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3670 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3671   \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3672 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3673   \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3674 \@namedef{bbl@ADJ@justify.arabic@on}{%
3675   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3676 \@namedef{bbl@ADJ@justify.arabic@off}{%
3677   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3678 %
3679 \def\bbl@adjust@layout#1{%
3680   \ifvmode
3681     #1%
3682     \expandafter\expandafter\expandafter\@gobble
3683   \fi
3684   {\bbl@error{layout-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3685 \@namedef{bbl@ADJ@layout.tabular@on}{%
3686   \ifnum\bbl@tabular@mode=\tw@
3687     \bbl@adjust@layout{\let\@tabular\bbl@NL@tabular}%
3688   \else
3689     \chardef\bbl@tabular@mode\@ne
3690   \fi}
3691 \@namedef{bbl@ADJ@layout.tabular@off}{%
3692   \ifnum\bbl@tabular@mode=\tw@
3693     \bbl@adjust@layout{\let\@tabular\bbl@OL@tabular}%
3694   \fi}
```

```

3694 \else
3695   \chardef\bbl@tabular@mode\z@
3696 \fi}
3697 \@namedef{bbl@ADJ@layout.lists@on}{%
3698   \bbl@adjust@layout{\let\list\bbl@NL@list}}
3699 \@namedef{bbl@ADJ@layout.lists@off}{%
3700   \bbl@adjust@layout{\let\list\bbl@OL@list}}
3701 %
3702 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3703   \bbl@bcppallowedtrue}
3704 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3705   \bbl@bcppallowedfalse}
3706 \@namedef{bbl@ADJ@autoload.bcp47.prefix#1}{%
3707   \def\bbl@bcp@prefix{#1}}
3708 \def\bbl@bcp@prefix{bcp47-}
3709 \@namedef{bbl@ADJ@autoload.options#1}{%
3710   \def\bbl@autoload@options{#1}}
3711 \def\bbl@autoload@bcptoptions{import}
3712 \@namedef{bbl@ADJ@autoload.bcp47.options#1}{%
3713   \def\bbl@autoload@bcptoptions{#1}}
3714 \newif\ifbbl@bcptoname
3715 %
3716 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3717   \bbl@bcptonametrue}
3718 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3719   \bbl@bcptonamefalse}
3720 %
3721 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3722   \directlua{ Babel.ignore_pre_char = function(node)
3723     return (node.lang == \the\csname l@nohyphenation\endcsname)
3724   end }}
3725 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3726   \directlua{ Babel.ignore_pre_char = function(node)
3727     return false
3728   end }}
3729 %
3730 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3731   \def\bbl@ignoreinterchar{%
3732     \ifnum\language=\l@nohyphenation
3733       \expandafter\@gobble
3734     \else
3735       \expandafter\@firstofone
3736     \fi}}
3737 \@namedef{bbl@ADJ@interchar.disable@off}{%
3738   \let\bbl@ignoreinterchar\@firstofone}
3739 %
3740 \@namedef{bbl@ADJ@select.write@shift}{%
3741   \let\bbl@restorelastskip\relax
3742   \def\bbl@savelastskip{%
3743     \let\bbl@restorelastskip\relax
3744     \ifvmode
3745       \ifdim\lastskip=\z@
3746         \let\bbl@restorelastskip\nobreak
3747       \else
3748         \bbl@exp{%
3749           \def\\bbl@restorelastskip{%
3750             \skip@=\the\lastskip
3751             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3752         \fi
3753       \fi}}
3754 \@namedef{bbl@ADJ@select.write@keep}{%
3755   \let\bbl@restorelastskip\relax
3756   \let\bbl@savelastskip\relax}

```

```

3757 \@namedef{bbl@ADJ@select.write@omit}{%
3758   \AddBabelHook{babel-select}{beforestart}{%
3759     \expandafter\babel@aux\expandafter{\bbl@main@language}{}}%
3760   \let\bbl@restorelastskip\relax
3761   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3762 \@namedef{bbl@ADJ@select.encoding@off}{%
3763   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The key argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```

3764 << *More package options >> ≡
3765 \DeclareOption{safe=none}{\let\bbl@opt@safe\@empty}
3766 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3767 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3768 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3769 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3770 <</More package options>>

```

\@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```

3771 \bbl@trace{Cross referencing macros}
3772 \ifx\bbl@opt@safe\@empty\else % i.e., if 'ref' and/or 'bib'
3773   \def\@newl@bel#1#2#3{%
3774     {\@safe@activestrue
3775       \bbl@ifunset{#1@#2}%
3776       \relax
3777       {\gdef\@multiplelabels{%
3778         \@latex@warning@no@line{There were multiply-defined labels}}%
3779         \@latex@warning@no@line{Label `#2' multiply defined}}%
3780       \global\@namedef{#1@#2}{#3}}}

```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```

3781 \CheckCommand*\@testdef[3]{%
3782   \def\reserved@a{#3}%
3783   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3784     \else
3785       \@tempswatrue
3786       \fi}

```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```

3787 \def\@testdef#1#2#3{%
3788   \@safe@activestrue
3789   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3790   \def\bbl@tempb{#3}%
3791   \@safe@activesfalse
3792   \ifx\bbl@tempa\relax

```

```

3793 \else
3794 \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3795 \fi
3796 \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3797 \ifx\bbl@tempa\bbl@tempb
3798 \else
3799 \@tempswatrue
3800 \fi}
3801 \fi

```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```

3802 \bbl@xin@{R}\bbl@opt@safe
3803 \ifin@
3804 \edef\bbl@tempc{\expandafter\string\csname ref code\endcsname}%
3805 \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3806 {\expandafter\strip@prefix\meaning\ref}%
3807 \ifin@
3808 \bbl@redefine\@kernel@ref#1{%
3809 \@safe@activetrue\org@@kernel@ref{#1}\@safe@activetrue}
3810 \bbl@redefine\@kernel@pageref#1{%
3811 \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activetrue}
3812 \bbl@redefine\@kernel@sref#1{%
3813 \@safe@activetrue\org@@kernel@sref{#1}\@safe@activetrue}
3814 \bbl@redefine\@kernel@spageref#1{%
3815 \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activetrue}
3816 \else
3817 \bbl@redefineroobust\ref#1{%
3818 \@safe@activetrue\org@ref{#1}\@safe@activetrue}
3819 \bbl@redefineroobust\pageref#1{%
3820 \@safe@activetrue\org@pageref{#1}\@safe@activetrue}
3821 \fi
3822 \else
3823 \let\org@ref\ref
3824 \let\org@pageref\pageref
3825 \fi

```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```

3826 \bbl@xin@{B}\bbl@opt@safe
3827 \ifin@
3828 \bbl@redefine\@citex[#1]#2{%
3829 \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activetrue}
3830 \org@@citex[#1]{\bbl@tempa}}

```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```

3831 \AtBeginDocument{%
3832 \@ifpackageloaded{natbib}{%
3833 \def\@citex[#1][#2]#3{%
3834 \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activetrue}
3835 \org@@citex[#1][#2]{\bbl@tempa}}%

```

```
3836 }{}}
```

The package cite has a definition of \@citex where the shorthands need to be turned off in both arguments.

```
3837 \AtBeginDocument{%
3838   \@ifpackageloaded{cite}{%
3839     \def\@citex[#1]#2{%
3840       \@safe@activetrue\org@citex[#1]#2\@safe@activesfalse}%
3841     }{}}
```

\nocite The macro \nocite which is used to instruct BiB_T_EX to extract uncited references from the database.

```
3842 \bbl@redefine\nocite#1{%
3843   \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}
```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \bibcite is needed we define \bibcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \bibcite. This new definition is then activated.

```
3844 \bbl@redefine\bibcite{%
3845   \bbl@cite@choice
3846   \bibcite}
```

\bbl@bibcite The macro \bbl@bibcite holds the definition of \bibcite needed when neither natbib nor cite is loaded.

```
3847 \def\bbl@bibcite#1#2{%
3848   \org@bibcite{#1}\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3849 \def\bbl@cite@choice{%
3850   \global\let\bibcite\bbl@bibcite
3851   \@ifpackageloaded{natbib}{\global\let\bibcite\org@bibcite}{%
3852     \@ifpackageloaded{cite}{\global\let\bibcite\org@bibcite}{%
3853     \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3854 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal L^AT_EX macros called by \bibitem that write the citation label on the aux file.

```
3855 \bbl@redefine\@bibitem#1{%
3856   \@safe@activetrue\org@bibitem{#1}\@safe@activesfalse}
3857 \else
3858   \let\org@nocite\nocite
3859   \let\org@citex\citex
3860   \let\org@bibcite\bibcite
3861   \let\org@bibitem\@bibitem
3862 \fi
```

5.2. Layout

```

3863 \newcommand\BabelPatchSection[1]{%
3864   \@ifundefined{#1}{}{%
3865     \bbl@exp{\let\<bbl@ss@#1>\<#1>}%
3866     \@namedef{#1}{%
3867       \@ifstar{\bbl@presec@#1}%
3868       {\@dblarg{\bbl@presec@x{#1}}}}%
3869 \def\bbl@presec@x#1[#2]#3{%
3870   \bbl@exp{%
3871     \\\select@language@x{\bbl@main@language}%
3872     \\\bbl@cs{sspre@#1}%
3873     \\\bbl@cs{ss@#1}%
3874     [\\\foreignlanguage{\language}\unexpanded{#2}}}%
3875     {\\\foreignlanguage{\language}\unexpanded{#3}}}%
3876     \\\select@language@x{\language}}%
3877 \def\bbl@presec@#1#2{%
3878   \bbl@exp{%
3879     \\\select@language@x{\bbl@main@language}%
3880     \\\bbl@cs{sspre@#1}%
3881     \\\bbl@cs{ss@#1}*%
3882     {\\\foreignlanguage{\language}\unexpanded{#2}}}%
3883     \\\select@language@x{\language}}%
3884 %
3885 \IfBabelLayout{sectioning}%
3886   {\BabelPatchSection{part}%
3887    \BabelPatchSection{chapter}%
3888    \BabelPatchSection{section}%
3889    \BabelPatchSection{subsection}%
3890    \BabelPatchSection{subsubsection}%
3891    \BabelPatchSection{paragraph}%
3892    \BabelPatchSection{subparagraph}}%
3893   \def\babel@toc#1{%
3894     \select@language@x{\bbl@main@language}}}%
3895 \IfBabelLayout{captions}%
3896   {\BabelPatchSection{caption}}}%

```

\BabelFootnote Footnotes.

```

3897 \bbl@trace{Footnotes}
3898 \def\bbl@footnote#1#2#3{%
3899   \@ifnextchar[%
3900     {\bbl@footnote@o{#1}{#2}{#3}}%
3901     {\bbl@footnote@x{#1}{#2}{#3}}}%
3902 \long\def\bbl@footnote@x#1#2#3#4{%
3903   \bgroup
3904     \select@language@x{\bbl@main@language}%
3905     \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3906   \egroup}
3907 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3908   \bgroup
3909     \select@language@x{\bbl@main@language}%
3910     \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3911   \egroup}
3912 \def\bbl@footnotetext#1#2#3{%
3913   \@ifnextchar[%
3914     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3915     {\bbl@footnotetext@x{#1}{#2}{#3}}}%
3916 \long\def\bbl@footnotetext@x#1#2#3#4{%
3917   \bgroup
3918     \select@language@x{\bbl@main@language}%
3919     \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3920   \egroup}
3921 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3922   \bgroup

```

```

3923 \select@language@x{\bbl@main@language}%
3924 \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3925 \egroup}
3926 \def\babelFootnote#1#2#3#4{%
3927 \ifx\bbl@fn@footnote\@undefined
3928 \let\bbl@fn@footnote\footnote
3929 \fi
3930 \ifx\bbl@fn@footnotetext\@undefined
3931 \let\bbl@fn@footnotetext\footnotetext
3932 \fi
3933 \bbl@ifblank{#2}%
3934 {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3935 \namedef{\bbl@stripslash#1text}%
3936 {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3937 {\def#1{\bbl@exp{\bbl@footnote{\bbl@foreignlanguage{#2}}}{#3}{#4}}%
3938 \namedef{\bbl@stripslash#1text}%
3939 {\bbl@exp{\bbl@footnotetext{\bbl@foreignlanguage{#2}}}{#3}{#4}}}%
3940 \IfBabelLayout{footnotes}%
3941 {\let\bbl@OL@footnote\footnote
3942 \babelFootnote\footnote\language\{}}%
3943 \babelFootnote\localfootnote\language\{}}%
3944 \babelFootnote\mainfootnote\{}}%
3945 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3946 \bbl@trace{Marks}
3947 \IfBabelLayout{sectioning}
3948 {\ifx\bbl@opt@headfoot\@nnil
3949 \g@addto@macro\@resetactivechars{%
3950 \set@typeset@protect
3951 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3952 \let\protect\noexpand
3953 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3954 \edef\thepage{%
3955 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3956 \fi}%
3957 \fi}
3958 {\ifbbl@single\else
3959 \bbl@ifunset{markright} \bbl@redefine\bbl@redefineroobust
3960 \markright#1{%
3961 \bbl@ifblank{#1}%
3962 {\org@markright}}%
3963 {\toks@{#1}%
3964 \bbl@exp{%
3965 \bbl@foreignlanguage{\language\{}}%
3966 \bbl@restore@actives\the\toks@}}}%

```

\markboth

\mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\mkboth`. Therefore we need to check whether `\mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, `LaTeX` stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3967 \ifx\mkboth\markboth

```

```

3968     \def\bbl@tempc{\let\@mkboth\markboth}%
3969     \else
3970     \def\bbl@tempc{}%
3971     \fi
3972     \bbl@ifunset{markboth }{\bbl@redefine\bbl@redefineroobust
3973     \markboth#1#2{%
3974     \protected@edef\bbl@tempb##1{%
3975     \protect\foreignlanguage
3976     {\language\name}{\protect\bbl@restore@actives##1}}}%
3977     \bbl@ifblank{#1}%
3978     {\toks@{}}%
3979     {\toks@\expandafter{\bbl@tempb{#1}}}%
3980     \bbl@ifblank{#2}%
3981     {\@temptokena{}}%
3982     {\@temptokena\expandafter{\bbl@tempb{#2}}}%
3983     \bbl@exp{\@org@markboth{\the\toks@}{\the\@temptokena}}}%
3984     \bbl@tempc
3985     \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}{
%     {code for odd pages}
%     {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

3986 \bbl@trace{Preventing clashes with other packages}
3987 \ifx\org@ref\undefined\else
3988     \bbl@xin@{R}\bbl@opt@safe
3989     \ifin@
3990     \AtBeginDocument{%
3991     \ifpackageloaded{ifthen}{%
3992     \bbl@redefine@long\ifthenelse#1#2#3{%
3993     \let\bbl@temp@pref\pageref
3994     \let\pageref\org@pageref
3995     \let\bbl@temp@ref\ref
3996     \let\ref\org@ref
3997     \@safe@activestrue
3998     \org@ifthenelse{#1}%
3999     {\let\pageref\bbl@temp@pref
4000     \let\ref\bbl@temp@ref
4001     \@safe@activesfalse
4002     #2}%
4003     {\let\pageref\bbl@temp@pref
4004     \let\ref\bbl@temp@ref
4005     \@safe@activesfalse
4006     #3}%
4007     }%
4008     }{}%
4009     }

```


4010 \fi

5.4.2. varioref

\@@vpageref
\vrefpagenum

\Ref When the package varioref is in use we need to modify its internal command \@@vpageref in order to prevent problems when an active character ends up in the argument of \vref. The same needs to happen for \vrefpagenum.

```
4011 \AtBeginDocument{%
4012   \ifpackageloaded{varioref}{%
4013     \bbl@redefine\@@vpageref#1[#2]#3{%
4014       \@safe@activetrue
4015       \org@@vpageref{#1}[#2]#3}%
4016     \@safe@activesfalse}%
4017   \bbl@redefine\vrefpagenum#1#2{%
4018     \@safe@activetrue
4019     \org@vrefpagenum{#1}#2}%
4020   \@safe@activesfalse}%
```

The package varioref defines \Ref to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of \ref. So we employ a little trick here. We redefine the (internal) command \Ref_␣ to call \org@ref instead of \ref. The disadvantage of this solution is that whenever the definition of \Ref changes, this definition needs to be updated as well.

```
4021   \expandafter\def\csname Ref \endcsname#1{%
4022     \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4023   }{}%
4024 }
4025 \fi
```

5.4.3. hhline

\hhline Delaying the activation of the shorthand characters has introduced a problem with the hhline package. The reason is that it uses the ‘:’ character which is made active by the french support in babel. Therefore we need to *reload* the package when the ‘:’ is an active character. Note that this happens *after* the category code of the @-sign has been changed to other, so we need to temporarily change it to letter again.

```
4026 \AtEndOfPackage{%
4027   \AtBeginDocument{%
4028     \ifpackageloaded{hhline}%
4029     {\expandafter\ifx\csname normal@char\string\endcsname\relax
4030       \else
4031         \makeatletter
4032         \def\@currname{hhline}\input{hhline.sty}\makeatother
4033       \fi}%
4034     {}}}
```

\substitutefontfamily *Deprecated.* It creates an fd file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by L^AT_EX (\DeclareFontFamilySubstitution).

```
4035 \def\substitutefontfamily#1#2#3{%
4036   \lowercase{\immediate\openout15=#1#2.fd\relax}%
4037   \immediate\write15{%
4038     \string\ProvidesFile{#1#2.fd}%
4039     [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4040     \space generated font description file]^J
4041     \string\DeclareFontFamily{#1}{#2}{}^J
4042     \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}^J
4043     \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}^J
4044     \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}^J
4045     \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}^J
```

```

4046 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^^J
4047 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^^J
4048 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^^J
4049 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^^J
4050 }%
4051 \closeout15
4052 }
4053 \@onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of `\TeX` and `\LaTeX` for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```

4054 \bbl@trace{Encoding and fonts}
4055 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4056 \newcommand\BabelNonText{TS1,T3,TS3}
4057 \let\org@TeX\TeX
4058 \let\org@LaTeX\LaTeX
4059 \let\ensureascii@firstofone
4060 \let\asciientcoding\@empty
4061 \AtBeginDocument{%
4062   \def\@elt#1{,#1,}%
4063   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4064   \let\@elt\relax
4065   \let\bbl@tempb\@empty
4066   \def\bbl@tempc{OT1}%
4067   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4068     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4069   \bbl@foreach\bbl@tempa{%
4070     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4071     \ifin@
4072       \def\bbl@tempb{#1}% Store last non-ascii
4073     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4074       \ifin@else
4075         \def\bbl@tempc{#1}% Store last ascii
4076       \fi
4077     \fi}%
4078   \ifx\bbl@tempb\@empty\else
4079     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4080     \ifin@else
4081       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4082     \fi
4083     \let\asciientcoding\bbl@tempc
4084     \renewcommand\ensureascii[1]{%
4085       {\fontencoding{\asciientcoding}\selectfont#1}}%
4086     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4087     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4088   \fi}

```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than ‘latin’ (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```

4089 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}

```

But this might be overruled with a later loading of the package `fontenc`. Therefore we check at the execution of `\begin{document}` whether it was loaded with the `T1` option. The normal way to do this (using `\ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```

4090 \AtBeginDocument{%
4091   \ifpackageloaded{fontspec}%
4092     {\xdef\latinencoding{%
4093       \ifx\UTFencname\undefined
4094         EU\ifcase\bbl@engine\or2\or1\fi
4095       \else
4096         \UTFencname
4097       \fi}}%
4098   {\gdef\latinencoding{OT1}%
4099     \ifx\cf@encoding\bbl@t@one
4100       \xdef\latinencoding{\bbl@t@one}%
4101     \else
4102       \def\@elt#1{, #1,}%
4103       \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4104       \let\@elt\relax
4105       \bbl@xin@{, T1, }\bbl@tempa
4106       \ifin@
4107         \xdef\latinencoding{\bbl@t@one}%
4108       \fi
4109     \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4110 \DeclareRobustCommand{\latintext}{%
4111   \fontencoding{\latinencoding}\selectfont
4112   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4113 \ifx\@undefined\DeclareTextFontCommand
4114   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4115 \else
4116   \DeclareTextFontCommand{\textlatin}{\latintext}
4117 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\text{\TeX}$ 2021-06-01, there is a hook for this purpose.

```

4118 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This `babel` module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at `ARABI` (by Youssef Jabri), which is compatible with `babel`.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdftex` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\text{\TeX}$ grouping.

- luatex can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As LuaTeX-jā shows, vertical typesetting is possible, too.

```

4119 \bbl@trace{Loading basic (internal) bidi support}
4120 \ifodd\bbl@engine
4121 \else % Any xe+lua bidi
4122   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4123     \bbl@error{bidi-only-lua}{}}{}{}%
4124     \let\bbl@beforeforeign\leavevmode
4125     \AtEndOfPackage{%
4126       \EnableBabelHook{babel-bidi}%
4127       \bbl@xebidipar}
4128   \fi\fi
4129   \def\bbl@loadxebidi#1{%
4130     \ifx\RTLfootnotetext\@undefined
4131       \AtEndOfPackage{%
4132         \EnableBabelHook{babel-bidi}%
4133         \ifx\fontspec\@undefined
4134           \usepackage{fontspec}% bidi needs fontspec
4135         \fi
4136         \usepackage#1{bidi}%
4137         \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4138         \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4139           \ifnum\@nameuse\bbl@wdir\@languagename=\tw@ % 'AL' bidi
4140             \bbl@digitsdotdash % So ignore in 'R' bidi
4141           \fi}}%
4142     \fi}
4143   \ifnum\bbl@bidimode>200 % Any xe bidi=
4144     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4145       \bbl@tentative{bidi=bidi}
4146       \bbl@loadxebidi{}
4147     \or
4148       \bbl@loadxebidi{[rldocument]}
4149     \or
4150       \bbl@loadxebidi{}
4151     \fi
4152   \fi
4153 \fi
4154 \ifnum\bbl@bidimode=\@ne % bidi=default
4155   \let\bbl@beforeforeign\leavevmode
4156   \ifodd\bbl@engine % lua
4157     \newattribute\bbl@attr@dir
4158     \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4159     \bbl@exp{\output{\bodydir\pagedir\the\output}}
4160   \fi
4161   \AtEndOfPackage{%
4162     \EnableBabelHook{babel-bidi}% pdf/lua/xe
4163     \ifodd\bbl@engine\else % pdf/xe
4164       \bbl@xebidipar
4165     \fi}
4166 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4167 \bbl@trace{Macros to switch the text direction}
4168 \def\bbl@alscripts{%
4169   ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4170 \def\bbl@rscripts{%
4171   Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4172   Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4173   Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%

```

```

4174 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4175 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4176 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4177 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4178 Meroitic,N'Ko,Orkhon,Todhri}
4179 %
4180 \def\bbl@provide@dirs#1{%
4181 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4182 \ifin@
4183 \global\bbl@csarg\chardef{wdir@#1}\@ne
4184 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4185 \ifin@
4186 \global\bbl@csarg\chardef{wdir@#1}\tw@
4187 \fi
4188 \else
4189 \global\bbl@csarg\chardef{wdir@#1}\z@
4190 \fi
4191 \ifodd\bbl@engine
4192 \bbl@csarg\ifcase{wdir@#1}%
4193 \directlua{ Babel.locale_props[\the\localeid].texdir = 'l' }%
4194 \or
4195 \directlua{ Babel.locale_props[\the\localeid].texdir = 'r' }%
4196 \or
4197 \directlua{ Babel.locale_props[\the\localeid].texdir = 'al' }%
4198 \fi
4199 \fi}
4200 %
4201 \def\bbl@switchdir{%
4202 \bbl@ifunset{\bbl@lsys@{language}}{\bbl@provide@lsys@{language}}{}%
4203 \bbl@ifunset{\bbl@wdir@{language}}{\bbl@provide@dirs@{language}}{}%
4204 \bbl@expf{\bbl@setdirs\bbl@cl{wdir}}}%
4205 \def\bbl@setdirs#1{%
4206 \ifcase\bbl@select@type
4207 \bbl@bodydir{#1}%
4208 \bbl@pardir{#1}% <- Must precede \bbl@texdir
4209 \fi
4210 \bbl@texdir{#1}}
4211 \ifnum\bbl@bidimode>\z@
4212 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4213 \DisableBabelHook{babel-bidi}
4214 \fi

```

Now the engine-dependent macros.

```

4215 \ifodd\bbl@engine % luatex=1
4216 \else % pdftex=0, xetex=2
4217 \newcount\bbl@dirlevel
4218 \chardef\bbl@thetexdir\z@
4219 \chardef\bbl@thepardir\z@
4220 \def\bbl@texdir#1{%
4221 \ifcase#1\relax
4222 \chardef\bbl@thetexdir\z@
4223 \@nameuse{setlatin}%
4224 \bbl@texdir@i\beginL\endL
4225 \else
4226 \chardef\bbl@thetexdir\@ne
4227 \@nameuse{setnonlatin}%
4228 \bbl@texdir@i\beginR\endR
4229 \fi}
4230 \def\bbl@texdir@i#1#2{%
4231 \ifhmode
4232 \ifnum\currentgrouplevel>\z@
4233 \ifnum\currentgrouplevel=\bbl@dirlevel
4234 \bbl@error{multiple-bidi}{\bbl@texdir@i#1#2}{}%

```

```

4235     \bgroup\aftergroup#2\aftergroup\egroup
4236   \else
4237     \ifcase\currentgrouptype\or % 0 bottom
4238       \aftergroup#2% 1 simple {}
4239     \or
4240       \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4241     \or
4242       \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4243     \or\or\or % vbox vtop align
4244     \or
4245       \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4246     \or\or\or\or\or\or % output math disc insert vcent mathchoice
4247     \or
4248       \aftergroup#2% 14 \begingroup
4249     \else
4250       \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4251     \fi
4252   \fi
4253   \bbl@dirlevel\currentgrouplevel
4254 \fi
4255 #1%
4256 \fi}
4257 \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4258 \let\bbl@bodydir@gobble
4259 \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4260 \def\bbl@xebidipar{%
4261   \let\bbl@xebidipar\relax
4262   \TeXeTstate\@ne
4263   \def\bbl@xeeverypar{%
4264     \ifcase\bbl@thepardir
4265       \ifcase\bbl@thetextdir\else\beginR\fi
4266     \else
4267       {\setbox\z@\lastbox\beginR\box\z@}%
4268     \fi}%
4269   \AddToHook{para/begin}{\bbl@xeeverypar}}
4270 \ifnum\bbl@bidimode>200 % Any xe bidi=
4271   \let\bbl@textdir@i@gobbletwo
4272   \let\bbl@xebidipar\empty
4273   \AddBabelHook{bidi}{foreign}{%
4274     \ifcase\bbl@thetextdir
4275       \BabelWrapText{\LR{##1}}%
4276     \else
4277       \BabelWrapText{\RL{##1}}%
4278     \fi}
4279   \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4280 \fi
4281 \fi

```

A tool for weak L (mainly digits). We also disable warnings with hyperref.

```

4282 \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4283 \AtBeginDocument{%
4284   \ifx\pdfstringdefDisableCommands\undefined\else
4285     \ifx\pdfstringdefDisableCommands\relax\else
4286       \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4287     \fi
4288   \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension .cfg. For instance the file norsk.cfg will be loaded when the language definition file norsk.ldf is loaded.

For plain-based formats we don't want to override the definition of \loadlocalcfg from plain.def.

```
4289 \bbl@trace{Local Language Configuration}
4290 \ifx\loadlocalcfg\undefined
4291   \ifpackagewith{babel}{noconfigs}%
4292     {\let\loadlocalcfg@gobble}%
4293     {\def\loadlocalcfg#1{%
4294       \InputIfFileExists{#1.cfg}%
4295       {\typeout{*****^J%
4296                 * Local config file #1.cfg used^^J%
4297                 *}}%
4298       \@empty}}
4299 \fi
```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```
4300 \bbl@trace{Language options}
4301 \def\BabelDefinitionFile#1#2#3{
4302 \let\bbl@afterlang\relax
4303 \let\BabelModifiers\relax
4304 \let\bbl@loaded\@empty
4305 \def\bbl@load@language#1{%
4306   \InputIfFileExists{#1.ldf}%
4307   {\edef\bbl@loaded{\CurrentOption
4308     \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4309     \expandafter\let\expandafter\bbl@afterlang
4310       \csname\CurrentOption.ldf-h@k\endcsname
4311     \expandafter\let\expandafter\BabelModifiers
4312       \csname bbl@mod@\CurrentOption\endcsname
4313     \bbl@exp{\AtBeginDocument{%
4314       \bbl@usehooks@lang{\CurrentOption}{begindocument}{\CurrentOption}}}%
4315     {\bbl@error{unknown-package-option}}{}}{}}
```

Another way to extend the list of 'known' options for babel was to create the file bblopts.cfg in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option config=<name>, which will load <name>.cfg instead.

If the language has been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetada we also force it with \foreignlanguage (this is also done in bidi texts).

```
4316 \ifx\GetDocumentProperties\undefined\else
4317   \let\bbl@beforeforeign\leavevmode
4318   \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4319   \let\bbl@collect@other\@empty
4320   \bbl@foreach\bbl@tempa{%
4321     \edef\bbl@metalang{#1}%
4322     \ifx\bbl@metalang\@empty\else
4323       \begingroup
4324         \expandafter
4325         \bbl@bcplookup{\bbl@metalang}%-\@empty-\@empty-\@empty@@
4326         \ifx\bbl@bcp\relax
```

```

4327         \ifx\bbbl@opt@main\@nnil
4328         \bbbl@error{no-locale-for-meta}{\bbbl@metalang}{\}%
4329         \fi
4330     \else
4331         \bbbl@read@ini{\bbbl@bcp}\m@ne
4332         \xdef\bbbl@collect@other{\bbbl@collect@other,\language}%
4333         \GenericInfo{ }(babel) \spaces\spaces\spaces
4334         Passing '\language' to babel\@gobble}%
4335     \fi
4336 \endgroup
4337 \fi}
4338 \ifx\bbbl@collect@other\@empty\else
4339     \xdef\bbbl@language@opts{\bbbl@collect@other,\bbbl@language@opts}%
4340 \fi
4341 \edef\bbbl@metalang{\GetDocumentProperties{document/language}}%
4342 \ifx\bbbl@metalang\@empty\else
4343     \begingroup
4344         \expandafter
4345         \bbbl@bcplookup{\bbbl@metalang}%-\@empty-\@empty-\@empty\@@
4346         \ifx\bbbl@bcp\relax
4347             \ifx\bbbl@opt@main\@nnil
4348                 \bbbl@error{no-locale-for-meta}{\bbbl@metalang}{\}%
4349             \fi
4350         \else
4351             \bbbl@read@ini{\bbbl@bcp}\m@ne
4352             \xdef\bbbl@language@opts{\bbbl@language@opts,\language}%
4353             \ifx\bbbl@opt@main\@nnil
4354                 \global\let\bbbl@opt@main\language
4355             \fi
4356             \GenericInfo{ }(babel) \spaces\spaces\spaces
4357             Passing '\language' to babel\@gobble}%
4358         \fi
4359     \endgroup
4360 \fi
4361 \fi
4362 \ifx\bbbl@opt@config\@nnil
4363     \ifpackagewith{babel}{noconfigs}{\}%
4364     {\InputIfFileExists{bblopts.cfg}%
4365     {\bbbl@info{Configuration files are deprecated, as\%
4366         they can break document portability.\%
4367         Reported}%
4368     \typeout{*****^J%
4369         * Local config file bblopts.cfg used^J%
4370         *}}%
4371     }{\}%
4372 \else
4373     \InputIfFileExists{\bbbl@opt@config.cfg}%
4374     {\bbbl@info{Configuration files are deprecated, as\%
4375         they can break document portability.\%
4376         Reported}%
4377     \typeout{*****^J%
4378         * Local config file \bbbl@opt@config.cfg used^J%
4379         *}}%
4380     {\bbbl@error{config-not-found}{\}{\}%
4381 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\bbbl@tempa` and `\bbbl@tempb` and saves the third argument for the moment of the actual loading. If

there is no `\BabelDefinitionFile` the last element is usually empty, and the `ini` file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding `ini` file can be loaded directly (with `@import`).

```

4382 \def\BabelBeforeIni#1#2{%
4383   \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4384   \let\bbl@tempb\@empty
4385   #2%
4386   \edef\bbl@toload{%
4387     \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4388     \bbl@toload@last}%
4389   \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}
4390 \def\BabelDefinitionFile#1#2#3{%
4391   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4392   \@namedef{\bbl@preldf@CurrentOption}{#3}%
4393   \endinput}%

```

For efficiency, first preprocess the class options to remove those with `=`, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\TeX$ layer.

```

4394 \def\bbl@tempf{,}
4395 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4396   \in@{=}{#1}%
4397   \ifin@
4398     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4399   \fi}

```

Store the class/package options in a list. If there is an explicit main, it’s placed as the last option. Then loop it to read the `tex` files, which can have a `\BabelDefinitionFile`. If there is no `tex` file, we attempt loading the `ldf` for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4400 \let\bbl@toload\@empty
4401 \let\bbl@toload@last\@empty
4402 \let\bbl@unkopt\@gobble %% <- Ugly
4403 \edef\bbl@tempc{%
4404   \bbl@tempf,@,\bbl@language@opts
4405   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4406 \let\BabelLocalesTentative\bbl@tempc
4407 %
4408 \bbl@foreach\bbl@tempc{%
4409   \in@{@@}{#1}% <- Ugly
4410   \ifin@
4411     \def\bbl@unkopt##1{%
4412       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}}%
4413   \else
4414     \def\CurrentOption{#1}%
4415     \bbl@xin@{/#1//}{\bbl@toload@last}% Collapse consecutive
4416     \ifin@
4417       \lowercase{\InputIfFileExists{babel-#1.tex}}{}%
4418       \IfFileExists{#1.ldf}%
4419       {\edef\bbl@toload{%
4420         \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4421         \bbl@toload@last}%
4422         \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4423       {\bbl@unkopt{#1}}}%
4424     \fi
4425   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to ‘nil’, with a special tag), and (2) the main language. With an explicit ‘main’ language, remove repeated elements. The number 1 flags it as the main language (relevant in `ini` locales), because with 0 becomes 10.

```

4426 \ifx\bbload@opt@main\@nnil
4427 \ifx\bbload@toload@last\@empty
4428 \def\bbload@toload@last{0/0//nil//und-x-nil/nil}
4429 \bbload@info{%
4430   You haven't specified a language as a class or package\\%
4431   option. I'll load 'nil'. Reported}
4432 \fi
4433 \else
4434 \let\bbload@tempc\@empty
4435 \bbload@foreach\bbload@toload{%
4436   \bbload@xin@{/\bbload@opt@main//}{#1}%
4437   \ifin@else
4438     \bbload@add@list\bbload@tempc{#1}%
4439   \fi}
4440 \let\bbload@toload\bbload@tempc
4441 \fi
4442 \edef\bbload@toload{\bbload@toload,1\bbload@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4443 \def\AfterBabelLanguage#1{%
4444   \bbload@ifsamestring\CurrentOption{#1}{\global\bbload@add\bbload@afterlang{}}}
4445 \NewHook{babel/presets}
4446 \UseHook{babel/presets}
4447 %
4448 \let\bbload@tempb\@empty
4449 \def\bbload@tempc#1/#2//#3//#4/#5\@@{%
4450   \count@\z@
4451   \ifnum#2=\@m % if no \BabelDefinitionFile
4452     \ifnum#1=\z@ % not main. -- % if provide+=, provide*=!
4453       \ifnum\bbload@ldfflag>\@ne\bbload@tempc 0/0//#3//#4/#3\@@
4454       \else\bbload@tempd{#1}{#2}{#3}{#4}{#5}%
4455       \fi
4456     \else % 10 = main -- % if provide+=, provide*=!
4457       \ifodd\bbload@ldfflag\bbload@tempc 10/0//#3//#4/#3\@@
4458       \else\bbload@tempd{#1}{#2}{#3}{#4}{#5}%
4459       \fi
4460     \fi
4461   \else
4462     \ifnum#1=\z@ % not main
4463       \ifnum\bbload@iniflag>\@ne\else % if ø, provide
4464         \ifcase#2\count@\@ne\else\ifcase\bbload@engine\count@\@ne\fi\fi
4465       \fi
4466     \else % 10 = main
4467       \ifodd\bbload@iniflag\else % if provide+, provide*
4468         \ifcase#2\count@\@ne\else\ifcase\bbload@engine\count@\@ne\fi\fi
4469       \fi
4470     \fi
4471     \bbload@tempd{#1}{#2}{#3}{#4}{#5}%
4472   \fi}

```

Based on the value of \count@, do the actual loading. If ‘ldf’, we load the basic info from the ‘ini’ file before.

```

4473 \def\bbload@tempd#1#2#3#4#5{%
4474   \DeclareOption{#3}{}}
4475 \ifcase\count@
4476   \bbload@exp{\bbload@add\bbload@tempb{%
4477     \global\let\bbload@afterload\@empty
4478     \bbload@nameuse\bbload@preini{#3}%
4479     \bbload@ldfinit
4480     \def\CurrentOption{#3}%
4481     \bbload@babelprovide[\import=#4,\ifnum#1=\z@\else\bbload@opt@provide,main\fi]{#3}%

```

```

4482     \\bbl@afterldf
4483     \\bbl@afterload}}}%
4484 \else
4485     \bbl@add\bbl@tempb{%
4486     \global\let\bbl@afterload\@empty
4487     \def\CurrentOption{#3}%
4488     \let\localename\CurrentOption
4489     \let\languagename\localename
4490     \def\BabelIniTag{#4}%
4491     \@nameuse{bbl@preldf@#3}%
4492     \begingroup
4493         \bbl@id@assign
4494         \bbl@read@ini{\BabelIniTag}0%
4495     \endgroup
4496     \bbl@load@language{#5}%
4497     \bbl@afterload}%
4498 \fi}
4499 %
4500 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4501 \bbl@tempb
4502 \DeclareOption*{}
4503 \ProcessOptions
4504 %
4505 \bbl@exp{%
4506     \\AtBeginDocument{\\bbl@usehooks@lang{/{begindocument}}{}}}%
4507 \def\AfterBabelLanguage{\bbl@error{late-after-babel}}{}}{}
4508 \AddToHook{begindocument/end}{%
4509     \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4510 </package>

```

6. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4511 <{*kernel}
4512 \let\bbl@onlyswitch\@empty
4513 \input babel.def
4514 \let\bbl@onlyswitch\@undefined
4515 </kernel>

```

7. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^`, `M`, `%` and `=` are reset before loading the file.

```

4516 <{*errors}
4517 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4518 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4519 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4520 \catcode`\@=11 \catcode`\^=7
4521 %

```

```

4522 \ifx\MessageBreak\undefined
4523 \gdef\bbl@error@i#1#2{%
4524   \begingroup
4525     \newlinechar=`^^J
4526     \def\{^^J(babel) }%
4527     \errhelp{#2}\errmessage{\{#1}%
4528   \endgroup}
4529 \else
4530 \gdef\bbl@error@i#1#2{%
4531   \begingroup
4532     \def\{MessageBreak}%
4533     \PackageError{babel}{#1}{#2}%
4534   \endgroup}
4535 \fi
4536 \def\bbl@errmessage#1#2#3{%
4537   \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4538     \bbl@error@i{#2}{#3}}
4539 % Implicit #2#3#4:
4540 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4541 %
4542 \bbl@errmessage{not-yet-available}
4543   {Not yet available}%
4544   {Find an armchair, sit down and wait}
4545 \bbl@errmessage{bad-package-option}%
4546   {Bad option '#1=#2'. Either you have misspelled the\\%
4547   key or there is a previous setting of '#1'. Valid\\%
4548   keys are, among others, 'shorthands', 'main', 'bidi',\\%
4549   'strings', 'config', 'headfoot', 'safe', 'math'.}%
4550   {See the manual for further details.}
4551 \bbl@errmessage{base-on-the-fly}
4552   {For a language to be defined on the fly 'base'\\%
4553   is not enough, and the whole package must be\\%
4554   loaded. Either delete the 'base' option or\\%
4555   request the languages explicitly}%
4556   {See the manual for further details.}
4557 \bbl@errmessage{undefined-language}
4558   {You haven't defined the language '#1' yet.\\%
4559   Perhaps you misspelled it or your installation\\%
4560   is not complete}%
4561   {Your command will be ignored, type <return> to proceed}
4562 \bbl@errmessage{invalid-ini-name}
4563   {'#1' not valid with the 'ini' mechanism.\\%
4564   I think you want '#2' instead. You may continue,\\%
4565   but you should fix the name. See the babel manual\\%
4566   for the available locales with 'provide'}%
4567   {See the manual for further details.}
4568 \bbl@errmessage{shorthand-is-off}
4569   {I can't declare a shorthand turned off (\string#2)}
4570   {Sorry, but you can't use shorthands which have been\\%
4571   turned off in the package options}
4572 \bbl@errmessage{not-a-shorthand}
4573   {The character '\string #1' should be made a shorthand character;\\%
4574   add the command \string\usesshorthands\string{#1\string} to
4575   the preamble.\\%
4576   I will ignore your instruction}%
4577   {You may proceed, but expect unexpected results}
4578 \bbl@errmessage{not-a-shorthand-b}
4579   {I can't switch '\string#2' on or off--not a shorthand\\%
4580   This character is not a shorthand. Maybe you made\\%
4581   a typing mistake?}%
4582   {I will ignore your instruction.}
4583 \bbl@errmessage{unknown-attribute}
4584   {The attribute #2 is unknown for language #1.}%

```

```

4585 {Your command will be ignored, type <return> to proceed}
4586 \bbl@errmessage{missing-group}
4587 {Missing group for string \string#1}%
4588 {You must assign strings to some category, typically\\%
4589 captions or extras, but you set none}
4590 \bbl@errmessage{only-lua-xe}
4591 {This macro is available only in LuaLaTeX and XeLaTeX.}%
4592 {Consider switching to these engines.}
4593 \bbl@errmessage{only-lua}
4594 {This macro is available only in LuaLaTeX}%
4595 {Consider switching to that engine.}
4596 \bbl@errmessage{unknown-provide-key}
4597 {Unknown key '#1' in \string\babelprovide}%
4598 {See the manual for valid keys}%
4599 \bbl@errmessage{unknown-mapfont}
4600 {Option '\bbl@KVP@mapfont' unknown for\\%
4601 mapfont. Use 'direction'}%
4602 {See the manual for details.}
4603 \bbl@errmessage{no-ini-file}
4604 {There is no ini file for the requested language\\%
4605 (#1: \language). Perhaps you misspelled it or your\\%
4606 installation is not complete}%
4607 {Fix the name or reinstall babel.}
4608 \bbl@errmessage{digits-is-reserved}
4609 {The counter name 'digits' is reserved for mapping\\%
4610 decimal digits}%
4611 {Use another name.}
4612 \bbl@errmessage{limit-two-digits}
4613 {Currently two-digit years are restricted to the\\5
4614 range 0-9999}%
4615 {There is little you can do. Sorry.}
4616 \bbl@errmessage{counter-too-large}
4617 {Counter too large (#1)}%
4618 {Currently this is the limit.}
4619 \bbl@errmessage{no-ini-info}
4620 {I've found no info for the current locale.\\%
4621 The corresponding ini file has not been loaded\\%
4622 Perhaps it doesn't exist}%
4623 {See the manual for details.}
4624 \bbl@errmessage{unknown-ini-field}
4625 {Unknown field '#1' in \string\BCPdata.\\%
4626 Perhaps you misspelled it}%
4627 {See the manual for details.}
4628 \bbl@errmessage{unknown-locale-key}
4629 {Unknown key for locale '#2':\\%
4630 #3\\%
4631 \string#1 will be set to \string\relax}%
4632 {Perhaps you misspelled it.}%
4633 \bbl@errmessage{adjust-only-vertical}
4634 {Currently, #1 related features can be adjusted only\\%
4635 in the main vertical list}%
4636 {Maybe things change in the future, but this is what it is.}
4637 \bbl@errmessage{layout-only-vertical}
4638 {Currently, layout related features can be adjusted only\\%
4639 in vertical mode}%
4640 {Maybe things change in the future, but this is what it is.}
4641 \bbl@errmessage{bidi-only-lua}
4642 {The bidi method 'basic' is available only in\\%
4643 luatex. I'll continue with 'bidi=default', so\\%
4644 expect wrong results.\\%
4645 Suggested actions:\\%
4646 * If possible, switch to luatex, as xetex is not\\%
4647 recommend anymore.\\

```

```

4648 * If you can't, try 'bidi=bidi' with xetex.\\%
4649 * With pdftex, only 'bidi=default' is available.}%
4650 {See the manual for further details.}
4651 \bbl@errmessage{multiple-bidi}
4652 {Multiple bidi settings inside a group\\%
4653 I'll insert a new group, but expect wrong results.\\%
4654 Suggested action:\\%
4655 * Add a new group where appropriate.}
4656 {See the manual for further details.}
4657 \bbl@errmessage{unknown-package-option}
4658 {Unknown option '\CurrentOption'.\\%
4659 Suggested actions:\\%
4660 * Make sure you haven't misspelled it\\%
4661 * Check in the babel manual that it's supported\\%
4662 * If supported and it's a language, you may\\%
4663 \space\space need in some distributions a separate\\%
4664 \space\space installation\\%
4665 * If installed, check there isn't an old\\%
4666 \space\space version of the required files in your system\\%
4667 * If it's an unsupported language, create it with\\%
4668 \string\babelprovide (see the manual)}
4669 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4670 activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4671 headfoot=, strings=, config=, hyphenmap=, or a language name.}
4672 \bbl@errmessage{config-not-found}
4673 {Local config file '\bbl@opt@config.cfg' not found.\\%
4674 Suggested actions:\\%
4675 * Make sure you haven't misspelled it in config=\\%
4676 * Check it exists and it's in the correct path}%
4677 {Perhaps you misspelled it.}
4678 \bbl@errmessage{late-after-babel}
4679 {Too late for \string\AfterBabelLanguage}%
4680 {Languages have been loaded, so I can do nothing}
4681 \bbl@errmessage{double-hyphens-class}
4682 {Double hyphens aren't allowed in \string\babelcharclass\\%
4683 because it's potentially ambiguous}%
4684 {See the manual for further info}
4685 \bbl@errmessage{unknown-interchar}
4686 {'#1' for '\language' cannot be enabled.\\%
4687 Maybe there is a typo}%
4688 {See the manual for further details.}
4689 \bbl@errmessage{unknown-interchar-b}
4690 {'#1' for '\language' cannot be disabled.\\%
4691 Maybe there is a typo}%
4692 {See the manual for further details.}
4693 \bbl@errmessage{charproperty-only-vertical}
4694 {\string\babelcharproperty\space can be used only in\\%
4695 vertical mode (preamble or between paragraphs)}%
4696 {See the manual for further info}
4697 \bbl@errmessage{unknown-char-property}
4698 {No property named '#2'. Allowed values are\\%
4699 direction (bc), mirror (bmg), and linebreak (lb)}%
4700 {See the manual for further info}
4701 \bbl@errmessage{bad-transform-option}
4702 {Bad option '#1' in a transform.\\%
4703 I'll ignore it but expect more errors}%
4704 {See the manual for further info.}
4705 \bbl@errmessage{font-conflict-transforms}
4706 {Transforms cannot be re-assigned to different\\%
4707 fonts. The conflict is in '\bbl@kv@label'.\\%
4708 Apply the same fonts or use a different label}%
4709 {See the manual for further details.}
4710 \bbl@errmessage{transform-not-available}

```

```

4711 { '#1' for '\language' cannot be enabled.\\%
4712 Maybe there is a typo or it's a font-dependent transform}%
4713 {See the manual for further details.}
4714 \bbl@errmessage{transform-not-available-b}
4715 { '#1' for '\language' cannot be disabled.\\%
4716 Maybe there is a typo or it's a font-dependent transform}%
4717 {See the manual for further details.}
4718 \bbl@errmessage{year-out-range}
4719 {Year out of range.\\%
4720 The allowed range is #1}%
4721 {See the manual for further details.}
4722 \bbl@errmessage{only-pdfTeX-lang}
4723 {The '#1' ldf style doesn't work with #2,\\%
4724 but you can use the ini locale instead.\\%
4725 Try adding 'provide=*' to the option list. You may\\%
4726 also want to set 'bidi=' to some value}%
4727 {See the manual for further details.}
4728 \bbl@errmessage{hyphenmins-args}
4729 {\string\babelhyphenmins\ accepts either the optional\\%
4730 argument or the star, but not both at the same time}%
4731 {See the manual for further details.}
4732 \bbl@errmessage{no-locale-for-meta}
4733 {There isn't currently a locale for the 'lang' requested\\%
4734 in the PDF metadata ('#1'). To fix it, you can\\%
4735 set explicitly a similar language (using the same\\%
4736 script) with the key main= when loading babel. If you\\%
4737 continue, I'll fallback to the 'nil' language, with\\%
4738 tag 'und' and script 'Latn', but expect a bad font\\%
4739 rendering with other scripts. You may also need set\\%
4740 explicitly captions and date, too}%
4741 {See the manual for further details.}
4742 </errors>
4743 <*patterns>

```

8. Loading hyphenation patterns

The following code is meant to be read by `iniTeX` because it should instruct `TeX` to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4744 <@Make sure ProvidesFile is defined@>
4745 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4746 \xdef\bbl@format{\jobname}
4747 \def\bbl@version{<@version@>}
4748 \def\bbl@date{<@date@>}
4749 \ifx\AtBeginDocument\undefined
4750 \def\empty{}
4751 \fi
4752 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```

4753 \def\process@line#1#2 #3 #4 {%
4754 \ifx=#1%
4755 \process@synonym{#2}%
4756 \else
4757 \process@language{#1#2}{#3}{#4}%
4758 \fi
4759 \ignorespaces}

```

\process@synonym This macro takes care of the lines which start with an =. It needs an empty token register to begin with. \bbl@languages is also set to empty.

```
4760 \toks@{}
4761 \def\bbl@languages{}
```

When no languages have been loaded yet, the name following the = will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The \relax just helps to the \if below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the hyphenmin parameters for the synonym.

```
4762 \def\process@synonym#1{%
4763   \ifnum\last@language=\m@ne
4764     \toks@{\expandafter{\the\toks@\relax\process@synonym{#1}}}%
4765   \else
4766     \expandafter\chardef\csname l@#1\endcsname\last@language
4767     \wlog{\string\l@#1=\string\language\the\last@language}%
4768     \expandafter\let\csname #1hyphenmins\endcsname
4769       \csname\language\hyphenmins\endcsname
4770     \let\bbl@elt\relax
4771     \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%
4772   \fi}
```

\process@language The macro \process@language is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call \addlanguage to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file language.dat by adding for instance ‘:T1’ to the name of the language. The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to \lefthyphenmin and \righthyphenmin. T_EX does not keep track of these assignments. Therefore we try to detect such assignments and store them in the \<language>hyphenmins macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the \lccode en \uccode arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the \patterns command acts globally so its effect will be remembered.

Then we globally store the settings of \lefthyphenmin and \righthyphenmin and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

\bbl@languages saves a snapshot of the loaded languages in the form \bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}. Note the last 2 arguments are empty in ‘dialects’ defined in language.dat with =. Note also the language name can have encoding info.

Finally, if the counter \language is equal to zero we execute the synonyms stored.

```
4773 \def\process@language#1#2#3{%
4774   \expandafter\addlanguage\csname l@#1\endcsname
4775   \expandafter\language\csname l@#1\endcsname
4776   \edef\language{#1}%
4777   \bbl@hook@everylanguage{#1}%
4778   % > luatex
4779   \bbl@get@enc#1:.\@@@
4780   \begingroup
4781     \lefthyphenmin\m@ne
4782     \bbl@hook@loadpatterns{#2}%
4783     % > luatex
4784     \ifnum\lefthyphenmin=\m@ne
4785     \else
```



```

4786 \expandafter\xdef\csname #1hyphenmins\endcsname{%
4787 \the\lefthyphenmin\the\righthyphenmin}%
4788 \fi
4789 \endgroup
4790 \def\bbl@tempa{#3}%
4791 \ifx\bbl@tempa\@empty\else
4792 \bbl@hook@loadexceptions{#3}%
4793 % > luatex
4794 \fi
4795 \let\bbl@elt\relax
4796 \edef\bbl@languages{%
4797 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4798 \ifnum\the\language=\z@
4799 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4800 \set@hyphenmins\tw@\thr@@\relax
4801 \else
4802 \expandafter\expandafter\expandafter\set@hyphenmins
4803 \csname #1hyphenmins\endcsname
4804 \fi
4805 \the\toks@
4806 \toks@{}%
4807 \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```
4808 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}
```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4809 \def\bbl@hook@everylanguage#1{}
4810 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4811 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4812 \def\bbl@hook@loadkernel#1{%
4813 \def\addlanguage{\csname newlanguage\endcsname}%
4814 \def\adddialect##1##2{%
4815 \global\chardef##1##2\relax
4816 \wlog{\string##1 = a dialect from \string\language##2}}%
4817 \def\iflanguage##1{%
4818 \expandafter\ifx\csname l@##1\endcsname\relax
4819 \nolannerr{##1}%
4820 \else
4821 \ifnum\csname l@##1\endcsname=\language
4822 \expandafter\expandafter\expandafter\@firstoftwo
4823 \else
4824 \expandafter\expandafter\expandafter\@secondoftwo
4825 \fi
4826 \fi}%
4827 \def\providehyphenmins##1##2{%
4828 \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4829 \@namedef{##1hyphenmins}{##2}%
4830 \fi}%
4831 \def\set@hyphenmins##1##2{%
4832 \lefthyphenmin##1\relax
4833 \righthyphenmin##2\relax}%
4834 \def\selectlanguage{%
4835 \errhelp{Selecting a language requires a package supporting it}%
4836 \errmessage{No multilingual package has been loaded}}%
4837 \let\foreignlanguage\selectlanguage
4838 \let\otherlanguage\selectlanguage
4839 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4840 \def\bbl@usehooks##1##2{%

```

```

4841 \def\setlocale{%
4842   \errhelp{Find an armchair, sit down and wait}%
4843   \errmessage{(babel) Not yet available}}%
4844 \let\uselocale\setlocale
4845 \let\locale\setlocale
4846 \let\selectlocale\setlocale
4847 \let\localename\setlocale
4848 \let\textlocale\setlocale
4849 \let\textlanguage\setlocale
4850 \let\languagetext\setlocale}
4851 \begingroup
4852 \def\AddBabelHook#1#2{%
4853   \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4854   \def\next{\toks1}%
4855   \else
4856     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname####1}%
4857   \fi
4858   \next}
4859 \ifx\directlua\@undefined
4860   \ifx\XeTeXinputencoding\@undefined\else
4861     \input xebabel.def
4862   \fi
4863   \else
4864     \input luababel.def
4865   \fi
4866   \openin1 = babel-\bbl@format.cfg
4867   \ifeof1
4868   \else
4869     \input babel-\bbl@format.cfg\relax
4870   \fi
4871   \closein1
4872 \endgroup
4873 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4874 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4875 \def\language{english}%
4876 \ifeof1
4877   \message{I couldn't find the file language.dat,\space
4878           I will try the file hyphen.tex}
4879   \input hyphen.tex\relax
4880   \chardef\@english\z@
4881 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```

4882 \last@language\m@ne

```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```

4883 \loop
4884   \endlinechar\m@ne
4885   \read1 to \bbl@line
4886   \endlinechar\^^M

```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```

4887 \if T\ifeof1F\fi T\relax
4888 \ifx\bbl@line\empty\else
4889 \edef\bbl@line{\bbl@line\space\space\space}%
4890 \expandafter\process@line\bbl@line\relax
4891 \fi
4892 \repeat

```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```

4893 \begingroup
4894 \def\bbl@elt#1#2#3#4{%
4895 \global\language=#2\relax
4896 \gdef\language#1}%
4897 \def\bbl@elt##1##2##3##4{}}%
4898 \bbl@languages
4899 \endgroup
4900 \fi
4901 \closein1

```

We add a message about the fact that babel is loaded in the format and with which language patterns to the `\everyjob` register.

```

4902 \if/\the\toks@\else
4903 \errhelp{language.dat loads no language, only synonyms}
4904 \errmessage{Orphan language synonym}
4905 \fi

```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```

4906 \let\bbl@line\@undefined
4907 \let\process@line\@undefined
4908 \let\process@synonym\@undefined
4909 \let\process@language\@undefined
4910 \let\bbl@get@enc\@undefined
4911 \let\bbl@hyph@enc\@undefined
4912 \let\bbl@tempa\@undefined
4913 \let\bbl@hook@loadkernel\@undefined
4914 \let\bbl@hook@everylanguage\@undefined
4915 \let\bbl@hook@loadpatterns\@undefined
4916 \let\bbl@hook@loadexceptions\@undefined
4917 </patterns>

```

Here the code for `iniTeX` ends.

9. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdfTeX`).

```

4918 <<More package options>> ≡
4919 \chardef\bbl@bidimode\z@
4920 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4921 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4922 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4923 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4924 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4925 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4926 <</More package options>>

```

\belfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `bbl@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```

4927 <<Font selection>> ≡

```

```

4928 \bbl@trace{Font handling with fontspec}
4929 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4930 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@ckckstdfonts}
4931 \DisableBabelHook{babel-fontspec}
4932 \@onlypreamble\bblfont
4933 \ifx\NewDocumentCommand\undefined\else % Not plain
4934   \NewDocumentCommand\bblfont{0}{m0}{m0}}{%
4935     \bbl@bblfont@o[#1]{#2}{#3,#5}{#4}}
4936 \fi
4937 \newcommand\bbl@bblfont@o[2][]{% 1=langs/scripts 2=fam
4938   \ifx\fontspec\undefined
4939     \usepackage{fontspec}%
4940   \fi
4941   \EnableBabelHook{babel-fontspec}%
4942   \edef\bbl@tempa{#1}%
4943   \def\bbl@tempb{#2}% Used by \bbl@bblfont
4944   \bbl@bblfont}
4945 \newcommand\bbl@bblfont[2][]{% 1=features 2=fontname, @font=rm|sf|tt
4946   \bbl@ifunset{\bbl@tempb family}%
4947     {\bbl@providefam{\bbl@tempb}}%
4948     {}%
4949   % For the default font, just in case:
4950   \bbl@ifunset{\bbl@lsys{\languagename}}{\bbl@provide@lsys{\languagename}}{%
4951     \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
4952     {\bbl@csarg\edef{\bbl@tempb dflt@}{<>{#1}{#2}}% save bbl@rmdflt@
4953     \bbl@exp{%
4954       \let<\bbl@tempb dflt@\languagename>\<\bbl@tempb dflt@>%
4955       \\\bbl@font@set<\bbl@tempb dflt@\languagename>%
4956       \<\bbl@tempb default>\<\bbl@tempb family>}}%
4957     {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
4958       \bbl@csarg\def{\bbl@tempb dflt@##1}{<>{#1}{#2}}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4959 \def\bbl@providefam#1{%
4960   \bbl@exp{%
4961     \\\newcommand<#1default>{}% Just define it
4962     \\\bbl@add@list\\bbl@font@fams{#1}%
4963     \\\NewHook{#1family}%
4964     \\\DeclareRobustCommand<#1family>{%
4965       \\\not@math@alphabet<#1family>\relax
4966       % \\\prepare@family@series@update{#1}<#1default>% TODO. Fails
4967       \\\fontfamily<#1default>%
4968       \\\UseHook{#1family}%
4969       \\\selectfont}%
4970     \\\DeclareTextFontCommand{\<text#1>}{\<#1family>}}}

```

The following macro is activated when the hook babel-fontspec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4971 \def\bbl@nostdfont#1{%
4972   \bbl@once{nostdfam-f@family}%
4973   {\bbl@infowarn{The current font is not a babel standard family:\\%
4974     #1%
4975     \fontname\font\\%
4976     There is nothing intrinsically wrong, and you can\\%,
4977     ignore this message altogether if you do not need\\%
4978     this font. If they are used in the document, be aware\\%
4979     'babel' will not set Script and Language for it, so\\%
4980     you may consider defining a new family with \string\bblfont.\\%
4981     See the manual for further details about \string\bblfont.
4982     Reported}}%
4983   }%
4984 \gdef\bbl@switchfont{%
4985   \bbl@ifunset{\bbl@lsys{\languagename}}{\bbl@provide@lsys{\languagename}}{%
4986     \bbl@exp{% e.g., Arabic -> arabic

```

```

4987 \lowercase{\edef\\bbl@tempa{\bbl@cl{sname}}}}}%
4988 \bbl@foreach\bbl@font@fams{%
4989 \bbl@ifunset{\bbl@##1dflt@\language}% (1) language?
4990 {\bbl@ifunset{\bbl@##1dflt*@\bbl@tempa}% (2) from script?
4991 {\bbl@ifunset{\bbl@##1dflt@}% 2=F - (3) from generic?
4992 {}% 123=F - nothing!
4993 {\bbl@exp{% 3=T - from generic
4994 \global\let<\bbl@##1dflt@\language>%
4995 \<\bbl@##1dflt@>}}}%
4996 {\bbl@exp{% 2=T - from script
4997 \global\let<\bbl@##1dflt@\language>%
4998 \<\bbl@##1dflt*@\bbl@tempa>}}}%
4999 {}}% 1=T - language, already defined
5000 \def\bbl@tempa{\bbl@nostdfont{}}}%
5001 \bbl@foreach\bbl@font@fams{% don't gather with prev for
5002 \bbl@ifunset{\bbl@##1dflt@\language}%
5003 {\bbl@cs{famrst@##1}%
5004 \global\bbl@csarg\let{famrst@##1}\relax}%
5005 {\bbl@exp{% order is relevant.
5006 \\bbl@add\\originalTeX{%
5007 \\bbl@font@rst{\bbl@cl{##1dflt}}}%
5008 \<##1default>\<##1family>{##1}}}%
5009 \\bbl@font@set<\bbl@##1dflt@\language>% the main part!
5010 \<##1default>\<##1family>}}}%
5011 \bbl@ifrestoring{\bbl@tempa}}}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with `\babel font`.

```

5012 \ifx\f@family\undefined\else % if latex
5013 \ifcase\bbl@engine % if pdftex
5014 \let\bbl@ckeckstdfonts\relax
5015 \else
5016 \def\bbl@ckeckstdfonts{%
5017 \begingroup
5018 \global\let\bbl@ckeckstdfonts\relax
5019 \let\bbl@tempa\empty
5020 \bbl@foreach\bbl@font@fams{%
5021 \bbl@ifunset{\bbl@##1dflt@}%
5022 {\@nameuse{##1family}%
5023 \bbl@csarg\gdef{WFF@f@family}{}% Flag
5024 \bbl@exp{\\bbl@add\\bbl@tempa{* \<##1family>= \f@family\\}%
5025 \space\space\fontname\font\\}%
5026 \bbl@csarg\xdef{##1dflt@}{\f@family}%
5027 \expandafter\xdef\csname ##1default\endcsname{\f@family}}}%
5028 {}}%
5029 \ifx\bbl@tempa\empty\else
5030 \bbl@infowarn{The following font families will use the default\\%
5031 settings for all or some languages:\\%
5032 \bbl@tempa
5033 There is nothing intrinsically wrong with it, but\\%
5034 'babel' will no set Script and Language, which could\\%
5035 be relevant in some languages. If your document uses\\%
5036 these families, consider redefining them with \string\babelfont.\\%
5037 Reported}%
5038 \fi
5039 \endgroup}
5040 \fi
5041 \fi

```

Now the macros defining the font with `fontspec`.

When there are repeated keys in `fontspec`, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a

single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes `bx/sc` is the correct font, but sometimes points to `b/n`, even if `b/sc` exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5042 \def\bbfont@set#1#2#3{% e.g., \bb@rmdflt@lang \rmdefault \rmfamily
5043 \bb@xin@{<>}{#1}%
5044 \ifin@
5045 \bb@exp{\bb@fontspec@set\#1\expandafter\@gobbletwo#1\#3}%
5046 \fi
5047 \bb@exp{% 'Unprotected' macros return prev values
5048 \def\#2{#1}% e.g., \rmdefault{\bb@rmdflt@lang}
5049 \bb@ifsamestring{#2}{\f@family}%
5050 {\#3%
5051 \bb@ifsamestring{\f@series}{\bfdefault}{\bfseries}}%
5052 \let\bb@tempa\relax}%
5053 {}}}
```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5054 \def\bbfont@set#1#2#3#4{% eg \bb@rmdflt@lang fnt-opt fnt-nme \xxfamily
5055 \let\bb@tempe\bb@mapselect
5056 \edef\bb@tempb{\bb@stripslash#4}% Catcodes hack (better pass it).
5057 \bb@exp{\bb@replace\bb@tempb{\bb@stripslash\family/}}}%
5058 \let\bb@mapselect\relax
5059 \let\bb@temp@fam#4% e.g., '\rmfamily', to be restored below
5060 \let#4@empty % Make sure \renewfontfamily is valid
5061 \bb@set@renderer
5062 \bb@exp{%
5063 \let\bb@temp@pfam<\bb@stripslash#4\space>% e.g., '\rmfamily '
5064 \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bb@cl{sname}}%
5065 {\newfontscript{\bb@cl{sname}}{\bb@cl{sotf}}}%
5066 \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bb@cl{lname}}%
5067 {\newfontlanguage{\bb@cl{lname}}{\bb@cl{lotf}}}%
5068 \renewfontfamily\#4%
5069 [\bb@cl{lsys},% xetex removes unknown features :-(
5070 \ifcase\bb@engine\or RawFeature={family=\bb@tempb},\fi
5071 #2}}{#3}% i.e., \bb@exp{...}{#3}
5072 \bb@unset@renderer
5073 \begingroup
5074 #4%
5075 \xdef#1{\f@family}% e.g., \bb@rmdflt@lang{FreeSerif(0)}
5076 \endgroup
5077 \bb@xin@{\detokenize{>ssub*}}%
5078 {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5079 \ifin@
5080 \global\bb@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5081 \fi
5082 \bb@xin@{\detokenize{>ssub*}}%
5083 {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5084 \ifin@
5085 \global\bb@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5086 \fi
5087 \let#4\bb@temp@fam
5088 \bb@exp{\let<\bb@stripslash#4\space>\bb@temp@pfam
5089 \let\bb@mapselect\bb@tempe}%

```

`font@rst` and `famrst` are only used when there is no global settings, to save and restore previous families. Not really necessary, but done for optimization.

```

5090 \def\bbfont@rst#1#2#3#4{%

```

```
5091 \bbl@csarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}
```

The default font families. They are eurocentric, but the list can be expanded easily with `\babelfont`.

```
5092 \def\bbl@font@fams{rm,sf,tt}
5093 <</Font selection>>
```

10. Hooks for XeTeX and LuaTeX

10.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to `utf8`, which seems a sensible default.

Now, the code.

```
5094 <*<xtex>
5095 \def\BabelStringsDefault{unicode}
5096 \let\xebbl@stop\relax
5097 \AddBabelHook{xetex}{encodedcommands}{%
5098   \def\bbl@tempa{#1}%
5099   \ifx\bbl@tempa\@empty
5100     \XeTeXinputencoding"bytes"%
5101   \else
5102     \XeTeXinputencoding"#1"%
5103   \fi
5104   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5105 \AddBabelHook{xetex}{stopcommands}{%
5106   \xebbl@stop
5107   \let\xebbl@stop\relax}
5108 \def\bbl@input@classes{% Used in CJK intraspaces
5109   \input{load-unicode-xetex-classes.tex}%
5110   \let\bbl@input@classes\relax}
5111 \def\bbl@intraspace#1 #2 #3\@@{%
5112   \bbl@csarg\gdef{xeisp@\languagename}%
5113     {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5114 \def\bbl@intrapenalty#1\@@{%
5115   \bbl@csarg\gdef{xeipn@\languagename}%
5116     {\XeTeXlinebreakpenalty #1\relax}}
5117 \def\bbl@provide@intraspace{%
5118   \bbl@xin@{/s}{\bbl@cl{lnbrk}}}%
5119   \ifin@else\bbl@xin@{/c}{\bbl@cl{lnbrk}}}\fi
5120   \ifin@
5121     \bbl@ifunset{bbl@intsp@\languagename}{}%
5122     {\expandafter\ifx\csname bbl@intsp@\languagename\endcsname\@empty\else
5123       \ifx\bbl@KVP@intraspace\@nnil
5124         \bbl@exp{%
5125           \\bbl@intraspace\bbl@cl{intsp}\@@}%
5126       \fi
5127       \ifx\bbl@KVP@intrapenalty\@nnil
5128         \bbl@intrapenalty0\@@
5129       \fi
5130     \fi
5131     \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5132       \expandafter\bbl@intraspace\bbl@KVP@intraspace\@@
5133     \fi
5134     \ifx\bbl@KVP@intrapenalty\@nnil\else
5135       \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5136     \fi
5137     \bbl@exp{%
5138       \\bbl@add<extras\languagename>{%
5139         \XeTeXlinebreaklocale "\bbl@cl{tbcpr}"%
5140         <bbl@xeisp@\languagename>%
5141         <bbl@xeipn@\languagename>%
```

```

5142      \\bbl@tglobal\<extras\language>%
5143      \\bbl@add\<noextras\language>{%
5144      \XeTeXlinebreaklocale ""}%
5145      \\bbl@tglobal\<noextras\language>}%
5146      \ifx\bbl@ispace\undefined
5147      \gdef\bbl@ispace{\bbl@c{Xeisp}}%
5148      \ifx\AtBeginDocument\@notprerr
5149      \expandafter\@secondoftwo % to execute right now
5150      \fi
5151      \AtBeginDocument{\bbl@patchfont{\bbl@ispace}}%
5152      \fi}%
5153  \fi}
5154  \ifx\DisableBabelHook\undefined\endinput\fi
5155  \let\bbl@set@renderer\relax
5156  \let\bbl@unset@renderer\relax
5157  <@Font selection@>
5158  \def\bbl@provide@extra#1{}

Hack for unhyphenated line breaking. See \bbl@provide@lsys in the common code.

5159  \def\bbl@xeno-hyph@d{%
5160    \bbl@ifset{\bbl@prehc\language}%
5161    {\ifnum\hyphenchar\font=\default-hyphenchar
5162      \iffontchar\font\bbl@c{prehc}\relax
5163      \hyphenchar\font\bbl@c{prehc}\relax
5164      \else\iffontchar\font"200B
5165      \hyphenchar\font"200B
5166      \else
5167      \bbl@warning
5168      {Neither 0 nor ZERO WIDTH SPACE are available\\%
5169      in the current font, and therefore the hyphen\\%
5170      will be printed. Try changing the fontspec's\\%
5171      'HyphenChar' to another value, but be aware\\%
5172      this setting is not safe (see the manual).\\%
5173      Reported}%
5174      \hyphenchar\font\default-hyphenchar
5175      \fi\fi
5176      \fi}%
5177      {\hyphenchar\font\default-hyphenchar}}

```

10.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5178  \ifnum\Xe@alloc@intercharclass<\thr@@
5179  \Xe@alloc@intercharclass\thr@@
5180  \fi
5181  \chardef\bbl@xe@class@default=\z@
5182  \chardef\bbl@xe@class@cjkideogram=\@ne
5183  \chardef\bbl@xe@class@cjkleftpunctuation=\tw@
5184  \chardef\bbl@xe@class@cjkrightpunctuation=\thr@@
5185  \chardef\bbl@xe@class@boundary=4095
5186  \chardef\bbl@xe@class@ignore=4096

```

The machinery is activated with a hook (enabled only if actually used). Here \bbl@tempc is pre-set with \bbl@usingxe@class, defined below. The standard mechanism based on \originalTeX to save, set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```

5187  \AddBabelHook{babel-interchar}{beforeextras}{%
5188    \@nameuse{\bbl@xechars\language}}
5189  \DisableBabelHook{babel-interchar}
5190  \protected\def\bbl@charclass#1{%
5191    \ifnum\count@<\z@
5192      \count@-\count@

```



```

5193 \loop
5194 \bbl@exp{%
5195 \\\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5196 \XeTeXcharclass\count@ \bbl@tempc
5197 \ifnum\count@<`#1\relax
5198 \advance\count@\@ne
5199 \repeat
5200 \else
5201 \babel@savevariable{\XeTeXcharclass`#1}%
5202 \XeTeXcharclass`#1 \bbl@tempc
5203 \fi
5204 \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above has internally the form \bbl@usingxeclass\bbl@xeclass@punct@english\bbl@charclass{.} \bbl@charclass{,} (etc.), where \bbl@usingxeclass stores the class to be applied to the subsequent characters. The \ifcat part deals with the alternative way to enter characters as macros (e.g., \). As a special case, hyphens are stored as \bbl@upto, to deal with ranges.

```

5205 \newcommand\bbl@ifinterchar[1]{%
5206 \let\bbl@tempa\@gobble % Assume to ignore
5207 \edef\bbl@tempb{\zap@space#1 \@empty}%
5208 \ifx\bbl@KVP@interchar\@nnil\else
5209 \bbl@replace\bbl@KVP@interchar{ },{,%
5210 \bbl@foreach\bbl@tempb{%
5211 \bbl@xin@{,##1,},{,\bbl@KVP@interchar,%
5212 \ifin@
5213 \let\bbl@tempa\@firstofone
5214 \fi}%
5215 \fi
5216 \bbl@tempa}
5217 \newcommand\IfBabelIntercharT[2]{%
5218 \bbl@carg\bbl@add{\bbl@icsave@CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5219 \newcommand\babelcharclass[3]{%
5220 \EnableBabelHook{babel-interchar}%
5221 \bbl@csarg\newXeTeXintercharclass{xeclass@#2@#1}%
5222 \def\bbl@tempb##1{%
5223 \ifx##1\@empty\else
5224 \ifx##1-%
5225 \bbl@upto
5226 \else
5227 \bbl@charclass{%
5228 \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5229 \fi
5230 \expandafter\bbl@tempb
5231 \fi}%
5232 \bbl@ifunset{\bbl@xechars@#1}%
5233 {\toks@{%
5234 \babel@savevariable\XeTeXinterchartokenstate
5235 \XeTeXinterchartokenstate\@ne
5236 }}%
5237 {\toks@\expandafter\expandafter\expandafter{%
5238 \csname bbl@xechars@#1\endcsname}}%
5239 \bbl@csarg\edef{xechars@#1}{%
5240 \the\toks@
5241 \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5242 \bbl@tempb#3@empty}}
5243 \protected\def\bbl@usingxeclass#1{\count@\z@ \let\bbl@tempc#1}
5244 \protected\def\bbl@upto{%
5245 \ifnum\count@>\z@
5246 \advance\count@\@ne
5247 \count@-\count@
5248 \else\ifnum\count@=\z@

```

```

5249 \bbl@charclass{-}%
5250 \else
5251 \bbl@error{double-hyphens-class}{}{}{}%
5252 \fi\fi}

And finally, the command with the code to be inserted. If the language doesn't define a class, then
use the global one, as defined above. For the definition there is a intermediate macro, which can be
'disabled' with \bbl@ic@<label>@<language>.

5253 \def\bbl@ignoreinterchar{%
5254 \ifnum\language=\l@nohyphenation
5255 \expandafter\@gobble
5256 \else
5257 \expandafter\@firstofone
5258 \fi}
5259 \newcommand\babelinterchar[5][]{%
5260 \let\bbl@kv@label\@empty
5261 \bbl@forkv{#1}{\bbl@csarg\edef{kv@##1}{##2}}%
5262 \@namedef{\zap@space \bbl@xeinter@\bbl@kv@label @#3@#4@#2 \@empty}%
5263 {\bbl@ignoreinterchar{#5}}%
5264 \bbl@csarg\let{ic@\bbl@kv@label @#2}\@firstofone
5265 \bbl@exp{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%
5266 \bbl@exp{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%
5267 \XeTeXinterchartoks
5268 \@nameuse{\bbl@xeiclass@\bbl@tempa @#2}
5269 \bbl@ifunset{\bbl@xeiclass@\bbl@tempa @#2}{}{#2}} %
5270 \@nameuse{\bbl@xeiclass@\bbl@tempb @#2}
5271 \bbl@ifunset{\bbl@xeiclass@\bbl@tempb @#2}{}{#2}} %
5272 = \expandafter}%
5273 \csname \bbl@ic@\bbl@kv@label @#2\expandafter\endcsname
5274 \csname\zap@space \bbl@xeinter@\bbl@kv@label
5275 @#3@#4@#2 \@empty\endcsname}}}%
5276 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5277 \bbl@ifunset{\bbl@ic@#1@<language>}%
5278 {\bbl@error{unknown-interchar}{#1}{}{}}%
5279 {\bbl@csarg\let{ic@#1@<language>}\@firstofone}}
5280 \DeclareRobustCommand\disablelocaleinterchar[1]{%
5281 \bbl@ifunset{\bbl@ic@#1@<language>}%
5282 {\bbl@error{unknown-interchar-b}{#1}{}{}}%
5283 {\bbl@csarg\let{ic@#1@<language>}\@gobble}}
5284 </xetex>

```

10.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titleps, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdf_{TE}X and xet_{EX}.

```

5285 < *xetex | texxet>
5286 \providecommand\bbl@provide@intraspace{}
5287 \bbl@trace{Redefinitions for bidi layout}

```

Finish here if there in no layout.

```

5288 \ifx\bbl@opt@layout\@nnil\else % if layout=.
5289 \ifx\bbl@opt@layout\@undefined\else % Plain
5290 \IfBabelLayout{nopars}
5291 {}
5292 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5293 \fi
5294 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5295 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}

```

```

5296 \ifnum\bbl@bidimode>\z@
5297 \IfBabelLayout{pars}
5298 {\def\hangfrom#1{%
5299   \setbox\@tempboxa\hbox{#{1}}%
5300   \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5301   \noindent\box\@tempboxa}
5302 \def\raggedright{%
5303   \let\\\@centercr
5304   \bbl@startskip\z@skip
5305   \@rightskip\@flushglue
5306   \bbl@endskip\@rightskip
5307   \parindent\z@
5308   \parfillskip\bbl@startskip}
5309 \def\raggedleft{%
5310   \let\\\@centercr
5311   \bbl@startskip\@flushglue
5312   \bbl@endskip\z@skip
5313   \parindent\z@
5314   \parfillskip\bbl@endskip}}
5315 {}
5316 \fi
5317 \IfBabelLayout{lists}
5318 {\bbl@sreplace\list
5319   {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5320   \def\bbl@listleftmargin{%
5321     \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5322   \ifcase\bbl@engine
5323     \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5324     \def\p@enumiii{\p@enumii}\theenumii}%
5325   \fi
5326   \bbl@sreplace\@verbatim
5327   {\leftskip\@totalleftmargin}%
5328   {\bbl@startskip\textwidth
5329     \advance\bbl@startskip-\linewidth}%
5330   \bbl@sreplace\@verbatim
5331   {\rightskip\z@skip}%
5332   {\bbl@endskip\z@skip}}%
5333 {}
5334 \IfBabelLayout{contents}
5335 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5336   \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5337 {}
5338 \IfBabelLayout{columns}
5339 {\bbl@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bbl@outpuhbox}%
5340   \def\bbl@outpuhbox#1{%
5341     \hb@xt@\textwidth{%
5342       \hskip\columnwidth
5343       \hfil
5344       {\normalcolor\vrule \@width\columnseprule}%
5345       \hfil
5346       \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5347       \hskip-\textwidth
5348       \hb@xt@\columnwidth{\box\@outputbox \hss}%
5349       \hskip\columnsep
5350       \hskip\columnwidth}}}%
5351 {}

```

Implicitly reverses sectioning labels in `bid=basic`, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5352 \IfBabelLayout{counters*}%
5353 {\bbl@add\bbl@opt@layout{.counters.}%
5354   \AddToHook{shipout/before}{%
5355     \let\bbl@tempa\babelsublr

```

```

5356 \let\babelsublr\@firstofone
5357 \let\bbl@save@thepage\thepage
5358 \protected@edef\thepage{\thepage}%
5359 \let\babelsublr\bbl@tempa}%
5360 \AddToHook{shipout/after}{%
5361 \let\thepage\bbl@save@thepage}}{}
5362 \IfBabelLayout{counters}%
5363 {\let\bbl@latinarabic=\@arabic
5364 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}}%
5365 \let\bbl@asciroman=\@roman
5366 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5367 \let\bbl@asciiRoman=\@Roman
5368 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5369 \fi % end if layout
5370 </xetex | texxet>

```

10.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```

5371 < *texxet>
5372 \def\bbl@provide@extra#1{%
5373 % == auto-select encoding ==
5374 \ifx\bbl@encoding@select@off\@empty\else
5375 \bbl@ifunset{\bbl@encoding@#1}%
5376 {\def\@elt##1{,##1,}%
5377 \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5378 \count@z@
5379 \bbl@foreach\bbl@tempe{%
5380 \def\bbl@tempd{##1}% Save last declared
5381 \advance\count@\@one}%
5382 \ifnum\count@>\@one % (1)
5383 \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5384 \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5385 \bbl@replace\bbl@tempa{ },}%
5386 \global\bbl@csarg\let{encoding@#1}\@empty
5387 \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5388 \ifin@else % if main encoding included in ini, do nothing
5389 \let\bbl@tempb\relax
5390 \bbl@foreach\bbl@tempa{%
5391 \ifx\bbl@tempb\relax
5392 \bbl@xin@{,##1,}{,\bbl@tempe,}%
5393 \ifin@\def\bbl@tempb{##1}\fi
5394 \fi}%
5395 \ifx\bbl@tempb\relax\else
5396 \bbl@exp{%
5397 \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>%
5398 \gdef\<bbl@encoding@#1>{%
5399 \\\babel@save\\f@encoding
5400 \\\bbl@add\\originalTeX{\\selectfont}%
5401 \\\fontencoding{\bbl@tempb}%
5402 \\\selectfont}}%
5403 \fi
5404 \fi
5405 \fi}%
5406 {}%
5407 \fi}
5408 </texxet>

```

10.5. LuaTeX

The loader for luatex is based solely on language.dat, which is read on the fly. The code shouldn't be executed when the format is build, so we check if \AddBabelHook is defined. Then comes a modified

version of the loader in `hyphen.cfg` (without the `hyphenmins` stuff, which is under the direct control of `babel`).

The names `\l@{language}` are defined and take some value from the beginning because all `ldf` files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the `ldf` finishes). If a language has been loaded, `\bbl@hyphendata@{num}` exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for ‘english’, so that it’s available without further intervention from the user. To avoid duplicating it, the following rule applies: if the “0th” language and the first language in `language.dat` have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won’t at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn’t happen very often – with `luatex` patterns are best loaded when the document is typeset, and the “0th” language is preloaded just for backwards compatibility.

As of 1.1b, `lua(e)tex` is taken into account. Formerly, loading of patterns on the fly didn’t work in this format, but with the new loader it does. Unfortunately, the format is not based on `babel`, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format `language.dat` is used (under the principle of a single source), instead of `language.def`.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by `babel`) provide a command to allocate them (although there are packages like `ctablestack`). FIX - This isn’t true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, `etex.sty` changes the way languages are allocated.

This files is read at three places: (1) when `plain.def`, `babel.sty` starts, to read the list of available languages from `language.dat` (for the base option); (2) at `hyphen.cfg`, to modify some macros; (3) in the middle of `plain.def` and `babel.sty`, by `babel.def`, with the commands and other definitions for `luatex` (e.g., `\babelpatterns`).

```

5409 (*luatex)
5410 \directlua{ Babel = Babel or {} } % DL2
5411 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5412 \bbl@trace{Read language.dat}
5413 \ifx\bbl@readstream\undefined
5414 \csname newread\endcsname\bbl@readstream
5415 \fi
5416 \beginngroup
5417 \toks@{}
5418 \count@\z@ % 0=start, 1=0th, 2=normal
5419 \def\bbl@process@line#1#2 #3 #4 {%
5420 \ifx=#1%
5421 \bbl@process@synonym{#2}%
5422 \else
5423 \bbl@process@language{#1#2}{#3}{#4}%
5424 \fi
5425 \ignorespaces}
5426 \def\bbl@manylang{%
5427 \ifnum\bbl@last>\@ne
5428 \bbl@info{Non-standard hyphenation setup}%
5429 \fi
5430 \let\bbl@manylang\relax}
5431 \def\bbl@process@language#1#2#3{%
5432 \ifcase\count@
5433 \@ifundefined{zth#1}{\count@\tw@}{\count@\@ne}%
5434 \or
5435 \count@\tw@
5436 \fi
5437 \ifnum\count@=\tw@
5438 \expandafter\addlanguage\csname l@#1\endcsname
5439 \language\allocationnumber
5440 \chardef\bbl@last\allocationnumber
5441 \bbl@manylang

```

```

5442 \let\bbl@elt\relax
5443 \xdef\bbl@languages{%
5444 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5445 \fi
5446 \the\toks@
5447 \toks@{}}
5448 \def\bbl@process@synonym@aux#1#2{%
5449 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5450 \let\bbl@elt\relax
5451 \xdef\bbl@languages{%
5452 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5453 \def\bbl@process@synonym#1{%
5454 \ifcase\count@
5455 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5456 \or
5457 \@ifundefined{zth#1}{\bbl@process@synonym@aux{#1}{0}}}%
5458 \else
5459 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5460 \fi}
5461 \ifx\bbl@languages\undefined % Just a (sensible?) guess
5462 \chardef\l@english\z@
5463 \chardef\l@USenglish\z@
5464 \chardef\bbl@last\z@
5465 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}
5466 \gdef\bbl@languages{%
5467 \bbl@elt{english}{0}{hyphen.tex}}%
5468 \bbl@elt{USenglish}{0}{}}
5469 \else
5470 \global\let\bbl@languages@format\bbl@languages
5471 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5472 \ifnum#2>\z@\else
5473 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5474 \fi}%
5475 \xdef\bbl@languages{\bbl@languages}%
5476 \fi
5477 \def\bbl@elt#1#2#3#4{\@namedef{zth#1}{}} % Define flags
5478 \bbl@languages
5479 \openin\bbl@readstream=language.dat
5480 \ifeof\bbl@readstream
5481 \bbl@warning{I couldn't find language.dat. No additional\\%
5482 patterns loaded. Reported}%
5483 \else
5484 \loop
5485 \endlinechar\m@ne
5486 \read\bbl@readstream to \bbl@line
5487 \endlinechar\^^M
5488 \if T\ifeof\bbl@readstream F\fi T\relax
5489 \ifx\bbl@line\empty\else
5490 \edef\bbl@line{\bbl@line\space\space\space}%
5491 \expandafter\bbl@process@line\bbl@line\relax
5492 \fi
5493 \repeat
5494 \fi
5495 \closein\bbl@readstream
5496 \endgroup
5497 \bbl@trace{Macros for reading patterns files}
5498 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5499 \ifx\babelcatcodetablenum\undefined
5500 \ifx\newcatcodetable\undefined
5501 \def\babelcatcodetablenum{5211}
5502 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5503 \else
5504 \newcatcodetable\babelcatcodetablenum

```

```

5505 \newcatcodetable\babel@pattcodes
5506 \fi
5507 \else
5508 \def\babel@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5509 \fi
5510 \def\babel@luapatterns#1#2{%
5511 \babel@get@enc#1::\@@@
5512 \setbox\z@\hbox\bgroup
5513 \begingroup
5514 \savecatcodetable\babelcatcodetablenum\relax
5515 \initcatcodetable\babel@pattcodes\relax
5516 \catcodetable\babel@pattcodes\relax
5517 \catcode\#=6 \catcode\$=3 \catcode\&=4 \catcode\^=7
5518 \catcode\_ =8 \catcode\{=1 \catcode\}=2 \catcode\~=13
5519 \catcode\@=11 \catcode\^^I=10 \catcode\^^J=12
5520 \catcode\<=12 \catcode\>=12 \catcode\*=12 \catcode\.=12
5521 \catcode\-=12 \catcode\/=12 \catcode\[=12 \catcode\]=12
5522 \catcode\`=12 \catcode\'=12 \catcode\"=12
5523 \input #1\relax
5524 \catcodetable\babelcatcodetablenum\relax
5525 \endgroup
5526 \def\babel@tempa{#2}%
5527 \ifx\babel@tempa\@empty\else
5528 \input #2\relax
5529 \fi
5530 \egroup}%
5531 \def\babel@patterns@lua#1{%
5532 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5533 \csname l@#1\endcsname
5534 \edef\babel@tempa{#1}%
5535 \else
5536 \csname l@#1:\f@encoding\endcsname
5537 \edef\babel@tempa{#1:\f@encoding}%
5538 \fi\relax
5539 \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5540 \@ifundefined{babel@hyphendata@the\language}%
5541 {\def\babel@elt##1##2##3##4{%
5542 \ifnum##2=\csname l@babel@tempa\endcsname % #2=spanish, dutch:OT1...
5543 \def\babel@tempb{##3}%
5544 \ifx\babel@tempb\@empty\else % if not a synonymous
5545 \def\babel@tempc{##3}{##4}%
5546 \fi
5547 \babel@csarg\xdef{hyphendata@##2}{\babel@tempc}%
5548 \fi}%
5549 \babel@languages
5550 \@ifundefined{babel@hyphendata@the\language}%
5551 {\babel@info{No hyphenation patterns were set for\%
5552 language '\babel@tempa'. Reported}}%
5553 {\expandafter\expandafter\expandafter\babel@luapatterns
5554 \csname babel@hyphendata@the\language\endcsname}}}%
5555 \endinput\fi

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

5556 \ifx\DisableBabelHook\@undefined
5557 \AddBabelHook{luatex}{everylanguage}{%
5558 \def\process@language##1##2##3{%
5559 \def\process@line####1####2 ####3 ####4 {}}}
5560 \AddBabelHook{luatex}{loadpatterns}{%
5561 \input #1\relax
5562 \expandafter\gdef\csname babel@hyphendata@the\language\endcsname
5563 {##1}{}}
5564 \AddBabelHook{luatex}{loadexceptions}{%
5565 \input #1\relax

```

```

5566 \def\bbl@tempb##1##2{{##1}{##2}}%
5567 \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5568 {\expandafter\expandafter\expandafter\bbl@tempb
5569 \csname bbl@hyphendata@the\language\endcsname}}
5570 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5571 \beginingroup
5572 \catcode`\%=12
5573 \catcode`\'=12
5574 \catcode`\%=12
5575 \catcode`\:=12
5576 \directlua{
5577   Babel.locale_props = Babel.locale_props or {}
5578
5579   function Babel.lua_error(e, a)
5580     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5581       e .. '}' .. (a or '') .. '}'})
5582   end
5583
5584   function Babel.bytes(line)
5585     return line:gsub(".",
5586       function (chr) return unicode.utf8.char(string.byte(chr)) end)
5587   end
5588
5589   function Babel.priority_in_callback(name,description)
5590     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5591       if v == description then return i end
5592     end
5593     return false
5594   end
5595
5596   function Babel.begin_process_input()
5597     if luatexbase and luatexbase.add_to_callback then
5598       luatexbase.add_to_callback('process_input_buffer',
5599         Babel.bytes, 'Babel.bytes')
5600     else
5601       Babel.callback = callback.find('process_input_buffer')
5602       callback.register('process_input_buffer', Babel.bytes)
5603     end
5604   end
5605
5606   function Babel.end_process_input ()
5607     if luatexbase and luatexbase.remove_from_callback then
5608       luatexbase.remove_from_callback('process_input_buffer', 'Babel.bytes')
5609     else
5610       callback.register('process_input_buffer', Babel.callback)
5611     end
5612   end
5613
5614   function Babel.str_to_nodes(fn, matches, base)
5615     local n, head, last
5616     if fn == nil then return nil end
5617     for s in string.utfvalues(fn(matches)) do
5618       if base.id == 7 then
5619         base = base.replace
5620       end
5621       n = node.copy(base)
5622       n.char = s
5623       if not head then
5624         head = n
5625       else
5626         last.next = n

```



```

5626     end
5627     last = n
5628 end
5629 return head
5630 end
5631
5632 Babel.linebreaking = Babel.linebreaking or {}
5633 Babel.linebreaking.before = {}
5634 Babel.linebreaking.after = {}
5635 Babel.locale = {}
5636 function Babel.linebreaking.add_before(func, pos)
5637     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5638     if pos == nil then
5639         table.insert(Babel.linebreaking.before, func)
5640     else
5641         table.insert(Babel.linebreaking.before, pos, func)
5642     end
5643 end
5644 function Babel.linebreaking.add_after(func)
5645     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5646     table.insert(Babel.linebreaking.after, func)
5647 end
5648
5649 function Babel.addpatterns(pp, lg)
5650     local lg = lang.new(lg)
5651     local pats = lang.patterns(lg) or ''
5652     lang.clear_patterns(lg)
5653     for p in pp:gmatch('^%s+') do
5654         ss = ''
5655         for i in string.utfcharacters(p:gsub('%d', '')) do
5656             ss = ss .. '%d?' .. i
5657         end
5658         ss = ss:gsub('^%d%?%.', '%%.') .. '%d?'
5659         ss = ss:gsub('%.%d%?$', '%%.')
5660         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5661         if n == 0 then
5662             tex.sprint(
5663                 [[\string\csname\space bbl@info\endcsname{New pattern: }]]
5664                 .. p .. [[]])
5665             pats = pats .. ' ' .. p
5666         else
5667             tex.sprint(
5668                 [[\string\csname\space bbl@info\endcsname{Renew pattern: }]]
5669                 .. p .. [[]])
5670         end
5671     end
5672     lang.patterns(lg, pats)
5673 end
5674
5675 Babel.characters = Babel.characters or {}
5676 Babel.ranges = Babel.ranges or {}
5677 function Babel.hlist_has_bidi(head)
5678     local has_bidi = false
5679     local ranges = Babel.ranges
5680     for item in node.traverse(head) do
5681         if item.id == node.id'glyph' then
5682             local itemchar = item.char
5683             local chardata = Babel.characters[itemchar]
5684             local dir = chardata and chardata.d or nil
5685             if not dir then
5686                 for nn, et in ipairs(ranges) do
5687                     if itemchar < et[1] then
5688                         break

```

```

5689         elseif itemchar <= et[2] then
5690             dir = et[3]
5691             break
5692         end
5693     end
5694 end
5695     if dir and (dir == 'al' or dir == 'r') then
5696         has_bidi = true
5697     end
5698 end
5699 end
5700 return has_bidi
5701 end
5702 function Babel.set_chranges_b (script, chrng)
5703     if chrng == '' then return end
5704     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')
5705     Babel.script_blocks[script] = {}
5706     for s, e in string.gmatch(chrng..' ', '(.)%.%.(.)%s') do
5707         table.insert(
5708             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5709     end
5710 end
5711
5712 function Babel.discard_sublr(str)
5713     if str:find( [[\string\indexentry]] ) and
5714         str:find( [[\string\babelsublr]] ) then
5715         str = str:gsub( [[\string\babelsublr%s*(%b{})]] ,
5716             function(m) return m:sub(2,-2) end )
5717     end
5718     return str
5719 end
5720 }
5721 \endgroup
5722 \ifx\newattribute\@undefined\else % Test for plain
5723 \newattribute\bbl@attr@locale % DL4
5724 \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5725 \AddBabelHook{luatex}{beforeextras}{%
5726 \setattribute\bbl@attr@locale\localeid}
5727 \fi
5728 %
5729 \def\BabelStringsDefault{unicode}
5730 \let\luabbl@stop\relax
5731 \AddBabelHook{luatex}{encodedcommands}{%
5732 \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5733 \ifx\bbl@tempa\bbl@tempb\else
5734 \directlua{Babel.begin_process_input()}%
5735 \def\luabbl@stop{%
5736 \directlua{Babel.end_process_input()}}%
5737 \fi}%
5738 \AddBabelHook{luatex}{stopcommands}{%
5739 \luabbl@stop
5740 \let\luabbl@stop\relax}
5741 %
5742 \AddBabelHook{luatex}{patterns}{%
5743 \@ifundefined{bbl@hyphendata@the\language}%
5744 {\def\bbl@elt##1##2##3##4{%
5745 \ifnum##2=\csname l@##2\endcsname % #2=spanish, dutch:OT1...
5746 \def\bbl@tempb{##3}%
5747 \ifx\bbl@tempb\empty\else % if not a synonymous
5748 \def\bbl@tempc{##3}{##4}}%
5749 \fi
5750 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5751 \fi}%

```

```

5752 \bbl@languages
5753 \ifundefined{bbl@hyphendata@the\language}%
5754 {\bbl@info{No hyphenation patterns were set for\%
5755 language '#2'. Reported}}%
5756 {\expandafter\expandafter\expandafter\bbl@luapatterns
5757 \csname bbl@hyphendata@the\language\endcsname}}}%
5758 \ifundefined{bbl@patterns@}\{%
5759 \begingroup
5760 \bbl@xin@{,\number\language,}{,\bbl@pttnlist}%
5761 \ifin@else
5762 \ifx\bbl@patterns@\empty\else
5763 \directlua{ Babel.addpatterns(
5764 [[\bbl@patterns@]], \number\language) }%
5765 \fi
5766 \ifundefined{bbl@patterns@#1}%
5767 \empty
5768 {\directlua{ Babel.addpatterns(
5769 [[\space\csname bbl@patterns@#1\endcsname]],
5770 \number\language) }}%
5771 \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5772 \fi
5773 \endgroup}%
5774 \bbl@exp{%
5775 \bbl@ifunset{bbl@prehc@languagename}}}%
5776 {\bbl@ifblank{\bbl@cs{prehc@languagename}}}%
5777 {\prehyphenchar=\bbl@cl{prehc}\relax}}}%

```

\babelpatterns This macro adds patterns. Two macros are used to store them: \bbl@patterns@ for the global ones and \bbl@patterns@<language> for language ones. We make sure there is a space between words when multiple commands are used.

```

5778 \onlypreamble\babelpatterns
5779 \AtEndOfPackage{%
5780 \newcommand\babelpatterns[2][\empty]{%
5781 \ifx\bbl@patterns@\relax
5782 \let\bbl@patterns@\empty
5783 \fi
5784 \ifx\bbl@pttnlist@\empty\else
5785 \bbl@warning{%
5786 You must not intermingle \string\selectlanguage\space and\%
5787 \string\babelpatterns\space or some patterns will not\%
5788 be taken into account. Reported}%
5789 \fi
5790 \ifx\empty#1%
5791 \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5792 \else
5793 \edef\bbl@tempb{\zap@space#1 \empty}%
5794 \bbl@for\bbl@tempa\bbl@tempb{%
5795 \bbl@fixname\bbl@tempa
5796 \bbl@iflanguage\bbl@tempa{%
5797 \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5798 \@ifundefined{bbl@patterns@\bbl@tempa}%
5799 \empty
5800 {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5801 #2}}}%
5802 \fi}}

```

10.6. Southeast Asian scripts

First, some general code for line breaking, used by \babelposthyphenation.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5803 \def\bbl@intraspace#1 #2 #3\@@{%
5804   \directlua{
5805     Babel.intraspaces = Babel.intraspaces or {}
5806     Babel.intraspaces['\csname bbl@sbcpr@languagename\endcsname'] = %
5807       {b = #1, p = #2, m = #3}
5808     Babel.locale_props[\the\localeid].intraspace = %
5809       {b = #1, p = #2, m = #3}
5810   }}
5811 \def\bbl@intrapenalty#1\@@{%
5812   \directlua{
5813     Babel.intrapenalties = Babel.intrapenalties or {}
5814     Babel.intrapenalties['\csname bbl@sbcpr@languagename\endcsname'] = #1
5815     Babel.locale_props[\the\localeid].intrapenalty = #1
5816   }}
5817 \begingroup
5818 \catcode\%=12
5819 \catcode\&=14
5820 \catcode\'=12
5821 \catcode\-=12
5822 \gdef\bbl@seaintraspace{%
5823   \let\bbl@seaintraspace\relax
5824   \directlua{
5825     Babel.sea_enabled = true
5826     Babel.sea_ranges = Babel.sea_ranges or {}
5827     function Babel.set_chranges (script, chrng)
5828       local c = 0
5829       for s, e in string.gmatch(chrng..' ', '(.-%.%.(-)%s') do
5830         Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5831         c = c + 1
5832       end
5833     end
5834     function Babel.sea_disc_to_space (head)
5835       local sea_ranges = Babel.sea_ranges
5836       local last_char = nil
5837       local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5838       for item in node.traverse(head) do
5839         local i = item.id
5840         if i == node.id'glyph' then
5841           last_char = item
5842         elseif i == 7 and item.subtype == 3 and last_char
5843           and last_char.char > 0x0C99 then
5844           quad = font.getfont(last_char.font).size
5845           for lg, rg in pairs(sea_ranges) do
5846             if last_char.char > rg[1] and last_char.char < rg[2] then
5847               lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril1
5848               local intraspace = Babel.intraspaces[lg]
5849               local intrapenalty = Babel.intrapenalties[lg]
5850               local n
5851               if intrapenalty ~= 0 then
5852                 n = node.new(14, 0)      &% penalty
5853                 n.penalty = intrapenalty
5854                 node.insert_before(head, item, n)
5855               end
5856               n = node.new(12, 13)      &% (glue, spaceskip)
5857               node.setglue(n, intraspace.b * quad,
5858                 intraspace.p * quad,
5859                 intraspace.m * quad)
5860               node.insert_before(head, item, n)
5861               node.remove(head, item)
5862             end
5863           end
5864         end
5865       end

```

```

5866     end
5867   }&
5868   \bbl@luahyphenate}

```

10.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5869 \catcode`\%=14
5870 \gdef\bbl@cjkintraspacespace{%
5871   \let\bbl@cjkintraspacespace\relax
5872   \directlua{
5873     require('babel-data-cjk.lua')
5874     Babel.cjk_enabled = true
5875     function Babel.cjk_linebreak(head)
5876       local GLYPH = node.id'glyph'
5877       local last_char = nil
5878       local quad = 655360      % 10 pt = 655360 = 10 * 65536
5879       local last_class = nil
5880       local last_lang = nil
5881       for item in node.traverse(head) do
5882         if item.id == GLYPH then
5883           local lang = item.lang
5884           local LOCALE = node.get_attribute(item,
5885             Babel.attr_locale)
5886           local props = Babel.locale_props[LOCALE] or {}
5887           local class = Babel.cjk_class[item.char].c
5888           if props.cjk_quotes and props.cjk_quotes[item.char] then
5889             class = props.cjk_quotes[item.char]
5890           end
5891           if class == 'cp' then class = 'cl' % ) as CL
5892           elseif class == 'id' then class = 'I'
5893           elseif class == 'cj' then class = 'I' % loose
5894           end
5895           local br = 0
5896           if class and last_class and Babel.cjk_breaks[last_class][class] then
5897             br = Babel.cjk_breaks[last_class][class]
5898           end
5899           if br == 1 and props.linebreak == 'c' and
5900             lang ~= \the\l@nohyphenation\space and
5901             last_lang ~= \the\l@nohyphenation then
5902             local intrapenalty = props.intrapenalty
5903             if intrapenalty ~= 0 then
5904               local n = node.new(14, 0)      % penalty
5905               n.penalty = intrapenalty
5906               node.insert_before(head, item, n)
5907             end
5908             local intraspacespace = props.intraspacespace
5909             local n = node.new(12, 13)      % (glue, spaceskip)
5910             node.setglue(n, intraspacespace.b * quad,
5911               intraspacespace.p * quad,
5912               intraspacespace.m * quad)
5913             node.insert_before(head, item, n)
5914           end
5915           if font.getfont(item.font) then
5916             quad = font.getfont(item.font).size
5917           end
5918           last_class = class
5919           last_lang = lang

```

```

5920         else % if penalty, glue or anything else
5921             last_class = nil
5922         end
5923     end
5924     lang.hyphenate(head)
5925 end
5926 }%
5927 \bbl@luahyphenate}
5928 \gdef\bbl@luahyphenate{%
5929 \let\bbl@luahyphenate\relax
5930 \directlua{
5931     luatexbase.add_to_callback('hyphenate',
5932     function (head, tail)
5933         if Babel.linebreaking.before then
5934             for k, func in ipairs(Babel.linebreaking.before) do
5935                 func(head)
5936             end
5937         end
5938         lang.hyphenate(head)
5939         if Babel.cjk_enabled then
5940             Babel.cjk_linebreak(head)
5941         end
5942         if Babel.linebreaking.after then
5943             for k, func in ipairs(Babel.linebreaking.after) do
5944                 func(head)
5945             end
5946         end
5947         if Babel.set_hboxed then
5948             Babel.set_hboxed(head)
5949         end
5950         if Babel.sea_enabled then
5951             Babel.sea_disc_to_space(head)
5952         end
5953     end,
5954     'Babel.hyphenate')
5955 }%
5956 \endgroup
5957 %
5958 \def\bbl@provide@intraspace{%
5959     \bbl@ifunset\bbl@intsp@{language}\language\language}%
5960     {\expandafter\ifx\csname bbl@intsp@{language}\endcsname\empty\else
5961         \bbl@xin@{c}{\bbl@cl{lnbrk}}}%
5962     \ifin@ % cjk
5963         \bbl@cjk_intraspace
5964         \directlua{
5965             Babel.locale_props = Babel.locale_props or {}
5966             Babel.locale_props[\the\localeid].linebreak = 'c'
5967         }%
5968         \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\bbl@cl{}%
5969         \ifx\bbl@KVP@intrapenalty\@nnil
5970             \bbl@intrapenalty0\@
5971         \fi
5972     \else % sea
5973         \bbl@sea_intraspace
5974         \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\bbl@cl{}%
5975         \directlua{
5976             Babel.sea_ranges = Babel.sea_ranges or {}
5977             Babel.set_chranges('\bbl@cl{sbcpr}',
5978                 '\bbl@cl{chrng}')
5979         }%
5980         \ifx\bbl@KVP@intrapenalty\@nnil
5981             \bbl@intrapenalty0\@
5982         \fi

```

```

5983     \fi
5984     \fi
5985     \ifx\bbl@KVP@intrapenalty\@nnil\else
5986     \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5987     \fi}}

```

10.8. Arabic justification

WIP. \bbl@arabicjust is executed with both elongated an kashida. This must be fine tuned. The attribute kashida is set by transforms with kashida.

```

5988 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
5989 \def\bblar@chars{%
5990   0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
5991   0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
5992   0640,0641,0642,0643,0644,0645,0646,0647,0649}
5993 \def\bblar@elongated{%
5994   0626,0628,062A,062B,0633,0634,0635,0636,063B,%
5995   063C,063D,063E,063F,0641,0642,0643,0644,0646,%
5996   0649,064A}
5997 \begingroup
5998   \catcode\_ =11 \catcode\`:=11
5999   \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6000 \endgroup
6001 \gdef\bbl@arabicjust{%
6002   \let\bbl@arabicjust\relax
6003   \newattribute\bblar@kashida
6004   \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6005   \bblar@kashida=\z@
6006   \bbl@patchfont{\bbl@parsejalt}}%
6007   \directlua{
6008     Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6009     Babel.arabic.elong_map[\the\localeid] = {}
6010     luatexbase.add_to_callback('post_linebreak_filter',
6011       Babel.arabic.justify, 'Babel.arabic.justify')
6012     luatexbase.add_to_callback('hpack_filter',
6013       Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6014   }}%

```

Save both node lists to make replacement.

```

6015 \def\bblar@fetchjalt#1#2#3#4{%
6016   \bbl@exp{\bbl@foreach{#1}}{%
6017     \bbl@ifunset\bblar@JE@##1}%
6018     {\setbox\z@\hbox{\textdir TRT ^^^^200d\char"##1#2}}%
6019     {\setbox\z@\hbox{\textdir TRT ^^^^200d\char"\@nameuse\bblar@JE@##1#2}}%
6020   \directlua{%
6021     local last = nil
6022     for item in node.traverse(tex.box[0].head) do
6023       if item.id == node.id'glyph' and item.char > 0x600 and
6024         not (item.char == 0x200D) then
6025         last = item
6026       end
6027     end
6028     Babel.arabic.#3['##1#4'] = last.char
6029   }}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswb?). What about kaf? And diacritic positioning?

```

6030 \gdef\bbl@parsejalt{%
6031   \ifx\addfontfeature\undefined\else
6032     \bbl@xin@{/e}{/bbl@cl{\lnbrk}}%
6033     \ifin@
6034     \directlua{%
6035       if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then

```

```

6036         Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6037         tex.print([[string\curname\space bbl@parsejalti\endcsname]])
6038     end
6039 }%
6040 \fi
6041 \fi}
6042 \gdef\bbl@parsejalti{%
6043 \begingroup
6044 \let\bbl@parsejalt\relax % To avoid infinite loop
6045 \edef\bbl@tempb{\fontid\font}%
6046 \bblar@nofswarn
6047 \@nameuse{tracinglostchars}\z@
6048 \bblar@fetchjalt\bblar@elongated{}\{from\}\}%
6049 \bblar@fetchjalt\bblar@chars{^^^064a}\{from\}{a}% Alef maksura
6050 \bblar@fetchjalt\bblar@chars{^^^0649}\{from\}{y}% Yeh
6051 \addfontfeature{RawFeature+=jalt}%
6052 % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf
6053 \bblar@fetchjalt\bblar@elongated{}\{dest\}\}%
6054 \bblar@fetchjalt\bblar@chars{^^^064a}\{dest\}{a}%
6055 \bblar@fetchjalt\bblar@chars{^^^0649}\{dest\}{y}%
6056 \directlua{%
6057     for k, v in pairs(Babel.arabic.from) do
6058         if Babel.arabic.dest[k] and
6059             not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6060             Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6061             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6062         end
6063     end
6064 }%
6065 \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6066 \begingroup
6067 \catcode`\# = 11
6068 \catcode`\~ = 11
6069 \directlua{
6070
6071 Babel.arabic = Babel.arabic or {}
6072 Babel.arabic.from = {}
6073 Babel.arabic.dest = {}
6074 Babel.arabic.justify_factor = 0.95
6075 Babel.arabic.justify_enabled = true
6076 Babel.arabic.kashida_limit = -1
6077
6078 function Babel.arabic.justify(head)
6079     if not Babel.arabic.justify_enabled then return head end
6080     for line in node.traverse_id(node.id'hlist', head) do
6081         Babel.arabic.justify_hlist(head, line)
6082     end
6083     % In case the very first item is a line (eg, in \vbox):
6084     while head.prev do head = head.prev end
6085     return head
6086 end
6087
6088 function Babel.arabic.justify_hbox(head, gc, size, pack)
6089     local has_inf = false
6090     if Babel.arabic.justify_enabled and pack == 'exactly' then
6091         for n in node.traverse_id(12, head) do
6092             if n.stretch_order > 0 then has_inf = true end
6093         end
6094         if not has_inf then
6095             Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6096         end

```



```

6097 end
6098 return head
6099 end
6100
6101 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6102     local d, new
6103     local k_list, k_item, pos_inline
6104     local width, width_new, full, k_curr, wt_pos, goal, shift
6105     local subst_done = false
6106     local elong_map = Babel.arabic.elong_map
6107     local cnt
6108     local last_line
6109     local GLYPH = node.id'glyph'
6110     local KASHIDA = Babel.attr_kashida
6111     local LOCALE = Babel.attr_locale
6112
6113     if line == nil then
6114         line = {}
6115         line.glue_sign = 1
6116         line.glue_order = 0
6117         line.head = head
6118         line.shift = 0
6119         line.width = size
6120     end
6121
6122     % Exclude last line.
6123     if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6124         elongs = {} % Stores elongated candidates of each line
6125         k_list = {} % And all letters with kashida
6126         pos_inline = 0 % Not yet used
6127
6128         for n in node.traverse_id(GLYPH, line.head) do
6129             pos_inline = pos_inline + 1 % To find where it is. Not used.
6130
6131             % Elongated glyphs
6132             if elong_map then
6133                 local locale = node.get_attribute(n, LOCALE)
6134                 if elong_map[locale] and elong_map[locale][n.font] and
6135                     elong_map[locale][n.font][n.char] then
6136                     table.insert(elongs, {node = n, locale = locale} )
6137                     node.set_attribute(n.prev, KASHIDA, 0)
6138                 end
6139             end
6140
6141             % Tatwil. First create a list of nodes marked with kashida. The
6142             % rest of nodes can be ignored. The list of used weights is build
6143             % when transforms with the key kashida= are declared.
6144             if Babel.kashida_wts then
6145                 local k_wt = node.get_attribute(n, KASHIDA)
6146                 if k_wt > 0 then % todo. parameter for multi inserts
6147                     table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6148                 end
6149             end
6150
6151             end % of node.traverse_id
6152
6153             if #elongs == 0 and #k_list == 0 then goto next_line end
6154             full = line.width
6155             shift = line.shift
6156             goal = full * Babel.arabic.justify_factor % A bit crude
6157             width = node.dimensions(line.head) % The 'natural' width
6158
6159             % == Elongated ==

```

```

6160 % Original idea taken from 'chickenize'
6161 while (#elongs > 0 and width < goal) do
6162     subst_done = true
6163     local x = #elongs
6164     local curr = elongs[x].node
6165     local oldchar = curr.char
6166     curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6167     width = node.dimensions(line.head) % Check if the line is too wide
6168     % Substitute back if the line would be too wide and break:
6169     if width > goal then
6170         curr.char = oldchar
6171         break
6172     end
6173     % If continue, pop the just substituted node from the list:
6174     table.remove(elongs, x)
6175 end
6176
6177 % == Tatwil ==
6178 % Traverse the kashida node list so many times as required, until
6179 % the line is filled. The first pass adds a tatweel after each
6180 % node with kashida in the line, the second pass adds another one,
6181 % and so on. In each pass, add first the kashida with the highest
6182 % weight, then with lower weight and so on.
6183 if #k_list == 0 then goto next_line end
6184
6185 width = node.dimensions(line.head) % The 'natural' width
6186 k_curr = #k_list % Traverse backwards, from the end
6187 wt_pos = 1
6188
6189 while width < goal do
6190     subst_done = true
6191     k_item = k_list[k_curr].node
6192     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6193         d = node.copy(k_item)
6194         d.char = 0x0640
6195         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6196         d.xoffset = 0
6197         line.head, new = node.insert_after(line.head, k_item, d)
6198         width_new = node.dimensions(line.head)
6199         if width > goal or width == width_new then
6200             node.remove(line.head, new) % Better compute before
6201             break
6202         end
6203         if Babel.fix_diacr then
6204             Babel.fix_diacr(k_item.next)
6205         end
6206         width = width_new
6207     end
6208     if k_curr == 1 then
6209         k_curr = #k_list
6210         wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6211     else
6212         k_curr = k_curr - 1
6213     end
6214 end
6215
6216 % Limit the number of tatweel by removing them. Not very efficient,
6217 % but it does the job in a quite predictable way.
6218 if Babel.arabic.kashida_limit > -1 then
6219     cnt = 0
6220     for n in node.traverse_id(GLYPH, line.head) do
6221         if n.char == 0x0640 then
6222             cnt = cnt + 1

```

```

6223         if cnt > Babel.arabic.kashida_limit then
6224             node.remove(line.head, n)
6225         end
6226     else
6227         cnt = 0
6228     end
6229 end
6230 end
6231
6232 ::next_line::
6233
6234 % Must take into account marks and ins, see luatex manual.
6235 % Have to be executed only if there are changes. Investigate
6236 % what's going on exactly.
6237 if subst_done and not gc then
6238     d = node.hpack(line.head, full, 'exactly')
6239     d.shift = shift
6240     node.insert_before(head, line, d)
6241     node.remove(head, line)
6242 end
6243 end % if process line
6244 end
6245 }
6246 \endgroup
6247 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6248 \def\bbl@scr@node@list{%
6249   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6250   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6251 \ifnum\bbl@bidimode=102 % bidi-r
6252   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6253 \fi
6254 \def\bbl@set@renderer{%
6255   \bbl@xin{\bbl@cl{sname}}{\bbl@scr@node@list}%
6256   \ifin@
6257     \let\bbl@unset@renderer\relax
6258   \else
6259     \bbl@exp{%
6260       \def\\bbl@unset@renderer{%
6261         \def<g__fontspec_default_fontopts_clist>{%
6262           \[g__fontspec_default_fontopts_clist]}}%
6263       \def<g__fontspec_default_fontopts_clist>{%
6264         Renderer=Harfbuzz,\[g__fontspec_default_fontopts_clist]}}%
6265     \fi}
6266 <@Font selection@>

```

10.10. Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the letters option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6267 \directlua{% DL6
6268 Babel.script_blocks = {
6269   ['dflt'] = {},
6270   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6271               {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6272   ['Armn'] = {{0x0530, 0x058F}},
6273   ['Beng'] = {{0x0980, 0x09FF}},
6274   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0ABBF}},
6275   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6276   ['Cyril'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6277                {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6278   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6279   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6280                {0xAB00, 0xAB2F}},
6281   ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6282   % Don't follow strictly Unicode, which places some Coptic letters in
6283   % the 'Greek and Coptic' block
6284   ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6285   ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6286                {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6287                {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6288                {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6289                {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6290                {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6291   ['Hebr'] = {{0x0590, 0x05FF},
6292                {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6293   ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6294                {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6295   ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6296   ['Knda'] = {{0x0C80, 0x0CFF}},
6297   ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6298                {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6299                {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6300   ['Lao'] = {{0x0E80, 0x0EFF}},
6301   ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6302                {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6303                {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6304   ['Mahj'] = {{0x11150, 0x1117F}},
6305   ['Mlym'] = {{0x0D00, 0x0D7F}},
6306   ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6307   ['Orya'] = {{0x0B00, 0x0B7F}},
6308   ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6309   ['Syr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6310   ['Taml'] = {{0x0B80, 0x0BFF}},
6311   ['Telu'] = {{0x0C00, 0x0C7F}},
6312   ['Tfng'] = {{0x2D30, 0x2D7F}},
6313   ['Thai'] = {{0x0E00, 0x0E7F}},
6314   ['Tibt'] = {{0x0F00, 0x0FFF}},
6315   ['Vaii'] = {{0xA500, 0xA63F}},
6316   ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6317 }
6318
6319 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyril
6320 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6321 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6322
6323 function Babel.locale_map(head)
6324   if not Babel.locale_mapped then return head end
6325
6326   local LOCALE = Babel.attr_locale

```

```

6327 local GLYPH = node.id('glyph')
6328 local inmath = false
6329 local toloc_save
6330 for item in node.traverse(head) do
6331   local toloc
6332   if not inmath and item.id == GLYPH then
6333     % Optimization: build a table with the chars found
6334     if Babel.chr_to_loc[item.char] then
6335       toloc = Babel.chr_to_loc[item.char]
6336     else
6337       for lc, maps in pairs(Babel.loc_to_scr) do
6338         for _, rg in pairs(maps) do
6339           if item.char >= rg[1] and item.char <= rg[2] then
6340             Babel.chr_to_loc[item.char] = lc
6341             toloc = lc
6342             break
6343           end
6344         end
6345       end
6346       % Treat composite chars in a different fashion, because they
6347       % 'inherit' the previous locale.
6348       if (item.char >= 0x0300 and item.char <= 0x036F) or
6349         (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6350         (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6351         Babel.chr_to_loc[item.char] = -2000
6352         toloc = -2000
6353       end
6354       if not toloc then
6355         Babel.chr_to_loc[item.char] = -1000
6356       end
6357     end
6358     if toloc == -2000 then
6359       toloc = toloc_save
6360     elseif toloc == -1000 then
6361       toloc = nil
6362     end
6363     if toloc and Babel.locale_props[toloc] and
6364       Babel.locale_props[toloc].letters and
6365       tex.getcatcode(item.char) \string~= 11 then
6366       toloc = nil
6367     end
6368     if toloc and Babel.locale_props[toloc].script
6369       and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6370       and Babel.locale_props[toloc].script ==
6371       Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6372       toloc = nil
6373     end
6374     if toloc then
6375       if Babel.locale_props[toloc].lg then
6376         item.lang = Babel.locale_props[toloc].lg
6377         node.set_attribute(item, LOCALE, toloc)
6378       end
6379       if Babel.locale_props[toloc]['/'..item.font] then
6380         item.font = Babel.locale_props[toloc]['/'..item.font]
6381       end
6382     end
6383     toloc_save = toloc
6384   elseif not inmath and item.id == 7 then % Apply recursively
6385     item.replace = item.replace and Babel.locale_map(item.replace)
6386     item.pre = item.pre and Babel.locale_map(item.pre)
6387     item.post = item.post and Babel.locale_map(item.post)
6388   elseif item.id == node.id'math' then
6389     inmath = (item.subtype == 0)

```

```

6390 end
6391 end
6392 return head
6393 end
6394 }

```

The code for `\babelcharproperty` is straightforward. Just note the modified lua table can be different.

```

6395 \newcommand\babelcharproperty[1]{%
6396   \count@=#1\relax
6397   \ifvmode
6398     \expandafter\babel@chprop
6399   \else
6400     \babel@error{charproperty-only-vertical}{}{}{}%
6401   \fi}
6402 \newcommand\babel@chprop[3][\the\count@]{%
6403   \@tempcnta=#1\relax
6404   \babel@ifunset{babel@chprop@#2}% {unknown-char-property}
6405     {\babel@error{unknown-char-property}{}{#2}{}%
6406     }%
6407   \loop
6408     \babel@cs{chprop@#2}{#3}%
6409     \ifnum\count@<\@tempcnta
6410       \advance\count@\@ne
6411     \repeat}
6412 %
6413 \def\babel@chprop@direction#1{%
6414   \directlua{
6415     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6416     Babel.characters[\the\count@]['d'] = '#1'
6417   }}
6418 \let\babel@chprop@bc\babel@chprop@direction
6419 %
6420 \def\babel@chprop@mirror#1{%
6421   \directlua{
6422     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6423     Babel.characters[\the\count@]['m'] = '\number#1'
6424   }}
6425 \let\babel@chprop@bmg\babel@chprop@mirror
6426 %
6427 \def\babel@chprop@linebreak#1{%
6428   \directlua{
6429     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6430     Babel.cjk_characters[\the\count@]['c'] = '#1'
6431   }}
6432 \let\babel@chprop@lb\babel@chprop@linebreak
6433 %
6434 \def\babel@chprop@locale#1{%
6435   \directlua{
6436     Babel.chr_to_loc = Babel.chr_to_loc or {}
6437     Babel.chr_to_loc[\the\count@] =
6438       \babel@ifblank{#1}{-1000}{\the\babel@cs{id@#1}}\space
6439   }}

```

Post-handling hyphenation patterns for non-standard rules, like `ff` to `ff-f`. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6440 \directlua{% DL7
6441   Babel.nohyphenation = \the\l@nohyphenation
6442 }

```

Now the $\TeX$ high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the `{n}` syntax. For example, `pre={1}{1}` becomes `function(m) return m[1]..m[1]..'-' end`, where `m` are the matches returned after applying the pattern. With a mapped capture the functions are similar to

function(m) return Babel.capt_map(m[1],1) end, where the last argument identifies the mapping to be applied to m[1]. The way it is carried out is somewhat tricky, but the effect is not dissimilar to lua load – save the code as string in a TeX macro, and expand this macro at the appropriate place. As \directlua does not take into account the current catcode of @, we just avoid this character in macro names (which explains the internal group, too).

```

6443 \begingroup
6444 \catcode\~ =12
6445 \catcode\% =12
6446 \catcode\& =14
6447 \catcode\| =12
6448 \gdef\babelprehyphenation{%&%
6449 \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}[]}]
6450 \gdef\babelposthyphenation{%&%
6451 \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}[]}]
6452 %
6453 \gdef\bbl@settransform#1[#2]#3#4#5{%&%
6454 \ifcase#1
6455 \bbl@activateprehyphen
6456 \or
6457 \bbl@activateposthyphen
6458 \fi
6459 \begingroup
6460 \def\babeltempa{\bbl@add@list\babeltempb}%&%
6461 \let\babeltempb\empty
6462 \def\bbl@tempa{#5}%&%
6463 \bbl@replace\bbl@tempa{,}{ ,}%&% TODO. Ugly trick to preserve {}
6464 \expandafter\bbl@foreach\expandafter{\bbl@tempa}{%&%
6465 \bbl@ifsamestring{##1}{remove}%&%
6466 {\bbl@add@list\babeltempb{nil}}}%&%
6467 {\directlua{
6468 local rep = {[##1]=}
6469 local three_args = '%s*=%s*([%-d%.%a{ }|]+)%s+([%-d%.%a{ }|]+)%s+([%-d%.%a{ }|]+)'
6470 &% Numeric passes directly: kern, penalty...
6471 rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6472 rep = rep:gsub('^%s*(insert)%s*', 'insert = true, ')
6473 rep = rep:gsub('^%s*(after)%s*', 'after = true, ')
6474 rep = rep:gsub('(string)%s*=%s*([^\s,]*)', Babel.capture_func)
6475 rep = rep:gsub('node%s*=%s*(%a+)%s*([^\s,]*)', Babel.capture_node)
6476 rep = rep:gsub(' (norule)' .. three_args,
6477 'norule = { ' .. '%2, %3, %4' .. ' }')
6478 if #1 == 0 or #1 == 2 then
6479 rep = rep:gsub(' (space)' .. three_args,
6480 'space = { ' .. '%2, %3, %4' .. ' }')
6481 rep = rep:gsub(' (spacefactor)' .. three_args,
6482 'spacefactor = { ' .. '%2, %3, %4' .. ' }')
6483 rep = rep:gsub(' (kashida)%s*=%s*([^\s,]*)', Babel.capture_kashida)
6484 &% Transform values
6485 rep, n = rep:gsub(' {[([%a-%.]|)([([%a-%.]|)+)}',
6486 function(v,d)
6487 return string.format (
6488 '\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6489 v,
6490 load( 'return Babel.locale_props'..
6491 '\the\csname bbl@id@@#3\endcsname].' .. d)() )
6492 end )
6493 rep, n = rep:gsub(' {[([%a-%.]|)([([%-d%.] )+)}',
6494 '\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6495 end
6496 if #1 == 1 then
6497 rep = rep:gsub(' (no)%s*=%s*([^\s,]*)', Babel.capture_func)
6498 rep = rep:gsub(' (pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6499 rep = rep:gsub(' (post)%s*=%s*([^\s,]*)', Babel.capture_func)
6500 end

```

```

6501         tex.print([[\\string\\babeltempa{[] .. rep .. [[]]])
6502     }&%
6503     \\bbl@foreach\\babeltempb{&%
6504         \\bbl@forkv{##1}{&%
6505             \\in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6506                 post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6507             \\ifin@\\else
6508                 \\bbl@error{bad-transform-option}{###1}{}&%
6509             \\fi}&%
6510     \\let\\bbl@kv@attribute\\relax
6511     \\let\\bbl@kv@label\\relax
6512     \\let\\bbl@kv@fonts\\empty
6513     \\let\\bbl@kv@prepend\\relax
6514     \\bbl@forkv{#2}{\\bbl@csarg\\edef{kv##1}{##2}}&%
6515     \\ifx\\bbl@kv@fonts\\empty\\else\\bbl@settransfont\\fi
6516     \\ifx\\bbl@kv@attribute\\relax
6517         \\ifx\\bbl@kv@label\\relax\\else
6518             \\bbl@exp{\\bbl@trim@def\\bbl@kv@fonts{\\bbl@kv@fonts}}&%
6519             \\bbl@replace\\bbl@kv@fonts{ }{,}&%
6520             \\edef\\bbl@kv@attribute{\\bbl@ATR@\\bbl@kv@label @#3@\\bbl@kv@fonts}&%
6521             \\count@\\z@
6522             \\def\\bbl@elt##1##2##3{&%
6523                 \\bbl@ifsamestring{#3,\\bbl@kv@label}{##1,##2}&%
6524                 {\\bbl@ifsamestring{\\bbl@kv@fonts}{##3}&%
6525                     {\\count@\\@ne}&%
6526                     {\\bbl@error{font-conflict-transforms}{}}}&%
6527                 }&%
6528             \\bbl@transfont@list
6529             \\ifnum\\count@=\\z@
6530                 \\bbl@exp{\\global\\bbl@add\\bbl@transfont@list
6531                     {\\bbl@elt{#3}{\\bbl@kv@label}{\\bbl@kv@fonts}}}&%
6532             \\fi
6533             \\bbl@ifunset{\\bbl@kv@attribute}&%
6534             {\\global\\bbl@carg\\newattribute{\\bbl@kv@attribute}}&%
6535             {}&%
6536             \\global\\bbl@carg\\setattribute{\\bbl@kv@attribute}\\@ne
6537             \\fi
6538         \\else
6539             \\edef\\bbl@kv@attribute{\\expandafter\\bbl@stripslash\\bbl@kv@attribute}&%
6540             \\fi
6541     \\directlua{
6542         local lbr = Babel.linebreaking.replacements[#1]
6543         local u = unicode.utf8
6544         local id, attr, label
6545         if #1 == 0 then
6546             id = \\the\\csname bbl@id@@#3\\endcsname\\space
6547         else
6548             id = \\the\\csname l@#3\\endcsname\\space
6549         end
6550         \\ifx\\bbl@kv@attribute\\relax
6551             attr = -1
6552         \\else
6553             attr = luatexbase.registernumber'\\bbl@kv@attribute'
6554         \\fi
6555         \\ifx\\bbl@kv@label\\relax\\else &% Same refs:
6556             label = [==[\\bbl@kv@label]==]
6557         \\fi
6558         &% Convert pattern:
6559         local patt = string.gsub([==[#4]==], '%s', '')
6560         if #1 == 0 then
6561             patt = string.gsub(patt, '|', ' ')
6562         end
6563         if not u.find(patt, '()', nil, true) then

```



```

6564     patt = '()' .. patt .. '()'
6565     end
6566     patt = string.gsub(patt, '%(%)^', '^()')
6567     patt = string.gsub(patt, '%$%(%)', '()$')
6568     patt = u.gsub(patt, '{(.)}',
6569         function (n)
6570             return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6571         end)
6572     patt = u.gsub(patt, '{(%x%x%x%x+)}',
6573         function (n)
6574             return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6575         end)
6576     lbkr[id] = lbkr[id] or {}
6577     table.insert(lbkr[id], \ifx\bbk@kv@prepend\relax\else 1,\fi
6578         { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6579     }&%
6580 \endgroup}
6581 \endgroup
6582 %
6583 \let\bbk@transfont@list\@empty
6584 \def\bbk@settransfont{%
6585     \global\let\bbk@settransfont\relax % Execute only once
6586     \gdef\bbk@transfont{%
6587         \def\bbk@elt####1####2####3{%
6588             \bbk@ifblank{####3}%
6589             {\count@tw}% Do nothing if no fonts
6590             {\count@z@
6591             \bbk@vforeach{####3}{%
6592                 \def\bbk@tempd{#####1}%
6593                 \edef\bbk@tempe{\bbk@transfam/\f@series/\f@shape}%
6594                 \ifx\bbk@tempd\bbk@tempe
6595                     \count@ne
6596                 \else\ifx\bbk@tempd\bbk@transfam
6597                     \count@ne
6598                 \fi\fi}%
6599                 \ifcase\count@
6600                     \bbk@csarg\unsetattribute{ATR@####2@####1@####3}%
6601                 \or
6602                     \bbk@csarg\setattribute{ATR@####2@####1@####3}@ne
6603                 \fi}}%
6604             \bbk@transfont@list}%
6605     \AddToHook{selectfont}{\bbk@transfont}% Hooks are global.
6606     \gdef\bbk@transfam{-unknown-}%
6607     \bbk@foreach\bbk@font@fams{%
6608         \AddToHook{##1family}{\def\bbk@transfam{##1}}%
6609         \bbk@ifsamestring{\@nameuse{##1default}}\familydefault
6610         {\xdef\bbk@transfam{##1}}%
6611     }}}}
6612 %
6613 \DeclareRobustCommand\enablelocaletransform[1]{%
6614     \bbk@ifunset{\bbk@ATR@#1\language @}%
6615     {\bbk@error{transform-not-available}{#1}}}%
6616     {\bbk@csarg\setattribute{ATR@#1\language @}\@ne}}
6617 \DeclareRobustCommand\disablelocaletransform[1]{%
6618     \bbk@ifunset{\bbk@ATR@#1\language @}%
6619     {\bbk@error{transform-not-available-b}{#1}}}%
6620     {\bbk@csarg\unsetattribute{ATR@#1\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6621 \def\bbk@activateposthyphen{%
6622     \let\bbk@activateposthyphen\relax
6623     \ifx\bbk@attr@hboxed\undefined

```

```

6624 \newattribute\bbl@attr@hboxed
6625 \fi
6626 \directlua{
6627   require('babel-transforms.lua')
6628   Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6629 }
6630 \def\bbl@activateprehyphen{%
6631   \let\bbl@activateprehyphen\relax
6632   \ifx\bbl@attr@hboxed\undefined
6633     \newattribute\bbl@attr@hboxed
6634   \fi
6635   \directlua{
6636     require('babel-transforms.lua')
6637     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6638   }
6639 \newcommand\SetTransformValue[3]{%
6640   \directlua{
6641     Babel.locale_props[\the\csname bbl@id@#1\endcsname].vars["#2"] = #3
6642   }

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6643 \newcommand\ShowBabelTransforms[1]{%
6644   \bbl@activateprehyphen
6645   \bbl@activateposthyphen
6646   \begingroup
6647     \directlua{ Babel.show_transforms = true }%
6648     \setbox\z@\vbox{#1}%
6649     \directlua{ Babel.show_transforms = false }%
6650   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]==]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6651 \newcommand\localeprehyphenation[1]{%
6652   \directlua{ Babel.string_prehyphenation([==#1==], \the\localeid) }}

```

10.11.Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```

6653 \AtBeginDocument{\pagedir\bodydir}

```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by $\TeX$. Just in case, consider the possibility it has not been loaded.

```

6654 \def\bbl@activate@preotf{%
6655   \let\bbl@activate@preotf\relax % only once
6656   \directlua{
6657     function Babel.pre_otfload_v(head)
6658       if Babel.numbers and Babel.digits_mapped then
6659         head = Babel.numbers(head)
6660       end
6661       if Babel.bidi_enabled then
6662         head = Babel.bidi(head, false, dir)
6663       end
6664       return head
6665     end
6666     %
6667     function Babel.pre_otfload_h(head, gc, sz, pt, dir)

```

```

6668     if Babel.numbers and Babel.digits_mapped then
6669         head = Babel.numbers(head)
6670     end
6671     if Babel.bidi_enabled then
6672         head = Babel.bidi(head, false, dir)
6673     end
6674     return head
6675 end
6676 %
6677 luatexbase.add_to_callback('pre_linebreak_filter',
6678     Babel.pre_otfload_v,
6679     'Babel.pre_otfload_v',
6680     Babel.priority_in_callback('pre_linebreak_filter',
6681         'luaotfload.node_processor') or nil)
6682 %
6683 luatexbase.add_to_callback('hpack_filter',
6684     Babel.pre_otfload_h,
6685     'Babel.pre_otfload_h',
6686     Babel.priority_in_callback('hpack_filter',
6687         'luaotfload.node_processor') or nil)
6688 }}

```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in basic-r.

```

6689 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6690 \let\bbl@beforeforeign\leavevmode
6691 \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6692 \RequirePackage{luatexbase}
6693 \bbl@activate@preotf
6694 \directlua{
6695     require('babel-data-bidi.lua')
6696     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6697         require('babel-bidi-basic.lua')
6698     \or
6699         require('babel-bidi-basic-r.lua')
6700     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6701     table.insert(Babel.ranges, {0xF0000, 0xFFFFD, 'on'})
6702     table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6703 \fi}
6704 \newattribute\bbl@attr@dir
6705 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6706 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6707 \fi
6708 %
6709 \chardef\bbl@thetextdir\z@
6710 \chardef\bbl@thepardir\z@
6711 \def\bbl@setluadir#1#2{% l=\text/pardirection 2= 0:l 1:r 2:al
6712 \ifcase#2\relax
6713 \ifcase#1\else#1=\z@\fi
6714 \else
6715 \ifcase#1#1=\@ne\fi
6716 \fi}

```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (PP is the par dir) and `0x3` (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

```

6717 \def\bbl@thedir{0}
6718 \def\bbl@texkdir#1{%
6719 \bbl@setluadir\textdirection{#1}%
6720 \chardef\bbl@thetextdir#1\relax
6721 \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6722 \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6723 \def\bbl@pardir#1{% Used twice

```

```

6724 \bbl@setluadir\pardirection{#1}%
6725 \chardef\bbl@thepardir#1\relax}
6726 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6727 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to
'tabular', which is based on a fake math.

6728 \ifnum\bbl@bidimode>\z@ % Any bidi=
6729 \def\bbl@insidemath{0}%
6730 \def\bbl@everymath{\def\bbl@insidemath{1}}
6731 \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6732 \frozen@everymath\expandafter{%
6733 \expandafter\bbl@everymath\the\frozen@everymath}
6734 \frozen@everydisplay\expandafter{%
6735 \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6736 \AtBeginDocument{
6737 \directlua{
6738 function Babel.math_box_dir(head)
6739 if not (token.get_macro('bbl@insidemath') == '0') then
6740 if Babel.hlist_has_bidi(head) then
6741 local d = node.new(node.id'dir')
6742 d.dir = '+TRT'
6743 for item in node.traverse(head) do
6744 if item.id == 11 or item.id == node.id'glyph' then
6745 head = node.insert_before(head, item, d)
6746 break
6747 end
6748 end
6749 local inmath = false
6750 for item in node.traverse(head) do
6751 if item.id == 11 then
6752 inmath = (item.subtype == 0)
6753 elseif not inmath then
6754 node.set_attribute(item,
6755 Babel.attr_dir, token.get_macro('bbl@thedir'))
6756 end
6757 end
6758 end
6759 end
6760 return head
6761 end
6762 luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6763 "Babel.math_box_dir", 0)
6764 if Babel.unset_atdir then
6765 luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6766 "Babel.unset_atdir")
6767 luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6768 "Babel.unset_atdir")
6769 end
6770 luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6771 local dir = tex.mathdirection
6772 local n = node.new("dir")
6773 n.direction = dir
6774 head = node.insert_before(head, head, n)
6775 n = node.new("dir", 1)
6776 n.direction = dir
6777 head = node.insert_after(head, node.tail(head), n)
6778 return head
6779 end, "Babel.ensure_math_dir")
6780 }}%
6781 \fi

Experimental. Tentative name.

6782 \DeclareRobustCommand\localebox[1]{%

```

```

6783 {\def\bbl@insidemath{0}%
6784 \mbox{\foreignlanguage{\language}{#1}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidi=basic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I’ve made some progress in graphics, but they’re essentially hacks; I’ve also made some progress in ‘tabular’, but when I decided to tackle math (both standard math and ‘amsmath’) the nightmare began. I’m still not sure how ‘amsmath’ should be modified, but the main problem is that, boxes are “generic” containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with ‘math’ (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hhline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolumn` still fails.

```

6785 \bbl@trace{Redefinitions for bidi layout}
6786 %
6787 <<{*More package options}>> ≡
6788 \chardef\bbl@eqnpos\z@
6789 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6790 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6791 <</More package options>>

```

Fixes in luatex are sometimes done with flags. We first activate all all them.

```

6792 \ifnum\bbl@bidimode>\z@ % Any bidi=
6793 \matheqdirmode\@ne
6794 \matheptydisplaymode\@ne
6795 \breakafterdirmode\@ne
6796 \fixupboxesmode\@ne
6797 \let\bbl@eqnodir\relax
6798 \def\bbl@eqdel{()}
6799 \def\bbl@eqnum{%
6800   {\normalfont\normalcolor
6801     \expandafter\@firstoftwo\bbl@eqdel
6802     \theequation
6803     \expandafter\@secondoftwo\bbl@eqdel}}
6804 \def\bbl@puteqno#1{\eqno\hbox{#1}}
6805 \def\bbl@putleqno#1{\leqno\hbox{#1}}
6806 \def\bbl@eqno@flip#1{% So
6807   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6808     \eqno
6809     \hb@xt@.01pt{%
6810       \hb@xt@\displaywidth{\hss#1\glet\bbl@upset\@currentlabel}\hss}%
6811   \else
6812     \leqno\hbox{#1\glet\bbl@upset\@currentlabel}%
6813   \fi
6814   \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6815 \def\bbl@leqno@flip#1{%
6816   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6817     \leqno
6818     \hb@xt@.01pt{%
6819       \hss\hb@xt@\displaywidth{\#1\glet\bbl@upset\@currentlabel}\hss}%
6820   \else
6821     \eqno\hbox{#1\glet\bbl@upset\@currentlabel}%

```

```

6822 \fi
6823 \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6824 %
6825 \AtBeginDocument{%
6826 \ifx\bbl@noamsmath\relax\else
6827 \ifx\maketag@@@ \@undefined % Normal equation, eqnarray
6828 \ifnum\bbl@eqnpos=\tw@
6829 \bbl@replace\equation{\hb@xt@\linewidth}%
6830 {\hbox bdir\mathdirection to\linewidth}%
6831 \bbl@carg\bbl@sreplace{[ ]}{\hb@xt@\linewidth}%
6832 {\hbox bdir\mathdirection to\linewidth}%
6833 \fi
6834 \AddToHook{env/equation/begin}{%
6835 \ifnum\bbl@thetextdir>z@
6836 \let\@eqnnum\bbl@eqnum
6837 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6838 \chardef\bbl@thetextdir\z@
6839 \bbl@add\normalfont{\bbl@eqnodir}%
6840 \ifcase\bbl@eqnpos
6841 \let\bbl@puteqno\bbl@eqno@flip
6842 \or
6843 \let\bbl@puteqno\bbl@leqno@flip
6844 \fi
6845 \fi}%
6846 \ifnum\bbl@eqnpos=\tw@\else
6847 \def\endequation{\bbl@puteqno{\@eqnnum}$\$ \@ignoretrue}%
6848 \fi
6849 \AddToHook{env/eqnarray/begin}{%
6850 \ifnum\bbl@thetextdir>z@
6851 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6852 \chardef\bbl@thetextdir\z@
6853 \bbl@add\normalfont{\bbl@eqnodir}%
6854 \ifnum\bbl@eqnpos=\@ne
6855 \def\@eqnnum{%
6856 \setbox\z@\hbox{\bbl@eqnum}%
6857 \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%
6858 \else
6859 \let\@eqnnum\bbl@eqnum
6860 \fi
6861 \fi}
6862 \else % amstex
6863 \bbl@exp{% Hack to hide maybe undefined conditionals:
6864 \chardef\bbl@eqnpos=0<iftagsleft>1<else><if@fleqn>2<fi><fi>\relax}%
6865 \ifnum\bbl@eqnpos=\@ne
6866 \let\bbl@ams@lap\hbox
6867 \else
6868 \let\bbl@ams@lap\llap
6869 \fi
6870 \ExplSyntaxOn % Required by \bbl@sreplace with \intertext@
6871 \bbl@sreplace\intertext@{\normalbaselines}%
6872 {\normalbaselines
6873 \ifx\bbl@eqnodir\relax\else\bbl@pdir\@ne\bbl@eqnodir\fi}%
6874 \ExplSyntaxOff
6875 \def\bbl@ams@tagbox#1#2{#1{\bbl@eqnodir#2}}% #1=hbox|@lap|flip
6876 \ifx\bbl@ams@lap\hbox % leqno
6877 \def\bbl@ams@flip#1{%
6878 \hbox to 0.01pt{\hss\hbox to\displaywidth{#1}\hss}}%
6879 \else % eqno
6880 \def\bbl@ams@flip#1{%
6881 \hbox to 0.01pt{\hbox to\displaywidth{\hss{#1}}\hss}}%
6882 \fi
6883 \def\bbl@ams@preset#1{%
6884 \ifnum\bbl@thetextdir>z@

```

```

6885 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6886 \bbl@sreplace\textdef@{\hbox}{\bbl@ams@tagbox\hbox}%
6887 \bbl@sreplace\maketag@@@{\hbox}{\bbl@ams@tagbox#1}%
6888 \fi}%
6889 \def\bbl@ams@equation{%
6890 \ifnum\bbl@thetextdir>\z@
6891 \bbl@ams@preset\hbox
6892 \chardef\bbl@thetextdir\z@
6893 \bbl@add\normalfont{\bbl@eqnodir}%
6894 \ifcase\bbl@eqnpos
6895 \def\veqno##1##2{\bbl@eqno@flip{##1##2}}%
6896 \or
6897 \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6898 \fi
6899 \fi}%
6900 \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6901 \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6902 \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6903 \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6904 \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6905 \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6906 \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6907 \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6908 \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6909 \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6910 \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\hbox}%
6911 % Hackish, for proper alignment. Don't ask me why it works!!
6912 \bbl@exp{% Avoid a 'visible' conditional
6913 \\\AddToHook{env/align*/end}{\<iftag@>\<else>\\tag*{\<fi>}}%
6914 \\\AddToHook{env/alignat*/end}{\<iftag@>\<else>\\tag*{\<fi>}}%
6915 \AddToHook{env/flalign/begin}{\bbl@ams@preset\hbox}%
6916 \AddToHook{env/split/before}{%
6917 \ifnum\bbl@thetextdir>\z@
6918 \bbl@ifsamestring\@currentenv{equation}%
6919 {\ifx\bbl@ams@lap\hbox % leqno
6920 \def\bbl@ams@flip#1{%
6921 \hbox to 0.01pt{\hbox to\displaywidth{#1}\hss}\hss}}%
6922 \else
6923 \def\bbl@ams@flip#1{%
6924 \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}%
6925 \fi}%
6926 }%
6927 \fi}%
6928 \fi\fi}
6929 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6930 \def\bbl@provide@extra#1{%
6931 % == onchar ==
6932 \ifx\bbl@KVP@onchar\@nnil\else
6933 \bbl@luahyphenate
6934 \bbl@exp{%
6935 \\\AddToHook{env/document/before}{%
6936 {\let\\bbl@ifrestoring\\@firstoftwo
6937 \\\select@language{#1}}}}%
6938 \directlua{
6939 if Babel.locale_mapped == nil then
6940 Babel.locale_mapped = true
6941 Babel.linebreaking.add_before(Babel.locale_map, 1)
6942 Babel.loc_to_scr = {}
6943 Babel.chr_to_loc = Babel.chr_to_loc or {}
6944 end
6945 Babel.locale_props[\the\localeid].letters = false

```

```

6946 }%
6947 \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
6948 \ifin@
6949 \directlua{
6950   Babel.locale_props[\the\localeid].letters = true
6951 }%
6952 \fi
6953 \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
6954 \ifin@
6955 \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
6956   \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}%
6957 \fi
6958 \bbl@exp{\bbl@add{\bbl@starthyphens
6959   {\bbl@patterns@lua{\language name}}}%
6960 \directlua{
6961   if Babel.script_blocks['\bbl@cl{sbc}'] then
6962     Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
6963     Babel.locale_props[\the\localeid].lg = \the\@nameuse{l@language name}\space
6964   end
6965 }%
6966 \fi
6967 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
6968 \ifin@
6969 \bbl@ifunset{bbl@lsys@language name}{\bbl@provide@lsys@language name}%
6970 \bbl@ifunset{bbl@wdir@language name}{\bbl@provide@dirs@language name}%
6971 \directlua{
6972   if Babel.script_blocks['\bbl@cl{sbc}'] then
6973     Babel.loc_to_scr[\the\localeid] =
6974       Babel.script_blocks['\bbl@cl{sbc}']
6975   end}%
6976 \ifx\bbl@mapselect\@undefined
6977   \AtBeginDocument{%
6978     \bbl@patchfont{\bbl@mapselect}%
6979     {\selectfont}%
6980 \def\bbl@mapselect{%
6981   \let\bbl@mapselect\relax
6982   \edef\bbl@prefontid{\fontid\font}%
6983 \def\bbl@mapdir##1{%
6984   \begingroup
6985     \setbox\z@ \hbox{% Force text mode
6986       \def\language name{##1}%
6987       \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
6988       \bbl@switchfont
6989       \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
6990         \directlua{
6991           Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
6992             ['/\bbl@prefontid'] = \fontid\font\space}%
6993         \fi}%
6994       \endgroup}%
6995   \fi
6996   \bbl@exp{\bbl@add{\bbl@mapselect{\bbl@mapdir@language name}}}%
6997 \fi
6998 \fi
6999 % == mapfont ==
7000 % For bidi texts, to switch the font based on direction. Deprecated
7001 \ifx\bbl@KVP@mapfont\@nnil\else
7002   \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}%
7003   {\bbl@error{unknown-mapfont}}}%
7004 \bbl@ifunset{bbl@lsys@language name}{\bbl@provide@lsys@language name}%
7005 \bbl@ifunset{bbl@wdir@language name}{\bbl@provide@dirs@language name}%
7006 \ifx\bbl@mapselect\@undefined
7007   \AtBeginDocument{%
7008     \bbl@patchfont{\bbl@mapselect}%

```



```

7009     {\selectfont}}}%
7010 \def\bbl@mapselect{%
7011     \let\bbl@mapselect\relax
7012     \edef\bbl@prefontid{\fontid\font}}}%
7013 \def\bbl@mapdir##1{%
7014     {\def\language#1}%
7015     \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7016     \bbl@switchfont
7017     \directlua{Babel.fontmap
7018         [\the\csname bbl@wdir@##1\endcsname]%
7019         [\bbl@prefontid]=\fontid\font}}}%
7020 \fi
7021 \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\language}}}%
7022 \fi
7023 % == Line breaking: CJK quotes ==
7024 \ifcase\bbl@engine\or
7025     \bbl@xin@{/c}{/\bbl@ccl{lnbrk}}}%
7026     \ifin@
7027         \bbl@ifunset{bbl@quote@\language}{}%
7028         {\directlua{
7029             Babel.locale_props[\the\localeid].cjk_quotes = {}
7030             local cs = 'op'
7031             for c in string.utfvalues(
7032                 [[\csname bbl@quote@\language\endcsname]]) do
7033                 if Babel.cjk_characters[c].c == 'qu' then
7034                     Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7035                 end
7036                 cs = ( cs == 'op') and 'cl' or 'op'
7037             end
7038         }}%
7039 \fi
7040 \fi
7041 % == Counters: mapdigits ==
7042 % Native digits
7043 \ifx\bbl@KVP@mapdigits\@nnil\else
7044     \bbl@ifunset{bbl@dgnat@\language}{}%
7045     {\bbl@activate@preotf
7046     \directlua{
7047         Babel.digits_mapped = true
7048         Babel.digits = Babel.digits or {}
7049         Babel.digits[\the\localeid] =
7050             table.pack(string.utfvalue('\bbl@ccl{dgnat}'))
7051         if not Babel.numbers then
7052             function Babel.numbers(head)
7053                 local LOCALE = Babel.attr_locale
7054                 local GLYPH = node.id'glyph'
7055                 local inmath = false
7056                 for item in node.traverse(head) do
7057                     if not inmath and item.id == GLYPH then
7058                         local temp = node.get_attribute(item, LOCALE)
7059                         if Babel.digits[temp] then
7060                             local chr = item.char
7061                             if chr > 47 and chr < 58 then
7062                                 item.char = Babel.digits[temp][chr-47]
7063                             end
7064                         end
7065                     elseif item.id == node.id'math' then
7066                         inmath = (item.subtype == 0)
7067                     end
7068                 end
7069                 return head
7070             end
7071         end

```

```

7072     }}%
7073 \fi
7074 % == transforms ==
7075 \ifx\bbk@KVP@transforms\@nnil\else
7076   \def\bbk@elt##1##2##3{%
7077     \in@{$transforms.}{##1}%
7078     \ifin@
7079       \def\bbk@tempa{##1}%
7080       \bbk@replace\bbk@tempa{transforms.}{}%
7081       \bbk@carg\bbk@transforms{babel\bbk@tempa}{##2}{##3}%
7082     \fi}%
7083 \bbk@exp{%
7084   \\bbk@ifblank{\bbk@cl{dgnat}}}%
7085   {\let\\bbk@tempa\relax}%
7086   {\def\\bbk@tempa{%
7087     \\bbk@elt{transforms.prehyphenation}%
7088     {digits.native.1.0}{([0-9])}%
7089     \\bbk@elt{transforms.prehyphenation}%
7090     {digits.native.1.1}{string=|0123456789|string|\bbk@cl{dgnat}}}}}%
7091 \ifx\bbk@tempa\relax\else
7092   \toks@{\expandafter\expandafter\expandafter{%
7093     \csname bbl@inidata@\language\endcsname}%
7094     \bbk@csarg\edef{inidata@\language}{%
7095       \unexpanded\expandafter{\bbk@tempa}%
7096       \the\toks@}%
7097   \fi
7098   \csname bbl@inidata@\language\endcsname
7099   \bbk@release@transforms\relax % \relax closes the last item.
7100 \fi}

```

Start tabular here:

```

7101 \def\localerestoredirs{%
7102   \ifcase\bbk@thetextdir
7103     \ifnum\textdirection=\z@ \else \textdirection=\z@ \fi
7104   \else
7105     \ifnum\textdirection=\@ne \else \textdirection=\@ne \fi
7106   \fi
7107   \ifcase\bbk@thepardir
7108     \ifnum\pardirection=\z@ \else \pardirection=\z@ \bodydirection=\z@ \fi
7109   \else
7110     \ifnum\pardirection=\@ne \else \pardirection=\@ne \bodydirection=\@ne \fi
7111   \fi}
7112 %
7113 \IfBabelLayout{tabular}%
7114   {\chardef\bbk@tabular@mode\z@}% All RTL
7115   {\IfBabelLayout{rtabular}%
7116     {\chardef\bbk@tabular@mode\@ne}%
7117     {\chardef\bbk@tabular@mode\z@}}% Mixed, with LTR cols
7118 %
7119 \ifnum\bbk@bidimode>\@ne % Any lua bidi= except default=1
7120 % Redefine: vrules mess up dirs (why?).
7121 \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7122 \ifcase\bbk@tabular@mode \else % 1 = Mixed
7123   \let\bbk@parabefore\relax
7124   \AddToHook{para/before}{\bbk@parabefore}
7125   \AtBeginDocument{%
7126     \bbk@replace\@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7127     \ifnum\bbk@tabular@mode=\@ne
7128       \bbk@ifunset{\tabclassz}{}%
7129       \bbk@exp{% Hide conditionals
7130         \\bbk@sreplace\\ \tabclassz
7131         {\<ifcase>\\ \chnum}%
7132         {\localerestoredirs\<ifcase>\\ \chnum}}}%

```

```

7133 \@ifpackageloaded{colortbl}%
7134 {\bbl@sreplace\@classz
7135 {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7136 {\@ifpackageloaded{array}%
7137 {\bbl@exp{% Hide conditionals
7138 \\\bbl@sreplace\\ \@classz
7139 {\<ifcase>\\ \@chnum}%
7140 {\bgroup\\ \localerestoredirs\<ifcase>\\ \@chnum}%
7141 \\\bbl@sreplace\\ \@classz
7142 {\do@row@strut\<fi>}{\do@row@strut\<fi>\egroup}}}%
7143 {}}%
7144 \fi}%
7145 \fi

```

Very likely the `\output` routine must be patched in a quite general way to make sure the `\bodydir` is set to `\pagedir`. Note outside `\output` they can be different (and often are). For the moment, two *ad hoc* changes.

```

7146 \AtBeginDocument{%
7147 \@ifpackageloaded{multicol}%
7148 {\toks\expandafter{\multi@column@out}%
7149 \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7150 {}}%
7151 \@ifpackageloaded{paracol}%
7152 {\edef\pcol@output{%
7153 \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7154 {}}%
7155 \fi

```

Finish here if there in no layout.

```
7156 \ifx\bbl@opt@layout\@nnil\endinput\fi
```

`\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7157 \chardef\bbl@trace@vboxdir\z@
7158 \ifnum\bbl@bidimode>\z@ % Any bidi=
7159 \IfBabelLayout{nopars}{}
7160 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7161 \IfBabelLayout{pars}
7162 {\chardef\bbl@trace@vboxdir\@ne
7163 \def\@hangfrom#1{%
7164 \setbox\@tempboxa\hbox{#1}}%
7165 \hangindent\wd\@tempboxa
7166 \ifnum\bbl@vbox@bodydir=\pardirection\else
7167 \shapemode\@ne
7168 \fi
7169 \noindent\box\@tempboxa}}
7170 {}
7171 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7172 \IfBabelLayout{lists}
7173 {\chardef\bbl@trace@vboxdir\@ne
7174 \let\bbl@0L@list\list
7175 \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7176 \let\bbl@NL@list\list
7177 \def\bbl@listparshape#1#2#3{%
7178 \parshape #1 #2 #3 %
7179 \ifnum\bbl@vbox@bodydir=\pardirection\else
7180 \shapemode\tw@
7181 \fi}}
7182 {}
7183 %
7184 \ifcase\bbl@trace@vboxdir\else
7185 \AtBeginDocument{\chardef\bbl@vbox@bodydir\pardirection}%

```

```

7186 \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7187 \let\bbl@bidi@vbox\everyvbox
7188 \@nameuse{newtoks}\everyvbox % \outer in Plain
7189 \everyvbox\expandafter{\the\bbl@bidi@vbox}%
7190 \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7191 \fi
7192 %
7193 \IfBabelLayout{graphics}
7194 {\let\bbl@pictresetdir\relax
7195 \def\bbl@pictsetdir#1{%
7196 \ifcase\bbl@thetextdir
7197 \let\bbl@pictresetdir\relax
7198 \else
7199 \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7200 \or\textdir TLT
7201 \else\bodydir TLT \textdir TLT
7202 \fi
7203 % \(\text|par)dir required in pgf:
7204 \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7205 \fi}%
7206 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7207 \directlua{
7208 Babel.get_picture_dir = true
7209 Babel.picture_has_bidi = 0
7210 %
7211 function Babel.picture_dir (head)
7212 if not Babel.get_picture_dir then return head end
7213 if Babel.hlist_has_bidi(head) then
7214 Babel.picture_has_bidi = 1
7215 end
7216 return head
7217 end
7218 luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7219 "Babel.picture_dir")
7220 }%
7221 \AddToHook{package/graphics/after}{%
7222 \bbl@exp{%
7223 \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7224 {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#2}}}}%
7225 \AddToHook{package/graphicsx/after}{%
7226 \bbl@exp{%
7227 \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7228 {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#2}}}}%
7229 \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7230 {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#3}}}}%
7231 \\bbl@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}%
7232 \AtBeginDocument{%
7233 \long\def\put(#1,#2)#3{%
7234 \@killglue
7235 % Try:
7236 \ifx\bbl@pictresetdir\relax
7237 \def\bbl@tempc{0}%
7238 \else
7239 \directlua{
7240 Babel.get_picture_dir = true
7241 Babel.picture_has_bidi = 0
7242 }%
7243 \setbox\z@\hb@xt@{\z@}%
7244 \@defaultunitsset\@tempdimc{#1}\unitlength
7245 \kern\@tempdimc
7246 #3\hss}%
7247 \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7248 \fi

```

```

7249 % Do:
7250 \@defaultunitsset\@tempdimc{#2}\unitlength
7251 \raise\@tempdimc\hbxt@z@{%
7252 \@defaultunitsset\@tempdimc{#1}\unitlength
7253 \kern\@tempdimc
7254 {\ifnum\bbbl@tempc>z@\bbbl@pictresetdir\fi#3}\hss}%
7255 \ignorespaces}%
7256 \MakeRobust\put}%
7257 \AtBeginDocument
7258 {\AddToHook{cmd/diagbox@pict/before}{\let\bbbl@pictsetdir\@gobble}%
7259 \ifx\pgfpicture\@undefined\else
7260 \AddToHook{env/pgfpicture/begin}{\bbbl@pictsetdir\@ne}%
7261 \bbbl@add\pgfinterruptpicture{\bbbl@pictresetdir}%
7262 \bbbl@add\pgfsys@beginpicture{\bbbl@pictsetdir\z@}%
7263 \fi
7264 \ifx\tikzpicture\@undefined\else
7265 \AddToHook{env/tikzpicture/begin}{\bbbl@pictsetdir\tw@}%
7266 \bbbl@add\tikz@atbegin@node{\bbbl@pictresetdir}%
7267 \bbbl@sreplace\tikz{\begingroup}{\begingroup\bbbl@pictsetdir\tw@}%
7268 \bbbl@sreplace\tikzpicture{\begingroup}{\begingroup\bbbl@pictsetdir\tw@}%
7269 \fi
7270 }}
7271 {}

```

Implicitly reverses sectioning labels in `bidi=basic-r`, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes `bidi=basic`, but there are some additional readjustments for `bidi=default`.

```

7272 \IfBabelLayout{counters*}%
7273 {\bbbl@add\bbbl@opt@layout{.counters.}%
7274 \directlua{
7275 \lua{
7276 \lua{
7277 }{}
7278 \IfBabelLayout{counters}%
7279 {\let\bbbl@latin@arabic=\@arabic
7280 \let\bbbl@0L@arabic=\@arabic
7281 \def\@arabic#1{\babelsublr{\bbbl@latin@arabic#1}}%
7282 \@ifpackagewith{babel}{bidi=default}%
7283 {\let\bbbl@asci@roman=\@roman
7284 \let\bbbl@0L@roman=\@roman
7285 \def\@roman#1{\babelsublr{\ensureascii{\bbbl@asci@roman#1}}}%
7286 \let\bbbl@asci@Roman=\@Roman
7287 \let\bbbl@0L@Roman=\@Roman
7288 \def\@Roman#1{\babelsublr{\ensureascii{\bbbl@asci@Roman#1}}}%
7289 \let\bbbl@0L@labelenumii\labelenumii
7290 \def\labelenumii{}\theenumii}%
7291 \let\bbbl@0L@p@enumiii\p@enumiii
7292 \def\p@enumiii{\p@enumii}\theenumii}}{}{}

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7293 \IfBabelLayout{extras}%
7294 {\bbbl@ncarg\let\bbbl@0L@underline{underline }%
7295 \bbbl@carg\bbbl@sreplace{underline }{\hbox}%
7296 {\def\bbbl@insidemath{0}\hbox bdir\ifcase\bbbl@thetextdir\z@else\@ne\fi}%
7297 \let\bbbl@0L@LaTeXe\LaTeXe
7298 \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7299 \if b\expandafter\car\series\@nil\boldmath\fi
7300 \babelsublr{\LaTeXe\kern.15em2$_{\textstyle\varepsilon}$}}}
7301 {}
7302 </luatex>

```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```
7303 (*transforms)
7304 Babel.linebreaking.replacements = {}
7305 Babel.linebreaking.replacements[0] = {} -- pre
7306 Babel.linebreaking.replacements[1] = {} -- post
7307
7308 function Babel.tovalue(v)
7309   if type(v) == 'table' then
7310     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7311   else
7312     return v
7313   end
7314 end
7315
7316 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7317
7318 function Babel.set_hboxed(head, gc)
7319   for item in node.traverse(head) do
7320     node.set_attribute(item, Babel.attr_hboxed, 1)
7321   end
7322   return head
7323 end
7324
7325 Babel.fetch_subtext = {}
7326
7327 Babel.ignore_pre_char = function(node)
7328   return (node.lang == Babel.nohyphenation)
7329 end
7330
7331 Babel.show_transforms = false
7332
7333 -- Merging both functions doesn't seem feasible, because there are too
7334 -- many differences.
7335 Babel.fetch_subtext[0] = function(head)
7336   local word_string = ''
7337   local word_nodes = {}
7338   local lang
7339   local item = head
7340   local inmath = false
7341
7342   while item do
7343     if item.id == 11 then
7344       inmath = (item.subtype == 0)
7345     end
7346
7347     if inmath then
7348       -- pass
7349     elseif item.id == 29 then
7350       local locale = node.get_attribute(item, Babel.attr_locale)
```

```

7353
7354     if lang == locale or lang == nil then
7355         lang = lang or locale
7356         if Babel.ignore_pre_char(item) then
7357             word_string = word_string .. Babel.us_char
7358         else
7359             if node.has_attribute(item, Babel.attr_hboxed) then
7360                 word_string = word_string .. Babel.us_char
7361             else
7362                 word_string = word_string .. unicode.utf8.char(item.char)
7363             end
7364         end
7365         word_nodes[#word_nodes+1] = item
7366     else
7367         break
7368     end
7369
7370 elseif item.id == 12 and item.subtype == 13 then
7371     if node.has_attribute(item, Babel.attr_hboxed) then
7372         word_string = word_string .. Babel.us_char
7373     else
7374         word_string = word_string .. ' '
7375     end
7376     word_nodes[#word_nodes+1] = item
7377
7378 -- Ignore leading unrecognized nodes, too.
7379 elseif word_string ~= '' then
7380     word_string = word_string .. Babel.us_char
7381     word_nodes[#word_nodes+1] = item -- Will be ignored
7382 end
7383
7384 item = item.next
7385 end
7386
7387 -- Here and above we remove some trailing chars but not the
7388 -- corresponding nodes. But they aren't accessed.
7389 if word_string:sub(-1) == ' ' then
7390     word_string = word_string:sub(1,-2)
7391 end
7392 if Babel.show_transforms then texio.write_nl(word_string) end
7393 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7394 return word_string, word_nodes, item, lang
7395 end
7396
7397 Babel.fetch_subtext[1] = function(head)
7398     local word_string = ''
7399     local word_nodes = {}
7400     local lang
7401     local item = head
7402     local inmath = false
7403
7404     while item do
7405
7406         if item.id == 11 then
7407             inmath = (item.subtype == 0)
7408         end
7409
7410         if inmath then
7411             -- pass
7412         end
7413
7414         elseif item.id == 29 then
7415             if item.lang == lang or lang == nil then
7416                 lang = lang or item.lang

```

```

7416         if node.has_attribute(item, Babel.attr_hboxed) then
7417             word_string = word_string .. Babel.us_char
7418         elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7419             word_string = word_string .. Babel.us_char
7420         else
7421             word_string = word_string .. unicode.utf8.char(item.char)
7422         end
7423         word_nodes[#word_nodes+1] = item
7424     else
7425         break
7426     end
7427
7428 elseif item.id == 7 and item.subtype == 2 then
7429     if node.has_attribute(item, Babel.attr_hboxed) then
7430         word_string = word_string .. Babel.us_char
7431     else
7432         word_string = word_string .. '='
7433     end
7434     word_nodes[#word_nodes+1] = item
7435
7436 elseif item.id == 7 and item.subtype == 3 then
7437     if node.has_attribute(item, Babel.attr_hboxed) then
7438         word_string = word_string .. Babel.us_char
7439     else
7440         word_string = word_string .. '|'
7441     end
7442     word_nodes[#word_nodes+1] = item
7443
7444 -- (1) Go to next word if nothing was found, and (2) implicitly
7445 -- remove leading USs.
7446 elseif word_string == '' then
7447     -- pass
7448
7449 -- This is the responsible for splitting by words.
7450 elseif (item.id == 12 and item.subtype == 13) then
7451     break
7452
7453 else
7454     word_string = word_string .. Babel.us_char
7455     word_nodes[#word_nodes+1] = item -- Will be ignored
7456 end
7457
7458 item = item.next
7459 end
7460 if Babel.show_transforms then texio.write_nl(word_string) end
7461 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7462 return word_string, word_nodes, item, lang
7463 end
7464
7465 function Babel.pre_hyphenate_replace(head)
7466     Babel.hyphenate_replace(head, 0)
7467 end
7468
7469 function Babel.post_hyphenate_replace(head)
7470     Babel.hyphenate_replace(head, 1)
7471 end
7472
7473 Babel.us_char = string.char(31)
7474
7475 function Babel.hyphenate_replace(head, mode)
7476     local u = unicode.utf8
7477     local lbkr = Babel.linebreaking.replacements[mode]
7478     local tovalue = Babel.tovalue

```



```

7479
7480 local word_head = head
7481
7482 if Babel.show_transforms then
7483   texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7484 end
7485
7486 while true do -- for each subtext block
7487
7488   local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7489
7490   if Babel.debug then
7491     print()
7492     print((mode == 0) and '====<' or '====>', w)
7493   end
7494
7495   if nw == nil and w == '' then break end
7496
7497   if not lang then goto next end
7498   if not lbkr[lang] then goto next end
7499
7500   -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7501   -- loops are nested.
7502   for k=1, #lbkr[lang] do
7503     local p = lbkr[lang][k].pattern
7504     local r = lbkr[lang][k].replace
7505     local attr = lbkr[lang][k].attr or -1
7506
7507     if Babel.debug then
7508       print('*****', p, mode)
7509     end
7510
7511     -- This variable is set in some cases below to the first *byte*
7512     -- after the match, either as found by u.match (faster) or the
7513     -- computed position based on sc if w has changed.
7514     local last_match = 0
7515     local step = 0
7516
7517     -- For every match.
7518     while true do
7519       if Babel.debug then
7520         print('====')
7521       end
7522       local new -- used when inserting and removing nodes
7523       local dummy_node -- used by after
7524
7525       local matches = { u.match(w, p, last_match) }
7526
7527       if #matches < 2 then break end
7528
7529       -- Get and remove empty captures (with ()'s, which return a
7530       -- number with the position), and keep actual captures
7531       -- (from (...)), if any, in matches.
7532       local first = table.remove(matches, 1)
7533       local last = table.remove(matches, #matches)
7534       -- Non re-fetched substrings may contain \31, which separates
7535       -- subsubstrings.
7536       if string.find(w:sub(first, last-1), Babel.us_char) then break end
7537
7538       local save_last = last -- with A()BC()D, points to D
7539
7540       -- Fix offsets, from bytes to unicode. Explained above.
7541       first = u.len(w:sub(1, first-1)) + 1

```

```

7542     last = u.len(w:sub(1, last-1)) -- now last points to C
7543
7544     -- This loop stores in a small table the nodes
7545     -- corresponding to the pattern. Used by 'data' to provide a
7546     -- predictable behavior with 'insert' (w_nodes is modified on
7547     -- the fly), and also access to 'remove'd nodes.
7548     local sc = first-1          -- Used below, too
7549     local data_nodes = {}
7550
7551     local enabled = true
7552     for q = 1, last-first+1 do
7553         data_nodes[q] = w_nodes[sc+q]
7554         if enabled
7555             and attr > -1
7556             and not node.has_attribute(data_nodes[q], attr)
7557         then
7558             enabled = false
7559         end
7560     end
7561
7562     -- This loop traverses the matched substring and takes the
7563     -- corresponding action stored in the replacement list.
7564     -- sc = the position in substr nodes / string
7565     -- rc = the replacement table index
7566     local rc = 0
7567
7568     ----- TODO. dummy_node?
7569     while rc < last-first+1 or dummy_node do -- for each replacement
7570         if Babel.debug then
7571             print('.....', rc + 1)
7572         end
7573         sc = sc + 1
7574         rc = rc + 1
7575
7576         if Babel.debug then
7577             Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7578             local ss = ''
7579             for itt in node.traverse(head) do
7580                 if itt.id == 29 then
7581                     ss = ss .. unicode.utf8.char(itt.char)
7582                 else
7583                     ss = ss .. '{' .. itt.id .. '}'
7584                 end
7585             end
7586             print('*****', ss)
7587         end
7588
7589         local crep = r[rc]
7590         local item = w_nodes[sc]
7591         local item_base = item
7592         local placeholder = Babel.us_char
7593         local d
7594
7595         if crep and crep.data then
7596             item_base = data_nodes[crep.data]
7597         end
7598
7599         if crep then
7600             step = crep.step or step
7601         end
7602
7603         if crep and crep.after then

```

```

7605         crep.insert = true
7606         if dummy_node then
7607             item = dummy_node
7608         else -- TODO. if there is a node after?
7609             d = node.copy(item_base)
7610             head, item = node.insert_after(head, item, d)
7611             dummy_node = item
7612         end
7613     end
7614
7615     if crep and not crep.after and dummy_node then
7616         node.remove(head, dummy_node)
7617         dummy_node = nil
7618     end
7619
7620     if not enabled then
7621         last_match = save_last
7622         goto next
7623
7624     elseif crep and next(crep) == nil then -- = {}
7625         if step == 0 then
7626             last_match = save_last    -- Optimization
7627         else
7628             last_match = utf8.offset(w, sc+step)
7629         end
7630         goto next
7631
7632     elseif crep == nil or crep.remove then
7633         node.remove(head, item)
7634         table.remove(w_nodes, sc)
7635         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7636         sc = sc - 1 -- Nothing has been inserted.
7637         last_match = utf8.offset(w, sc+1+step)
7638         goto next
7639
7640     elseif crep and crep.kashida then
7641         node.set_attribute(item,
7642             Babel.attr_kashida,
7643             crep.kashida)
7644         last_match = utf8.offset(w, sc+1+step)
7645         goto next
7646
7647     elseif crep and crep.string then
7648         local str = crep.string(matches)
7649         if str == '' then -- Gather with nil
7650             node.remove(head, item)
7651             table.remove(w_nodes, sc)
7652             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7653             sc = sc - 1 -- Nothing has been inserted.
7654         else
7655             local loop_first = true
7656             for s in string.utfvalues(str) do
7657                 d = node.copy(item_base)
7658                 d.char = s
7659                 if loop_first then
7660                     loop_first = false
7661                     head, new = node.insert_before(head, item, d)
7662                     if sc == 1 then
7663                         word_head = head
7664                     end
7665                     w_nodes[sc] = d
7666                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7667                 else

```

```

7668         sc = sc + 1
7669         head, new = node.insert_before(head, item, d)
7670         table.insert(w_nodes, sc, new)
7671         w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7672     end
7673     if Babel.debug then
7674         print('.....', 'str')
7675         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7676     end
7677     end -- for
7678     node.remove(head, item)
7679 end -- if ''
7680 last_match = utf8.offset(w, sc+1+step)
7681 goto next
7682
7683 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7684     d = node.new(7, 3) -- (disc, regular)
7685     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7686     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7687     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7688     d.attr = item_base.attr
7689     if crep.pre == nil then -- TeXbook p96
7690         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7691     else
7692         d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7693     end
7694     placeholder = '|'
7695     head, new = node.insert_before(head, item, d)
7696
7697 elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7698     -- ERROR
7699
7700 elseif crep and crep.penalty then
7701     d = node.new(14, 0) -- (penalty, userpenalty)
7702     d.attr = item_base.attr
7703     d.penalty = tovalue(crep.penalty)
7704     head, new = node.insert_before(head, item, d)
7705
7706 elseif crep and crep.space then
7707     -- 655360 = 10 pt = 10 * 65536 sp
7708     d = node.new(12, 13) -- (glue, spaceskip)
7709     local quad = font.getfont(item_base.font).size or 655360
7710     node.setglue(d, tovalue(crep.space[1]) * quad,
7711                 tovalue(crep.space[2]) * quad,
7712                 tovalue(crep.space[3]) * quad)
7713     if mode == 0 then
7714         placeholder = ' '
7715     end
7716     head, new = node.insert_before(head, item, d)
7717
7718 elseif crep and crep.norule then
7719     -- 655360 = 10 pt = 10 * 65536 sp
7720     d = node.new(2, 3) -- (rule, empty) = \no*rule
7721     local quad = font.getfont(item_base.font).size or 655360
7722     d.width = tovalue(crep.norule[1]) * quad
7723     d.height = tovalue(crep.norule[2]) * quad
7724     d.depth = tovalue(crep.norule[3]) * quad
7725     head, new = node.insert_before(head, item, d)
7726
7727 elseif crep and crep.spacefactor then
7728     d = node.new(12, 13) -- (glue, spaceskip)
7729     local base_font = font.getfont(item_base.font)
7730     node.setglue(d,

```

```

7731         tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7732         tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7733         tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7734     if mode == 0 then
7735         placeholder = ' '
7736     end
7737     head, new = node.insert_before(head, item, d)
7738
7739 elseif mode == 0 and crep and crep.space then
7740     -- ERROR
7741
7742 elseif crep and crep.kern then
7743     d = node.new(13, 1) -- (kern, user)
7744     local quad = font.getfont(item_base.font).size or 655360
7745     d.attr = item_base.attr
7746     d.kern = tovalue(crep.kern) * quad
7747     head, new = node.insert_before(head, item, d)
7748
7749 elseif crep and crep.node then
7750     d = node.new(crep.node[1], crep.node[2])
7751     d.attr = item_base.attr
7752     head, new = node.insert_before(head, item, d)
7753
7754 end -- i.e., replacement cases
7755
7756 -- Shared by disc, space(factor), kern, node and penalty.
7757 if sc == 1 then
7758     word_head = head
7759 end
7760 if crep.insert then
7761     w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7762     table.insert(w_nodes, sc, new)
7763     last = last + 1
7764 else
7765     w_nodes[sc] = d
7766     node.remove(head, item)
7767     w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7768 end
7769
7770 last_match = utf8.offset(w, sc+1+step)
7771
7772 ::next::
7773
7774 end -- for each replacement
7775
7776 if Babel.show_transforms then texio.write_nl('> ' .. w) end
7777 if Babel.debug then
7778     print('.....', '/')
7779     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7780 end
7781
7782 if dummy_node then
7783     node.remove(head, dummy_node)
7784     dummy_node = nil
7785 end
7786
7787 end -- for match
7788
7789 end -- for patterns
7790
7791 ::next::
7792 word_head = nw
7793 end -- for substring

```

```

7794
7795 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7796 return head
7797 end
7798
7799 -- This table stores capture maps, numbered consecutively
7800 Babel.capture_maps = {}
7801
7802 function Babel.esc_hex_to_char(h)
7803   if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7804     tex.getcatcode(tonumber(h, 16)) ~= 12 then
7805     return string.format([[\\Uchar"%X ]], tonumber(h,16))
7806   else
7807     return unicode.utf8.char(tonumber(h, 16))
7808   end
7809 end
7810
7811 -- The following functions belong to the next macro
7812 function Babel.capture_func(key, cap)
7813   local ret = "[[" .. cap:gsub('{{([0-9])}}', "]]..m[%1]..[" .. "]"
7814   local cnt
7815   local u = unicode.utf8
7816   ret = u.gsub(ret, '{{(%%x%%x%%x+}}', '\\x01%1\\x04')
7817   ret, cnt = ret:gsub('{{([0-9])|([^\^]|+)|([.-])}}', Babel.capture_func_map)
7818   ret = u.gsub(ret, '\\x01(%%x%%x%%x+)%x04', Babel.esc_hex_to_char)
7819   ret = ret:gsub("%[%[%]%%].%", '')
7820   ret = ret:gsub("%.%.%[%[%]%%]", '')
7821   return key .. [[=function(m) return ]] .. ret .. [[ end]]
7822 end
7823
7824 function Babel.capt_map(from, mapno)
7825   return Babel.capture_maps[mapno][from] or from
7826 end
7827
7828 -- Handle the {n|abc|ABC} syntax in captures
7829 function Babel.capture_func_map(capno, from, to)
7830   local u = unicode.utf8
7831   from = u.gsub(from, '\\x01(%%x%%x%%x+)%x04',
7832     function (n)
7833       return u.char(tonumber(n, 16))
7834     end)
7835   to = u.gsub(to, '\\x01(%%x%%x%%x+)%x04',
7836     function (n)
7837       return u.char(tonumber(n, 16))
7838     end)
7839   local froms = {}
7840   for s in string.utfcharacters(from) do
7841     table.insert(froms, s)
7842   end
7843   local cnt = 1
7844   table.insert(Babel.capture_maps, {})
7845   local mlen = table.getn(Babel.capture_maps)
7846   for s in string.utfcharacters(to) do
7847     Babel.capture_maps[mlen][froms[cnt]] = s
7848     cnt = cnt + 1
7849   end
7850   return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7851     (mlen) .. " ).." .. "[["
7852 end
7853
7854 -- Create/Extend reversed sorted list of kashida weights:
7855 function Babel.capture_kashida(key, wt)
7856   wt = tonumber(wt)

```

```

7857 if Babel.kashida_wts then
7858   for p, q in ipairs(Babel.kashida_wts) do
7859     if wt == q then
7860       break
7861     elseif wt > q then
7862       table.insert(Babel.kashida_wts, p, wt)
7863       break
7864     elseif table.getn(Babel.kashida_wts) == p then
7865       table.insert(Babel.kashida_wts, wt)
7866     end
7867   end
7868 else
7869   Babel.kashida_wts = { wt }
7870 end
7871 return 'kashida = ' .. wt
7872 end
7873
7874 function Babel.capture_node(id, subtype)
7875   local sbt = 0
7876   for k, v in pairs(node.subtypes(id)) do
7877     if v == subtype then sbt = k end
7878   end
7879   return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7880 end
7881
7882 -- Experimental: applies prehyphenation transforms to a string (letters
7883 -- and spaces).
7884 function Babel.string_prehyphenation(str, locale)
7885   local n, head, last, res
7886   head = node.new(8, 0) -- dummy (hack just to start)
7887   last = head
7888   for s in string.utfvalues(str) do
7889     if s == 20 then
7890       n = node.new(12, 0)
7891     else
7892       n = node.new(29, 0)
7893       n.char = s
7894     end
7895     node.set_attribute(n, Babel.attr_locale, locale)
7896     last.next = n
7897     last = n
7898   end
7899   head = Babel.hyphenate_replace(head, 0)
7900   res = ''
7901   for n in node.traverse(head) do
7902     if n.id == 12 then
7903       res = res .. ' '
7904     elseif n.id == 29 then
7905       res = res .. unicode.utf8.char(n.char)
7906     end
7907   end
7908   tex.print(res)
7909 end
7910 </transforms>

```

10.14 Lua: Auto bidi with basic and basic-r

The file babel-data-bidi.lua currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},

```

```
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%
```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In babel the `dir` is set by a higher protocol based on the language/script, which in turn sets the correct `dir` (`<l>`, `<r>` or `<al>`).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don’t think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where `luatex` excels, because everything related to bidi writing is under our control.

```
7911 (*basic-r)
7912 Babel.bidi_enabled = true
7913
7914 require('babel-data-bidi.lua')
7915
7916 local characters = Babel.characters
7917 local ranges = Babel.ranges
7918
7919 local DIR = node.id("dir")
7920
7921 local function dir_mark(head, from, to, outer)
7922   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7923   local d = node.new(DIR)
7924   d.dir = '+' .. dir
7925   node.insert_before(head, from, d)
7926   d = node.new(DIR)
7927   d.dir = '-' .. dir
7928   node.insert_after(head, to, d)
7929 end
7930
7931 function Babel.bidi(head, ispar)
7932   local first_n, last_n          -- first and last char with nums
7933   local last_es                  -- an auxiliary 'last' used with nums
7934   local first_d, last_d          -- first and last char in L/R block
7935   local dir, dir_real
```

Next also depends on `script/lang` (`<al>/<r>`). To be set by `babel.tex`. `pardir` is dangerous, could be (re)set but it should be changed only in `vmode`. There are two strong’s – `strong = l/al/r` and `strong_lr = l/r` (there must be a better way):

```
7936 local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7937 local strong_lr = (strong == 'l') and 'l' or 'r'
```



```

7938 local outer = strong
7939
7940 local new_dir = false
7941 local first_dir = false
7942 local inmath = false
7943
7944 local last_lr
7945
7946 local type_n = ''
7947
7948 for item in node.traverse(head) do
7949
7950   -- three cases: glyph, dir, otherwise
7951   if item.id == node.id'glyph'
7952     or (item.id == 7 and item.subtype == 2) then
7953
7954     local itemchar
7955     if item.id == 7 and item.subtype == 2 then
7956       itemchar = item.replace.char
7957     else
7958       itemchar = item.char
7959     end
7960     local chardata = characters[itemchar]
7961     dir = chardata and chardata.d or nil
7962     if not dir then
7963       for nn, et in ipairs(ranges) do
7964         if itemchar < et[1] then
7965           break
7966         elseif itemchar <= et[2] then
7967           dir = et[3]
7968           break
7969         end
7970       end
7971     end
7972     dir = dir or 'l'
7973     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

7974   if new_dir then
7975     attr_dir = 0
7976     for at in node.traverse(item.attr) do
7977       if at.number == Babel.attr_dir then
7978         attr_dir = at.value & 0x3
7979       end
7980     end
7981     if attr_dir == 1 then
7982       strong = 'r'
7983     elseif attr_dir == 2 then
7984       strong = 'al'
7985     else
7986       strong = 'l'
7987     end
7988     strong_lr = (strong == 'l') and 'l' or 'r'
7989     outer = strong_lr
7990     new_dir = false
7991   end
7992
7993   if dir == 'nsm' then dir = strong end -- W1

```

Numbers. The dual <al>/<r> system for R is somewhat cumbersome.

```

7994     dir_real = dir          -- We need dir_real to set strong below
7995     if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no <en> <et> <es> if strong == <al>, only <an>. Therefore, there are not <et en> nor <en et>, W5 can be ignored, and W6 applied:

```

7996     if strong == 'al' then
7997         if dir == 'en' then dir = 'an' end          -- W2
7998         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
7999         strong_lr = 'r'                             -- W3
8000     end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8001     elseif item.id == node.id'dir' and not inmath then
8002         new_dir = true
8003         dir = nil
8004     elseif item.id == node.id'math' then
8005         inmath = (item.subtype == 0)
8006     else
8007         dir = nil          -- Not a char
8008     end

```

Numbers in R mode. A sequence of <en>, <et>, <an>, <es> and <cs> is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only <an> is relevant if <al>.

```

8009     if dir == 'en' or dir == 'an' or dir == 'et' then
8010         if dir ~= 'et' then
8011             type_n = dir
8012         end
8013         first_n = first_n or item
8014         last_n = last_es or item
8015         last_es = nil
8016     elseif dir == 'es' and last_n then -- W3+W6
8017         last_es = item
8018     elseif dir == 'cs' then          -- it's right - do nothing
8019     elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8020         if strong_lr == 'r' and type_n ~= '' then
8021             dir_mark(head, first_n, last_n, 'r')
8022         elseif strong_lr == 'l' and first_d and type_n == 'an' then
8023             dir_mark(head, first_n, last_n, 'r')
8024             dir_mark(head, first_d, last_d, outer)
8025             first_d, last_d = nil, nil
8026         elseif strong_lr == 'l' and type_n ~= '' then
8027             last_d = last_n
8028         end
8029         type_n = ''
8030         first_n, last_n = nil, nil
8031     end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8032     if dir == 'l' or dir == 'r' then
8033         if dir ~= outer then
8034             first_d = first_d or item
8035             last_d = item
8036         elseif first_d and dir ~= strong_lr then
8037             dir_mark(head, first_d, last_d, outer)
8038             first_d, last_d = nil, nil
8039         end
8040     end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it’s clearly <r> and <l>, resp’tly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn’t hurt.

```

8041   if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8042       item.char = characters[item.char] and
8043           characters[item.char].m or item.char
8044   elseif (dir or new_dir) and last_lr ~= item then
8045       local mir = outer .. strong_lr .. (dir or outer)
8046       if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8047           for ch in node.traverse(node.next(last_lr)) do
8048               if ch == item then break end
8049               if ch.id == node.id'glyph' and characters[ch.char] then
8050                   ch.char = characters[ch.char].m or ch.char
8051               end
8052           end
8053       end
8054   end

```

Save some values for the next iteration. If the current node is ‘dir’, open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8055   if dir == 'l' or dir == 'r' then
8056       last_lr = item
8057       strong = dir_real          -- Don't search back - best save now
8058       strong_lr = (strong == 'l') and 'l' or 'r'
8059   elseif new_dir then
8060       last_lr = nil
8061   end
8062 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8063   if last_lr and outer == 'r' then
8064       for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8065           if characters[ch.char] then
8066               ch.char = characters[ch.char].m or ch.char
8067           end
8068       end
8069   end
8070   if first_n then
8071       dir_mark(head, first_n, last_n, outer)
8072   end
8073   if first_d then
8074       dir_mark(head, first_d, last_d, outer)
8075   end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8076   return node.prev(head) or head
8077 end
8078 </basic-r>

```

And here the Lua code for bidi=basic:

```

8079 <(*basic)
8080 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8081
8082 Babel.fontmap = Babel.fontmap or {}
8083 Babel.fontmap[0] = {}          -- l
8084 Babel.fontmap[1] = {}          -- r
8085 Babel.fontmap[2] = {}          -- al/an
8086
8087 -- To cancel mirroring. Also OML, OMS, U?
8088 Babel.symbol_fonts = Babel.symbol_fonts or {}

```

```

8089 Babel.symbol_fonts[font.id('tenln')] = true
8090 Babel.symbol_fonts[font.id('tenlnw')] = true
8091 Babel.symbol_fonts[font.id('tencirc')] = true
8092 Babel.symbol_fonts[font.id('tencircw')] = true
8093
8094 Babel.bidi_enabled = true
8095 Babel.mirroring_enabled = true
8096
8097 require('babel-data-bidi.lua')
8098
8099 local characters = Babel.characters
8100 local ranges = Babel.ranges
8101
8102 local DIR = node.id('dir')
8103 local GLYPH = node.id('glyph')
8104
8105 local function insert_implicit(head, state, outer)
8106   local new_state = state
8107   if state.sim and state.eim and state.sim ~= state.eim then
8108     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8109     local d = node.new(DIR)
8110     d.dir = '+' .. dir
8111     node.insert_before(head, state.sim, d)
8112     local d = node.new(DIR)
8113     d.dir = '-' .. dir
8114     node.insert_after(head, state.eim, d)
8115   end
8116   new_state.sim, new_state.eim = nil, nil
8117   return head, new_state
8118 end
8119
8120 local function insert_numeric(head, state)
8121   local new
8122   local new_state = state
8123   if state.san and state.ean and state.san ~= state.ean then
8124     local d = node.new(DIR)
8125     d.dir = '+TLT'
8126     _, new = node.insert_before(head, state.san, d)
8127     if state.san == state.sim then state.sim = new end
8128     local d = node.new(DIR)
8129     d.dir = '-TLT'
8130     _, new = node.insert_after(head, state.ean, d)
8131     if state.ean == state.eim then state.eim = new end
8132   end
8133   new_state.san, new_state.ean = nil, nil
8134   return head, new_state
8135 end
8136
8137 local function glyph_not_symbol_font(node)
8138   if node.id == GLYPH then
8139     return not Babel.symbol_fonts[node.font]
8140   else
8141     return false
8142   end
8143 end
8144
8145 -- TODO - \hbox with an explicit dir can lead to wrong results
8146 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8147 -- was made to improve the situation, but the problem is the 3-dir
8148 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8149 -- well.
8150
8151 function Babel.bidi(head, ispar, hdir)

```

```

8152 local d    -- d is used mainly for computations in a loop
8153 local prev_d = ''
8154 local new_d = false
8155
8156 local nodes = {}
8157 local outer_first = nil
8158 local inmath = false
8159
8160 local glue_d = nil
8161 local glue_i = nil
8162
8163 local has_en = false
8164 local first_et = nil
8165
8166 local has_hyperlink = false
8167
8168 local ATDIR = Babel.attr_dir
8169 local attr_d, temp
8170 local locale_d
8171
8172 local save_outer
8173 local locale_d = node.get_attribute(head, ATDIR)
8174 if locale_d then
8175     locale_d = locale_d & 0x3
8176     save_outer = (locale_d == 0 and 'l') or
8177                  (locale_d == 1 and 'r') or
8178                  (locale_d == 2 and 'al')
8179 elseif ispar then      -- Or error? Shouldn't happen
8180     -- when the callback is called, we are just _after_ the box,
8181     -- and the textdir is that of the surrounding text
8182     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8183 else                  -- Empty box
8184     save_outer = ('TRT' == hdir) and 'r' or 'l'
8185 end
8186 local outer = save_outer
8187 local last = outer
8188 -- 'al' is only taken into account in the first, current loop
8189 if save_outer == 'al' then save_outer = 'r' end
8190
8191 local fontmap = Babel.fontmap
8192
8193 for item in node.traverse(head) do
8194
8195     -- Mask: DxxxPPTT (Done, Pardir [0-2], Textdir [0-2])
8196     locale_d = node.get_attribute(item, ATDIR)
8197     node.set_attribute(item, ATDIR, 0x80)
8198
8199     -- In what follows, #node is the last (previous) node, because the
8200     -- current one is not added until we start processing the neutrals.
8201     -- three cases: glyph, dir, otherwise
8202     if glyph_not_symbol_font(item)
8203         or (item.id == 7 and item.subtype == 2) then
8204
8205         if inmath then goto nextnode end
8206
8207         if locale_d == 0x80 then goto nextnode end
8208         locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8209
8210         local d_font = nil
8211         local item_r
8212         if item.id == 7 and item.subtype == 2 then
8213             item_r = item.replace    -- automatic discs have just 1 glyph
8214         else

```

```

8215     item_r = item
8216 end
8217
8218 local chardata = characters[item_r.char]
8219 d = chardata and chardata.d or nil
8220 if not d or d == 'nsm' then
8221     for nn, et in ipairs(ranges) do
8222         if item_r.char < et[1] then
8223             break
8224         elseif item_r.char <= et[2] then
8225             if not d then d = et[3]
8226             elseif d == 'nsm' then d_font = et[3]
8227             end
8228             break
8229         end
8230     end
8231 end
8232 d = d or 'l'
8233
8234 -- A short 'pause' in bidi for mapfont
8235 -- %%% TODO. move if fontmap here
8236 d_font = d_font or d
8237 d_font = (d_font == 'l' and 0) or
8238           (d_font == 'nsm' and 0) or
8239           (d_font == 'r' and 1) or
8240           (d_font == 'al' and 2) or
8241           (d_font == 'an' and 2) or nil
8242 if d_font and fontmap and fontmap[d_font][item_r.font] then
8243     item_r.font = fontmap[d_font][item_r.font]
8244 end
8245
8246 if new_d then
8247     table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8248     if inmath then
8249         attr_d = 0
8250     else
8251         attr_d = locale_d & 0x3
8252     end
8253     if attr_d == 1 then
8254         outer_first = 'r'
8255         last = 'r'
8256     elseif attr_d == 2 then
8257         outer_first = 'r'
8258         last = 'al'
8259     else
8260         outer_first = 'l'
8261         last = 'l'
8262     end
8263     outer = last
8264     has_en = false
8265     first_et = nil
8266     new_d = false
8267 end
8268
8269 if glue_d then
8270     if (d == 'l' and 'l' or 'r') ~= glue_d then
8271         table.insert(nodes, {glue_i, 'on', nil})
8272     end
8273     glue_d = nil
8274     glue_i = nil
8275 end
8276
8277 elseif item.id == DIR then

```

```

8278     if inmath then goto nextnode end
8279     d = nil
8280     new_d = true
8281
8282     elseif item.id == node.id'glue' and item.subtype == 13 then
8283         if inmath then goto nextnode end
8284         glue_d = d
8285         glue_i = item
8286         d = nil
8287
8288     elseif item.id == node.id'math' then
8289         if item.subtype == 0 then -- mathon
8290             inmath = true
8291             d = 'on'
8292             table.insert(nodes, {item, 'on', outer_first})
8293             outer_first = nil
8294             goto nextnode
8295         else -- mathoff
8296             inmath = false
8297         end
8298
8299     elseif item.id == 8 and item.subtype == 19 then
8300         has_hyperlink = true
8301
8302     else
8303         d = nil
8304     end
8305
8306     -- AL <= EN/ET/ES      -- W2 + W3 + W6
8307     if last == 'al' and d == 'en' then
8308         d = 'an'           -- W3
8309     elseif last == 'al' and (d == 'et' or d == 'es') then
8310         d = 'on'           -- W6
8311     end
8312
8313     -- EN + CS/ES + EN      -- W4
8314     if d == 'en' and #nodes >= 2 then
8315         if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8316             and nodes[#nodes-1][2] == 'en' then
8317             nodes[#nodes][2] = 'en'
8318         end
8319     end
8320
8321     -- AN + CS + AN         -- W4 too, because uax9 mixes both cases
8322     if d == 'an' and #nodes >= 2 then
8323         if (nodes[#nodes][2] == 'cs')
8324             and nodes[#nodes-1][2] == 'an' then
8325             nodes[#nodes][2] = 'an'
8326         end
8327     end
8328
8329     -- ET/EN                -- W5 + W7->l / W6->on
8330     if d == 'et' then
8331         first_et = first_et or (#nodes + 1)
8332     elseif d == 'en' then
8333         has_en = true
8334         first_et = first_et or (#nodes + 1)
8335     elseif first_et then -- d may be nil here !
8336         if has_en then
8337             if last == 'l' then
8338                 temp = 'l' -- W7
8339             else
8340                 temp = 'en' -- W5

```

```

8341     end
8342   else
8343     temp = 'on'      -- W6
8344   end
8345   for e = first_et, #nodes do
8346     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8347   end
8348   first_et = nil
8349   has_en = false
8350 end
8351
8352 -- Force mathdir in math if ON (currently works as expected only
8353 -- with 'l')
8354
8355 if inmath and d == 'on' then
8356   d = ('TRT' == tex.mathdir) and 'r' or 'l'
8357 end
8358
8359 if d then
8360   if d == 'al' then
8361     d = 'r'
8362     last = 'al'
8363   elseif d == 'l' or d == 'r' then
8364     last = d
8365   end
8366   prev_d = d
8367   table.insert(nodes, {item, d, outer_first})
8368 end
8369
8370 outer_first = nil
8371
8372 ::nextnode::
8373
8374 end -- for each node
8375
8376 -- TODO -- repeated here in case EN/ET is the last node. Find a
8377 -- better way of doing things:
8378 if first_et then      -- dir may be nil here !
8379   if has_en then
8380     if last == 'l' then
8381       temp = 'l'      -- W7
8382     else
8383       temp = 'en'     -- W5
8384     end
8385   else
8386     temp = 'on'       -- W6
8387   end
8388   for e = first_et, #nodes do
8389     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8390   end
8391 end
8392
8393 -- dummy node, to close things
8394 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8395
8396 ----- NEUTRAL -----
8397
8398 outer = save_outer
8399 last = outer
8400
8401 local first_on = nil
8402
8403 for q = 1, #nodes do

```



```

8404     local item
8405
8406     local outer_first = nodes[q][3]
8407     outer = outer_first or outer
8408     last = outer_first or last
8409
8410     local d = nodes[q][2]
8411     if d == 'an' or d == 'en' then d = 'r' end
8412     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8413
8414     if d == 'on' then
8415         first_on = first_on or q
8416     elseif first_on then
8417         if last == d then
8418             temp = d
8419         else
8420             temp = outer
8421         end
8422         for r = first_on, q - 1 do
8423             nodes[r][2] = temp
8424             item = nodes[r][1] -- MIRRORING
8425             if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8426                 and temp == 'r' and characters[item.char] then
8427                 local font_mode = ''
8428                 if item.font > 0 and font.fonts[item.font].properties then
8429                     font_mode = font.fonts[item.font].properties.mode
8430                 end
8431                 if font_mode ~= 'harf' and font_mode ~= 'plug' then
8432                     item.char = characters[item.char].m or item.char
8433                 end
8434             end
8435         end
8436         first_on = nil
8437     end
8438
8439     if d == 'r' or d == 'l' then last = d end
8440 end
8441
8442 ----- IMPLICIT, REORDER -----
8443
8444 outer = save_outer
8445 last = outer
8446
8447 local state = {}
8448 state.has_r = false
8449
8450 for q = 1, #nodes do
8451     local item = nodes[q][1]
8452
8453     outer = nodes[q][3] or outer
8454
8455     local d = nodes[q][2]
8456
8457     if d == 'nsm' then d = last end -- W1
8458     if d == 'en' then d = 'an' end
8459     local isdir = (d == 'r' or d == 'l')
8460
8461     if outer == 'l' and d == 'an' then
8462         state.san = state.san or item
8463         state.ean = item
8464     elseif state.san then
8465         head, state = insert_numeric(head, state)
8466     end
8467 end

```

```

8467     end
8468
8469     if outer == 'l' then
8470         if d == 'an' or d == 'r' then      -- im -> implicit
8471             if d == 'r' then state.has_r = true end
8472             state.sim = state.sim or item
8473             state.eim = item
8474         elseif d == 'l' and state.sim and state.has_r then
8475             head, state = insert_implicit(head, state, outer)
8476         elseif d == 'l' then
8477             state.sim, state.eim, state.has_r = nil, nil, false
8478         end
8479     else
8480         if d == 'an' or d == 'l' then
8481             if nodes[q][3] then -- nil except after an explicit dir
8482                 state.sim = item -- so we move sim 'inside' the group
8483             else
8484                 state.sim = state.sim or item
8485             end
8486             state.eim = item
8487         elseif d == 'r' and state.sim then
8488             head, state = insert_implicit(head, state, outer)
8489         elseif d == 'r' then
8490             state.sim, state.eim = nil, nil
8491         end
8492     end
8493
8494     if isdir then
8495         last = d      -- Don't search back - best save now
8496     elseif d == 'on' and state.san then
8497         state.san = state.san or item
8498         state.ean = item
8499     end
8500
8501 end
8502
8503 head = node.prev(head) or head
8504 % \end{macrocode}
8505 %
8506 % Now direction nodes has been distributed with relation to characters
8507 % and spaces, we need to take into account \TeX-specific elements in
8508 % the node list, to move them at an appropriate place. Firstly, with
8509 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8510 % that the latter are still discardable.
8511 %
8512 % \begin{macrocode}
8513 --- FIXES ---
8514 if has_hyperlink then
8515     local flag, linking = 0, 0
8516     for item in node.traverse(head) do
8517         if item.id == DIR then
8518             if item.dir == '+TRT' or item.dir == '+TLT' then
8519                 flag = flag + 1
8520             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8521                 flag = flag - 1
8522             end
8523         elseif item.id == 8 and item.subtype == 19 then
8524             linking = flag
8525         elseif item.id == 8 and item.subtype == 20 then
8526             if linking > 0 then
8527                 if item.prev.id == DIR and
8528                     (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8529                     d = node.new(DIR)

```

```

8530         d.dir = item.prev.dir
8531         node.remove(head, item.prev)
8532         node.insert_after(head, item, d)
8533     end
8534 end
8535     linking = 0
8536 end
8537 end
8538 end
8539
8540 for item in node.traverse_id(10, head) do
8541     local p = item
8542     local flag = false
8543     while p.prev and p.prev.id == 14 do
8544         flag = true
8545         p = p.prev
8546     end
8547     if flag then
8548         node.insert_before(head, p, node.copy(item))
8549         node.remove(head, item)
8550     end
8551 end
8552
8553 return head
8554 end
8555
8555 function Babel.unset_atdir(head)
8556     local ATDIR = Babel.attr_dir
8557     for item in node.traverse(head) do
8558         node.set_attribute(item, ATDIR, 0x80)
8559     end
8560     return head
8561 end
8562 </basic>

```

11. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%

```

For the meaning of these codes, see the Unicode standard.

12. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```

8563 < *nil >
8564 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8565 \LdfInit{nil}{datenil}

```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```

8566 \ifx\l@nil\@undefined
8567 \newlanguage\l@nil
8568 \@namedef{bbl@hyphendata@the\l@nil}{\{}}% Remove warning
8569 \let\bbl@elt\relax
8570 \edef\bbl@languages{% Add it to the list of languages
8571 \bbl@languages\bbl@elt{nil}{\the\l@nil}{\{}}
8572 \fi

```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```
8573 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}
```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

\captionnil

\datenil

```

8574 \let\captionnil\@empty
8575 \let\datenil\@empty

```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```

8576 \def\bbl@inidata@nil{%
8577 \bbl@elt{identification}{tag.ini}{und}%
8578 \bbl@elt{identification}{load.level}{0}%
8579 \bbl@elt{identification}{name.english}{nil}}
8580 \@namedef{bbl@tbc@nil}{und}
8581 \@namedef{bbl@lbc@nil}{und}
8582 \@namedef{bbl@casing@nil}{und}
8583 \@namedef{bbl@lotf@nil}{dflt}
8584 \@namedef{bbl@elname@nil}{nil}
8585 \@namedef{bbl@lname@nil}{nil}
8586 \@namedef{bbl@esname@nil}{Latin}
8587 \@namedef{bbl@sname@nil}{Latin}
8588 \@namedef{bbl@sbc@nil}{Latn}
8589 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8590 \ldf@finish{nil}
8591 \</nil>

```

13. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It’s based on the little library `calendar.js`, by John Walker, in the public domain.

```

8592 <<*Compute Julian day>> ≡
8593 \def\bbl@fpmmod#1#2{(#1-#2*floor((#1)/(#2)))}
8594 \def\bbl@cs@gregleap#1{%
8595 (\bbl@fpmmod{#1}{4} == 0) &&
8596 (!((\bbl@fpmmod{#1}{100} == 0) && (\bbl@fpmmod{#1}{400} != 0)))}
8597 \def\bbl@cs@jd#1#2#3{% year, month, day
8598 \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8599 floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8600 floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8601 ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3} }
8602 <</Compute Julian day>>

```

13.1. Islamic

The code for the Civil calendar is based on it, too.

```
8603 (*ca-islamic)
8604 <@Compute Julian day@>
8605 % == islamic (default)
8606 % Not yet implemented
8607 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{}
```

The Civil calendar.

```
8608 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8609 ((#3 + ceil(29.5 * (#2 - 1)) +
8610 (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8611 1948439.5) - 1) }
8612 \@namedef\bbl@ca@islamic-civil++{\bbl@ca@islamicvl@x{+2}}
8613 \@namedef\bbl@ca@islamic-civil+{\bbl@ca@islamicvl@x{+1}}
8614 \@namedef\bbl@ca@islamic-civil{\bbl@ca@islamicvl@x{}}
8615 \@namedef\bbl@ca@islamic-civil-{\bbl@ca@islamicvl@x{-1}}
8616 \@namedef\bbl@ca@islamic-civil--{\bbl@ca@islamicvl@x{-2}}
8617 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8618 \edef\bbl@tempa{%
8619 \fpeval{ floor(\bbl@cs@jld{#2}{#3}{#4})+0.5 #1}}%
8620 \edef#5{%
8621 \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8622 \edef#6{\fpeval{
8623 min(12,ceil(((\bbl@tempa-(29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8624 \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}
```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```
8625 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8626 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8627 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8628 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8629 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8630 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8631 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8632 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8633 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8634 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8635 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8636 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8637 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8638 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8639 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8640 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8641 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8642 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8643 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8644 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8645 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8646 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8647 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8648 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8649 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8650 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8651 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8652 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8653 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8654 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8655 65401,65431,65460,65490,65520}
8656 \@namedef\bbl@ca@islamic-umalqura+{\bbl@ca@islamcuqr@x{+1}}
```

```

8657 \@namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{}}
8658 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8659 \def\bbl@ca@islamcuqr@x#1#2-#3-#4@@#5#6#7{%
8660   \ifnum#2>2014 \ifnum#2<2038
8661     \bbl@afterfi\expandafter\@gobble
8662   \fi\fi
8663   {\bbl@error{year-out-range}{2014-2038}}{}}%
8664 \edef\bbl@tempd{\fpeval{ % (Julian) day
8665   \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8666 \count@\@ne
8667 \bbl@foreach\bbl@cs@umalqura@data{%
8668   \advance\count@\@ne
8669   \ifnum##1>\bbl@tempd\else
8670     \edef\bbl@tempe{\the\count@}%
8671     \edef\bbl@tempb{##1}%
8672     \fi}%
8673 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8674 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1) / 12) }}% annus
8675 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8676 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8677 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8678 \bbl@add\bbl@precalendar{%
8679   \bbl@replace\bbl@ld@calendar{-civil}}}%
8680 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8681 \bbl@replace\bbl@ld@calendar{+}}}%
8682 \bbl@replace\bbl@ld@calendar{-}}}%
8683 </ca-islamic>

```

13.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaption by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in hebcal.sty

```

8684 < *ca-hebrew>
8685 \newcount\bbl@cntcommon
8686 \def\bbl@remainder#1#2#3{%
8687   #3=#1\relax
8688   \divide #3 by #2\relax
8689   \multiply #3 by -#2\relax
8690   \advance #3 by #1\relax}%
8691 \newif\ifbbl@divisible
8692 \def\bbl@checkifdivisible#1#2{%
8693   {\countdef\tmp=0
8694     \bbl@remainder{#1}{#2}{\tmp}%
8695     \ifnum \tmp=0
8696       \global\bbl@divisibletrue
8697     \else
8698       \global\bbl@divisiblefalse
8699     \fi}}
8700 \newif\ifbbl@gregleap
8701 \def\bbl@ifgregleap#1{%
8702   \bbl@checkifdivisible{#1}{4}%
8703   \ifbbl@divisible
8704     \bbl@checkifdivisible{#1}{100}%
8705     \ifbbl@divisible
8706       \bbl@checkifdivisible{#1}{400}%
8707       \ifbbl@divisible
8708         \bbl@gregleaptrue
8709       \else
8710         \bbl@gregleapfalse
8711       \fi
8712     \else
8713       \bbl@gregleaptrue

```

```

8714     \fi
8715 \else
8716     \bbl@gregleapfalse
8717 \fi
8718 \ifbbl@gregleap}
8719 \def\bbl@gregdayspriormonths#1#2#3{%
8720     {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8721         181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8722     \bbl@ifgregleap{#2}%
8723     \ifnum #1 > 2
8724         \advance #3 by 1
8725     \fi
8726 \fi
8727 \global\bbl@cntcommon=#3}%
8728 #3=\bbl@cntcommon}
8729 \def\bbl@gregdaysprioryears#1#2{%
8730     {\countdef\tmpc=4
8731     \countdef\tmpb=2
8732     \tmpb=#1\relax
8733     \advance \tmpb by -1
8734     \tmpc=\tmpb
8735     \multiply \tmpc by 365
8736     #2=\tmpc
8737     \tmpc=\tmpb
8738     \divide \tmpc by 4
8739     \advance #2 by \tmpc
8740     \tmpc=\tmpb
8741     \divide \tmpc by 100
8742     \advance #2 by -\tmpc
8743     \tmpc=\tmpb
8744     \divide \tmpc by 400
8745     \advance #2 by \tmpc
8746     \global\bbl@cntcommon=#2\relax}%
8747 #2=\bbl@cntcommon}
8748 \def\bbl@absfromgreg#1#2#3#4{%
8749     {\countdef\tmpd=0
8750     #4=#1\relax
8751     \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8752     \advance #4 by \tmpd
8753     \bbl@gregdaysprioryears{#3}{\tmpd}%
8754     \advance #4 by \tmpd
8755     \global\bbl@cntcommon=#4\relax}%
8756 #4=\bbl@cntcommon}
8757 \newif\ifbbl@hebrleap
8758 \def\bbl@checkleaphebyear#1{%
8759     {\countdef\tmpa=0
8760     \countdef\tmpb=1
8761     \tmpa=#1\relax
8762     \multiply \tmpa by 7
8763     \advance \tmpa by 1
8764     \bbl@remainder{\tmpa}{19}{\tmpb}%
8765     \ifnum \tmpb < 7
8766         \global\bbl@hebrleaptrue
8767     \else
8768         \global\bbl@hebrleapfalse
8769     \fi}}
8770 \def\bbl@hebrlapsedmonths#1#2{%
8771     {\countdef\tmpa=0
8772     \countdef\tmpb=1
8773     \countdef\tmpc=2
8774     \tmpa=#1\relax
8775     \advance \tmpa by -1
8776     #2=\tmpa

```

```

8777 \divide #2 by 19
8778 \multiply #2 by 235
8779 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8780 \tmpc=\tmpb
8781 \multiply \tmpb by 12
8782 \advance #2 by \tmpb
8783 \multiply \tmpc by 7
8784 \advance \tmpc by 1
8785 \divide \tmpc by 19
8786 \advance #2 by \tmpc
8787 \global\bbl@cntcommon=#2}%
8788 #2=\bbl@cntcommon}
8789 \def\bbl@hebreleapseddays#1#2{%
8790 {\countdef\tmpa=0
8791 \countdef\tmpb=1
8792 \countdef\tmpc=2
8793 \bbl@hebreleapsedmonths{#1}{#2}%
8794 \tmpa=#2\relax
8795 \multiply \tmpa by 13753
8796 \advance \tmpa by 5604
8797 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8798 \divide \tmpa by 25920
8799 \multiply #2 by 29
8800 \advance #2 by 1
8801 \advance #2 by \tmpa
8802 \bbl@remainder{#2}{7}{\tmpa}%
8803 \ifnum \tmpc < 19440
8804 \ifnum \tmpc < 9924
8805 \else
8806 \ifnum \tmpa=2
8807 \bbl@checkleaphebrewyear{#1}% of a common year
8808 \ifbbl@hebrleap
8809 \else
8810 \advance #2 by 1
8811 \fi
8812 \fi
8813 \fi
8814 \ifnum \tmpc < 16789
8815 \else
8816 \ifnum \tmpa=1
8817 \advance #1 by -1
8818 \bbl@checkleaphebrewyear{#1}% at the end of leap year
8819 \ifbbl@hebrleap
8820 \advance #2 by 1
8821 \fi
8822 \fi
8823 \fi
8824 \else
8825 \advance #2 by 1
8826 \fi
8827 \bbl@remainder{#2}{7}{\tmpa}%
8828 \ifnum \tmpa=0
8829 \advance #2 by 1
8830 \else
8831 \ifnum \tmpa=3
8832 \advance #2 by 1
8833 \else
8834 \ifnum \tmpa=5
8835 \advance #2 by 1
8836 \fi
8837 \fi
8838 \fi
8839 \global\bbl@cntcommon=#2\relax}%

```



```

8840 #2=\bbl@cntcommon}
8841 \def\bbl@daysinhebrewyear#1#2{%
8842 {\countdef\tmpe=12
8843 \bbl@hebreleapseddays{#1}{\tmpe}%
8844 \advance #1 by 1
8845 \bbl@hebreleapseddays{#1}{#2}%
8846 \advance #2 by -\tmpe
8847 \global\bbl@cntcommon=#2}%
8848 #2=\bbl@cntcommon}
8849 \def\bbl@hebrdayspriormonths#1#2#3{%
8850 {\countdef\tmpf= 14
8851 #3=\ifcase #1
8852 0 \or
8853 0 \or
8854 30 \or
8855 59 \or
8856 89 \or
8857 118 \or
8858 148 \or
8859 148 \or
8860 177 \or
8861 207 \or
8862 236 \or
8863 266 \or
8864 295 \or
8865 325 \or
8866 400
8867 \fi
8868 \bbl@checkleaphebrewyear{#2}%
8869 \ifbbl@hebrleap
8870 \ifnum #1 > 6
8871 \advance #3 by 30
8872 \fi
8873 \fi
8874 \bbl@daysinhebrewyear{#2}{\tmpf}%
8875 \ifnum #1 > 3
8876 \ifnum \tmpf=353
8877 \advance #3 by -1
8878 \fi
8879 \ifnum \tmpf=383
8880 \advance #3 by -1
8881 \fi
8882 \fi
8883 \ifnum #1 > 2
8884 \ifnum \tmpf=355
8885 \advance #3 by 1
8886 \fi
8887 \ifnum \tmpf=385
8888 \advance #3 by 1
8889 \fi
8890 \fi
8891 \global\bbl@cntcommon=#3\relax}%
8892 #3=\bbl@cntcommon}
8893 \def\bbl@absfromhebr#1#2#3#4{%
8894 {#4=#1\relax
8895 \bbl@hebrdayspriormonths{#2}{#3}{#1}%
8896 \advance #4 by #1\relax
8897 \bbl@hebreleapseddays{#3}{#1}%
8898 \advance #4 by #1\relax
8899 \advance #4 by -1373429
8900 \global\bbl@cntcommon=#4\relax}%
8901 #4=\bbl@cntcommon}
8902 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%

```

```

8903 {\countdef\tmpx= 17
8904 \countdef\tmpy= 18
8905 \countdef\tmpz= 19
8906 #6=#3\relax
8907 \global\advance #6 by 3761
8908 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
8909 \tmpz=1 \tmpy=1
8910 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8911 \ifnum \tmpx > #4\relax
8912 \global\advance #6 by -1
8913 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8914 \fi
8915 \advance #4 by -\tmpx
8916 \advance #4 by 1
8917 #5=#4\relax
8918 \divide #5 by 30
8919 \loop
8920 \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
8921 \ifnum \tmpx < #4\relax
8922 \advance #5 by 1
8923 \tmpy=\tmpx
8924 \repeat
8925 \global\advance #5 by -1
8926 \global\advance #4 by -\tmpy}}
8927 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebryear
8928 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
8929 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
8930 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
8931 \bbl@hebrfromgreg
8932 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
8933 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebryear}%
8934 \edef#4{\the\bbl@hebryear}%
8935 \edef#5{\the\bbl@hebrmonth}%
8936 \edef#6{\the\bbl@hebrday}}
8937 </ca-hebrew>

```

13.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

8938 < *ca-persian >
8939 <@Compute Julian day@>
8940 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
8941 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
8942 \def\bbl@ca@persian#1-#2-#3\@#4#5#6{%
8943 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
8944 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
8945 \bbl@afterfi\expandafter\gobble
8946 \fi\fi
8947 {\bbl@error{year-out-range}{2013-2050}{}}}%
8948 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8949 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8950 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
8951 \def\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
8952 \ifnum\bbl@tempc<\bbl@tempb
8953 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
8954 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8955 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8956 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
8957 \fi

```

```

8958 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
8959 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
8960 \edef#5{\fpeval{% set Jalali month
8961   (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
8962 \edef#6{\fpeval{% set Jalali day
8963   (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : (((#5 - 1) * 30) + 6))}}}}
8964 </ca-persian>

```

13.4. Coptic and Ethiopic

Adapted from jquery.calendars.package-1.1.4, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

8965 <*ca-coptic>
8966 <@Compute Julian day@>
8967 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
8968   \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8969   \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
8970   \edef#4{\fpeval{%
8971     floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8972   \edef\bbl@tempc{\fpeval{%
8973     \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
8974   \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8975   \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}}}
8976 </ca-coptic>
8977 <*ca-ethiopic>
8978 <@Compute Julian day@>
8979 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%
8980   \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8981   \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
8982   \edef#4{\fpeval{%
8983     floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8984   \edef\bbl@tempc{\fpeval{%
8985     \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
8986   \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8987   \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}}}
8988 </ca-ethiopic>

```

13.5. Julian

Based on [ReinDersh].

```

8989 <*ca-julian>
8990 <@Compute Julian day@>
8991 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
8992   \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
8993   \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
8994   \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
8995   \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
8996   \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
8997   \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
8998   \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
8999   \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}}}
9000 </ca-julian>

```

The Indian National Calendar, Saka era.

```

9001 <*ca-indian>
9002 <@Compute Julian day@>
9003 \def\bbl@sakachaitra#1{%
9004   \bbl@cs@jd{#1}{3}%
9005   \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100}!=0)
9006   ||\bbl@fpmmod{#1}{400}==0) ? 21 : 22 }}
9007 \def\bbl@ca@indian#1-#2-#3\@@#4#5#6{%
9008   \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%

```

```

9009 \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9010 \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9011 \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9012 \edef\bbl@tempe{\fpeval{ ((\bbl@fmod{\bbl@tempc}{4})==0 &&
9013 \bbl@fmod{\bbl@tempc}{100}!=0) || \bbl@fmod{\bbl@tempc}{400}==0)?31:30}}%
9014 \edef#4{\fpeval{\bbl@tempc-78}}%
9015 \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9016 (\bbl@tempd-\bbl@tempe < 155 ?
9017 2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9018 7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0))}}%
9019 \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9020 (\bbl@tempd-\bbl@tempe < 155 ?
9021 \bbl@fmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9022 \bbl@fmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}}
9023 </ca-indian>
9024 %

```

13.6. Buddhist

That's very simple.

```

9025 <*ca-buddhist>
9026 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9027 \edef#4{\number\numexpr#1+543\relax}%
9028 \edef#5{#2}%
9029 \edef#6{#3}}
9030 </ca-buddhist>
9031 %
9032 % \subsection{Chinese}
9033 %
9034 % Brute force, with the Julian day of first day of each month. The
9035 % table has been computed with the help of \textsf{python-lunardate} by
9036 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9037 % is 2015-2044.
9038 %
9039 % \begin{macrocode}
9040 <*ca-chinese>
9041 \ExplSyntaxOn
9042 <@Compute Julian day@>
9043 \def\bbl@ca@chinese#1-#2-#3\@@#4#5#6{%
9044 \edef\bbl@tempd{\fpeval{%
9045 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9046 \count@\z@
9047 \@tempcnta=2015
9048 \bbl@foreach\bbl@cs@chinese@data{%
9049 \ifnum##1>\bbl@tempd\else
9050 \advance\count@\@ne
9051 \ifnum\count@>12
9052 \count@\@ne
9053 \advance\@tempcnta\@ne\fi
9054 \bbl@xin@{,##1,}{,\bbl@cs@chinese@leap,}%
9055 \ifin@
9056 \advance\count@\m@ne
9057 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9058 \else
9059 \edef\bbl@tempe{\the\count@}%
9060 \fi
9061 \edef\bbl@tempb{##1}%
9062 \fi}%
9063 \edef#4{\the\@tempcnta}%
9064 \edef#5{\bbl@tempe}%
9065 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9066 \def\bbl@cs@chinese@leap{%
9067 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}

```

```

9068 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9069 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9070 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9071 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9072 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9073 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9074 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9075 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9076 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9077 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9078 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9079 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9080 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9081 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9082 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9083 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9084 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9085 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9086 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9087 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9088 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9089 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9090 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9091 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9092 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9093 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9094 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9095 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9096 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9097 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9098 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9099 10896,10926,10956,10986,11015,11045,11074,11103}
9100 \ExplSyntaxOff
9101 </ca-chinese>

```

14. Support for Plain T_EX (plain.def)

14.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `locallyhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `blplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9102 <*\bplain | blplain>
9103 \catcode`\{=1 % left brace is begin-group character
9104 \catcode`\}=2 % right brace is end-group character
9105 \catcode`\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```

9106 \openin 0 hyphen.cfg

```

```

9107 \ifeof0
9108 \else
9109 \let\input

```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that's done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```

9110 \def\input #1 {%
9111 \let\input\a
9112 \a hyphen.cfg
9113 \let\a\undefined
9114 }
9115 \fi
9116 </bplain | blplain>

```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```

9117 <bplain>\a plain.tex
9118 <blplain>\a lplain.tex

```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```

9119 <bplain>\def\fmtname{babel-plain}
9120 <blplain>\def\fmtname{babel-lplain}

```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2. Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX} 2_{\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```

9121 <<*Emulate LaTeX>> ≡
9122 \def\@empty{}
9123 \def\loadlocalcfg#1{%
9124 \openin0#1.cfg
9125 \ifeof0
9126 \closein0
9127 \else
9128 \closein0
9129 {\immediate\writel6{*****}%
9130 \immediate\writel6{* Local config file #1.cfg used}%
9131 \immediate\writel6{*}%
9132 }
9133 \input #1.cfg\relax
9134 \fi
9135 \@endofldf}

```

14.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```

9136 \long\def\@firstofone#1{#1}
9137 \long\def\@firstoftwo#1#2{#1}
9138 \long\def\@secondoftwo#1#2{#2}
9139 \def\@nnil{\@nil}
9140 \def\@gobbletwo#1#2{}
9141 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9142 \def\@star@or@long#1{%
9143 \ifstar
9144 {\let\l@ngrel@x\relax#1}%
9145 {\let\l@ngrel@x\long#1}}

```

```

9146 \let\l@ngrel@x\relax
9147 \def\@car#1#2\@nil{#1}
9148 \def\@cdr#1#2\@nil{#2}
9149 \let\@typeset@protect\relax
9150 \let\protected@edef\edef
9151 \long\def\@gobble#1{}
9152 \edef\@backslashchar{\expandafter\@gobble\string\}
9153 \def\strip@prefix#1>{}
9154 \def\g@addto@macro#1#2{%
9155     \toks@\expandafter{#1#2}%
9156     \xdef#1{\the\toks@}}
9157 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9158 \def\@nameuse#1{\csname #1\endcsname}
9159 \def\@ifundefined#1{%
9160     \expandafter\ifx\csname#1\endcsname\relax
9161     \expandafter\@firstoftwo
9162     \else
9163     \expandafter\@secondoftwo
9164     \fi}
9165 \def\@expandtwoargs#1#2#3{%
9166     \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9167 \def\zap@space#1 #2{%
9168     #1%
9169     \ifx#2\@empty\else\expandafter\zap@space\fi
9170     #2}
9171 \let\bbl@trace\@gobble
9172 \def\bbl@error#1{% Implicit #2#3#4
9173     \begingroup
9174         \catcode`\=0 \catcode`\==12 \catcode`\`=12
9175         \catcode`\^M=5 \catcode`\%=14
9176         \input errbabel.def
9177     \endgroup
9178     \bbl@error{#1}}
9179 \def\bbl@warning#1{%
9180     \begingroup
9181         \newlinechar=`^^J
9182         \def\{^^J(babel) }%
9183         \message{\{#1}%
9184     \endgroup}
9185 \let\bbl@infowarn\bbl@warning
9186 \def\bbl@info#1{%
9187     \begingroup
9188         \newlinechar=`^^J
9189         \def\{^^J}%
9190         \wlog{#1}%
9191     \endgroup}

```

$\LaTeX 2\epsilon$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9192 \ifx\@preamblecmds\@undefined
9193     \def\@preamblecmds{}
9194 \fi
9195 \def\@onlypreamble#1{%
9196     \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9197         \@preamblecmds\do#1}}
9198 \@onlypreamble\@onlypreamble

```

Mimic $\LaTeX$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9199 \def\begindocument{%
9200     \@begindocumenthook
9201     \global\let\@begindocumenthook\@undefined
9202     \def\do##1{\global\let##1\@undefined}%
9203     \@preamblecmds
9204     \global\let\do\noexpand}

```

```

9205 \ifx\@begindocumenthook\undefined
9206   \def\@begindocumenthook{}
9207 \fi
9208 \onlypreamble\@begindocumenthook
9209 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\LaTeX$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endofldf`.

```

9210 \def\AtEndOfPackage#1{\g@addto@macro\@endofldf{#1}}
9211 \onlypreamble\AtEndOfPackage
9212 \def\@endofldf{}
9213 \onlypreamble\@endofldf
9214 \let\bbl@afterlang\empty
9215 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9216 \catcode`\&=\z@
9217 \ifx\if@files\undefined
9218   \expandafter\let\csname if@files\expandafter\endcsname
9219     \csname iffalse\endcsname
9220 \fi
9221 \catcode`\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9222 \def\newcommand{\@star@or@long\new@command}
9223 \def\new@command#1{%
9224   \@testopt{\@newcommand#1}0}
9225 \def\@newcommand#1[#2]{%
9226   \@ifnextchar [{\@xargdef#1[#2]}%
9227     {\@argdef#1[#2]}}
9228 \long\def\@argdef#1[#2]#3{%
9229   \@yargdef#1\@ne{#2}{#3}}
9230 \long\def\@xargdef#1[#2][#3]#4{%
9231   \expandafter\def\expandafter#1\expandafter{%
9232     \expandafter\@protected@testopt\expandafter #1%
9233     \csname\string#1\expandafter\endcsname{#3}}%
9234   \expandafter\@yargdef \csname\string#1\endcsname
9235   \tw@{#2}{#4}}
9236 \long\def\@yargdef#1#2#3{%
9237   \@tempcnta#3\relax
9238   \advance \@tempcnta \@ne
9239   \let\@hash\relax
9240   \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9241   \@tempcntb #2%
9242   \@whilenum \@tempcntb <\@tempcnta
9243   \do{%
9244     \edef\reserved@a{\reserved@a\@hash@the\@tempcntb}%
9245     \advance\@tempcntb \@ne}%
9246   \let\@hash##%
9247   \l@ngrel@x\expandafter\def\expandafter#1\reserved@a}
9248 \def\providecommand{\@star@or@long\provide@command}
9249 \def\provide@command#1{%
9250   \begingroup
9251     \escapechar\m@ne\xdef\@gtempa{\string#1}%
9252   \endgroup
9253   \expandafter\ifundefined\@gtempa
9254     {\def\reserved@a{\newcommand#1}}%
9255     {\let\reserved@a\relax
9256       \def\reserved@a{\newcommand\reserved@a}}%
9257   \reserved@a}%
9258 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}

```



```

9259 \def\declare@robustcommand#1{%
9260   \edef\reserved@a{\string#1}%
9261   \def\reserved@b{#1}%
9262   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9263   \edef#1{%
9264     \ifx\reserved@a\reserved@b
9265       \noexpand\x@protect
9266       \noexpand#1%
9267     \fi
9268     \noexpand\protect
9269     \expandafter\noexpand\csname
9270       \expandafter\@gobble\string#1 \endcsname
9271   }%
9272   \expandafter\new@command\csname
9273     \expandafter\@gobble\string#1 \endcsname
9274 }
9275 \def\x@protect#1{%
9276   \ifx\protect\@typeset@protect\else
9277     \@x@protect#1%
9278   \fi
9279 }
9280 \catcode`\&=\z@ % Trick to hide conditionals
9281 \def\@x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9282 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9283 \catcode`\&=4
9284 \ifx\in@\@undefined
9285   \def\in@#1#2{%
9286     \def\in@@##1##2##3\in@@{%
9287       \ifx\in@@##2\in@false\else\in@true\fi}%
9288     \in@@##1\in@\in@@}
9289 \else
9290   \let\bbl@tempa\@empty
9291 \fi
9292 \bbl@tempa

```

$\text{\LaTeX}$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (`activegrave` and `activeacute`). For plain $\text{\TeX}$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9293 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\text{\LaTeX}$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\text{\TeX}$ but we need the macro to be defined as a no-op.

```

9294 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\text{\LaTeX} 2_{\epsilon}$ versions; just enough to make things work in plain $\text{\TeX}$ environments.

```

9295 \ifx\@tempcnta\@undefined
9296   \csname newcount\endcsname\@tempcnta\relax
9297 \fi
9298 \ifx\@tempcntb\@undefined
9299   \csname newcount\endcsname\@tempcntb\relax
9300 \fi

```

To prevent wasting two counters in $\text{\LaTeX}$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9301 \ifx\bye\@undefined
9302   \advance\count10 by -2\relax

```

```

9303 \fi
9304 \ifx\@ifnextchar\@undefined
9305   \def\@ifnextchar#1#2#3{%
9306     \let\reserved@d=#1%
9307     \def\reserved@a{#2}\def\reserved@b{#3}%
9308     \futurelet\@let@token\@ifnch}
9309   \def\@ifnch{%
9310     \ifx\@let@token\@sptoken
9311       \let\reserved@c\@xifnch
9312     \else
9313       \ifx\@let@token\reserved@d
9314         \let\reserved@c\reserved@a
9315       \else
9316         \let\reserved@c\reserved@b
9317       \fi
9318     \fi
9319     \reserved@c}
9320   \def\:{\let\@sptoken= } \: % this makes \@sptoken a space token
9321   \def\:{\@xifnch} \expandafter\def\:{\futurelet\@let@token\@ifnch}
9322 \fi
9323 \def\@testopt#1#2{%
9324   \@ifnextchar[#{1}{#1[#2]}}
9325 \def\@protected@testopt#1{%
9326   \ifx\protect\@typeset@protect
9327     \expandafter\@testopt
9328   \else
9329     \@x@protect#1%
9330   \fi}
9331 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9332   #2\relax}\fi}
9333 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9334   \else\expandafter\@gobble\fi{#1}}

```

14.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9335 \def\DeclareTextCommand{%
9336   \@dec@text@cmd\providecommand
9337 }
9338 \def\ProvideTextCommand{%
9339   \@dec@text@cmd\providecommand
9340 }
9341 \def\DeclareTextSymbol#1#2#3{%
9342   \@dec@text@cmd\chardef#1{#2}#3\relax
9343 }
9344 \def\@dec@text@cmd#1#2#3{%
9345   \expandafter\def\expandafter#2%
9346     \expandafter{%
9347       \csname#3-cmd\expandafter\endcsname
9348       \expandafter#2%
9349       \csname#3\string#2\endcsname
9350     }%
9351   % \let\@ifdefinable\@rc@ifdefinable
9352   \expandafter#1\csname#3\string#2\endcsname
9353 }
9354 \def\@current@cmd#1{%
9355   \ifx\protect\@typeset@protect\else
9356     \noexpand#1\expandafter\@gobble
9357   \fi
9358 }
9359 \def\@changed@cmd#1#2{%
9360   \ifx\protect\@typeset@protect
9361     \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax

```

```

9362         \expandafter\ifx\csname ?\string#1\endcsname\relax
9363         \expandafter\def\csname ?\string#1\endcsname{%
9364             \@changed@x@err{#1}%
9365         }%
9366     \fi
9367     \global\expandafter\let
9368         \csname\cf@encoding \string#1\expandafter\endcsname
9369         \csname ?\string#1\endcsname
9370 \fi
9371 \csname\cf@encoding\string#1%
9372 \expandafter\endcsname
9373 \else
9374     \noexpand#1%
9375 \fi
9376 }
9377 \def\@changed@x@err#1{%
9378     \errhelp{Your command will be ignored, type <return> to proceed}%
9379     \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9380 \def\DeclareTextCommandDefault#1{%
9381     \DeclareTextCommand#1?%
9382 }
9383 \def\ProvideTextCommandDefault#1{%
9384     \ProvideTextCommand#1?%
9385 }
9386 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9387 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9388 \def\DeclareTextAccent#1#2#3{%
9389     \DeclareTextCommand#1{#2}[1]{\accent#3 ##1}
9390 }
9391 \def\DeclareTextCompositeCommand#1#2#3#4{%
9392     \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9393     \edef\reserved@b{\string##1}%
9394     \edef\reserved@c{%
9395         \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9396     \ifx\reserved@b\reserved@c
9397         \expandafter\expandafter\expandafter\ifx
9398             \expandafter\@car\reserved@a\relax\relax\@nil
9399             \@text@composite
9400         \else
9401             \edef\reserved@b##1{%
9402                 \def\expandafter\noexpand
9403                     \csname#2\string#1\endcsname###1{%
9404                         \noexpand\@text@composite
9405                         \expandafter\noexpand\csname#2\string#1\endcsname
9406                         ###1\noexpand\@empty\noexpand\@text@composite
9407                         {##1}%
9408                     }%
9409                 }%
9410             \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9411         \fi
9412         \expandafter\def\csname\expandafter\string\csname
9413             #2\endcsname\string#1-\string#3\endcsname{#4}
9414     \else
9415         \errhelp{Your command will be ignored, type <return> to proceed}%
9416         \errmessage{\string\DeclareTextCompositeCommand\space used on
9417             inappropriate command \protect#1}
9418     \fi
9419 }
9420 \def\@text@composite#1#2#3\@text@composite{%
9421     \expandafter\@text@composite@x
9422         \csname\string#1-\string#2\endcsname
9423 }
9424 \def\@text@composite@x#1#2{%

```

```

9425 \ifx#1\relax
9426     #2%
9427 \else
9428     #1%
9429 \fi
9430 }
9431 %
9432 \def\@strip@args#1:#2-#3\@strip@args{#2}
9433 \def\DeclareTextComposite#1#2#3#4{%
9434     \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9435     \bgroup
9436         \lccode\@=#4%
9437         \lowercase{%
9438     \egroup
9439     \reserved@a @%
9440 }%
9441 }
9442 %
9443 \def\UseTextSymbol#1#2{#2}
9444 \def\UseTextAccent#1#2#3{}
9445 \def\@use@text@encoding#1{}
9446 \def\DeclareTextSymbolDefault#1#2{%
9447     \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9448 }
9449 \def\DeclareTextAccentDefault#1#2{%
9450     \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9451 }
9452 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX 2}\epsilon$ method for accents for those that are known to be made active in *some* language definition file.

```

9453 \DeclareTextAccent{"}{OT1}{127}
9454 \DeclareTextAccent{'}{OT1}{19}
9455 \DeclareTextAccent{^}{OT1}{94}
9456 \DeclareTextAccent{\`}{OT1}{18}
9457 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for `PLAIN  $\text{\TeX}$` .

```

9458 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9459 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9460 \DeclareTextSymbol{\textquoteleft}{OT1}{``}
9461 \DeclareTextSymbol{\textquoteright}{OT1}{``'}
9462 \DeclareTextSymbol{\i}{OT1}{16}
9463 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because `plain  $\text{\TeX}$` doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9464 \ifx\scriptsize\undefined
9465     \let\scriptsize\sevenrm
9466 \fi

```

And a few more “dummy” definitions.

```

9467 \def\language{english}%
9468 \let\bbl@opt@shorthands\@nnil
9469 \def\bbl@ifshorthand#1#2#3{#2}%
9470 \let\bbl@language@opts\@empty
9471 \let\bbl@provide@locale\relax
9472 \ifx\babeloptionstrings\undefined
9473     \let\bbl@opt@strings\@nnil
9474 \else
9475     \let\bbl@opt@strings\babeloptionstrings
9476 \fi
9477 \def\BabelStringsDefault{generic}
9478 \def\bbl@tempa{normal}

```

```

9479 \ifx\babeloptionmath\bbl@tempa
9480 \def\bbl@mathnormal{\noexpand\textormath}
9481 \fi
9482 \def\AfterBabelLanguage#1#2{}
9483 \ifx\BabelModifiers\undefined\let\BabelModifiers\relax\fi
9484 \let\bbl@afterlang\relax
9485 \def\bbl@opt@safe{BR}
9486 \ifx\@uclclist\undefined\let\@uclclist\@empty\fi
9487 \ifx\bbl@trace\undefined\def\bbl@trace#1{}\fi
9488 \expandafter\newif\csname ifbbl@single\endcsname
9489 \chardef\bbl@bidimode\z@
9490 <</Emulate LaTeX>>

```

A proxy file:

```

9491 <*\plain>
9492 \input babel.def
9493 </\plain>

```

15. Acknowledgements

In the initial stages of the development of babel, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\text{\TeX}$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\text{\TeX}$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\text{\LaTeX}$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\text{\TeX}$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\text{\TeX}$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\text{\LaTeX}$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\text{\LaTeX}$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer, een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).